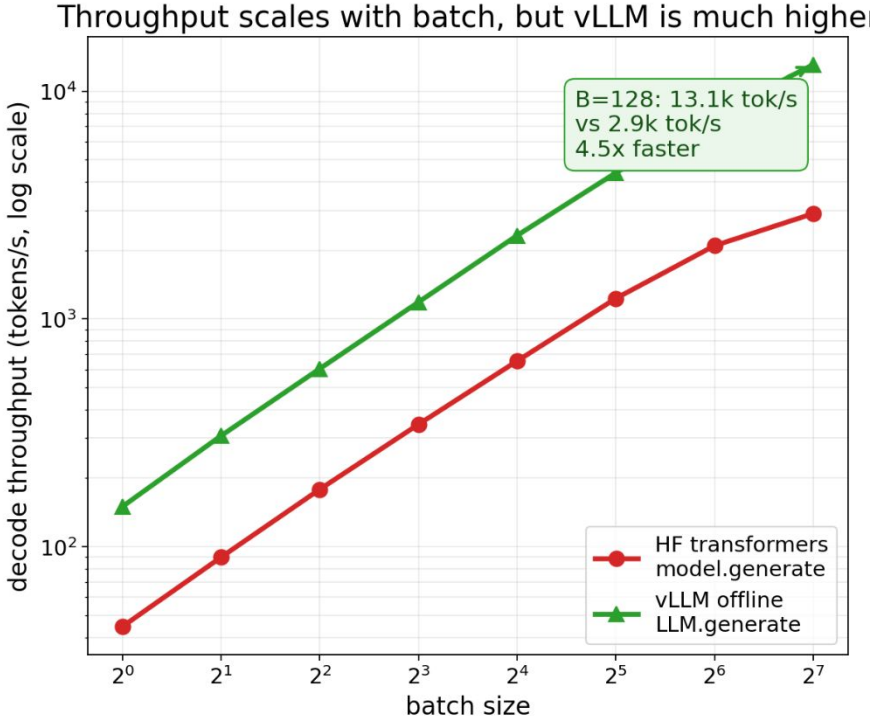
Lecture 2, Inference Optimizations

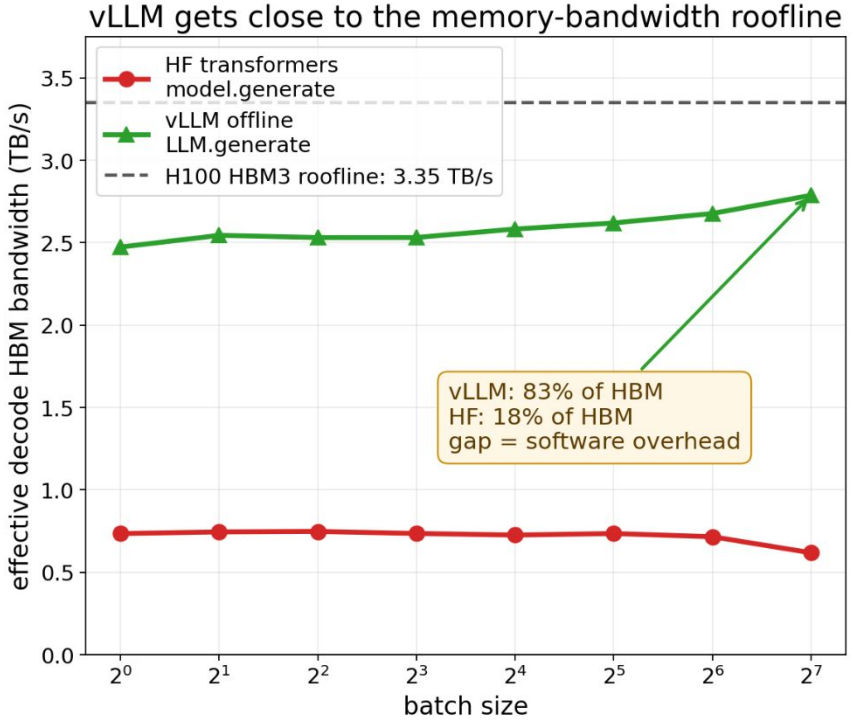
Nebius Academy · Performance Engineering

20 May, 2026 · Alexey Bukhtiyarov

vLLM converts batching into useful GPU memory bandwidth

Qwen3-8B BF16 on 1x H100 80GB | prompt=512 tokens | generate=128 tokens | greedy decode





Bandwidth is a roofline estimate: traffic = 16.4GB weights per decode step + batch x average context x 144KiB KV per token. Divide estimated traffic by measured wall time.

Inference Engines are not just a wrapper around model.forward()

It turns model execution into a fast, cost-effective, production service

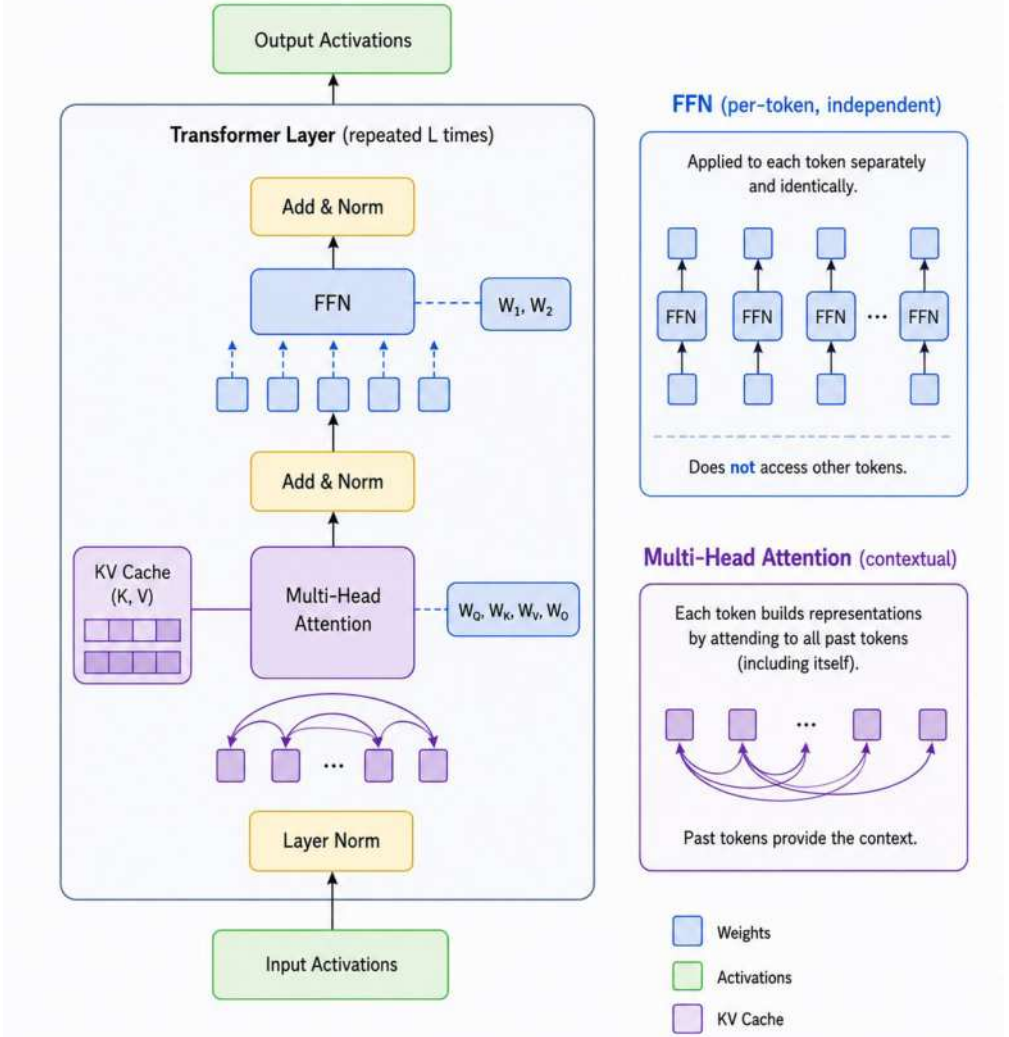
What we will cover today

- Inference Basics & Metrics
- Engine optimizations
  - Paged attention
  - Continuous batching
  - Scheduler
  - Tackling CPU overhead
  - Chunked prefill
    Compilation & CUDA graphs
  - Radix Attention
  - Constrained Decoding
  - Disaggregated P/D
- Multi-GPU inference: TP / PP / DP / EP
- Other inference stacks
- Monitoring

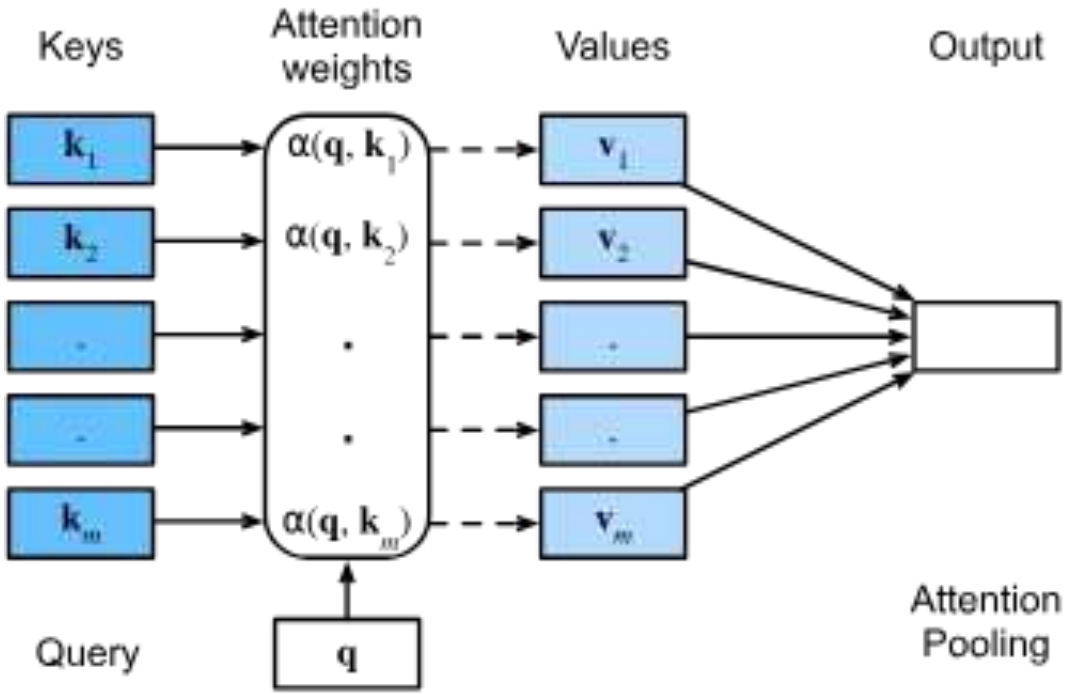
LLM Inference Basics

What's Stored During Inference?

What's Stored During Inference?



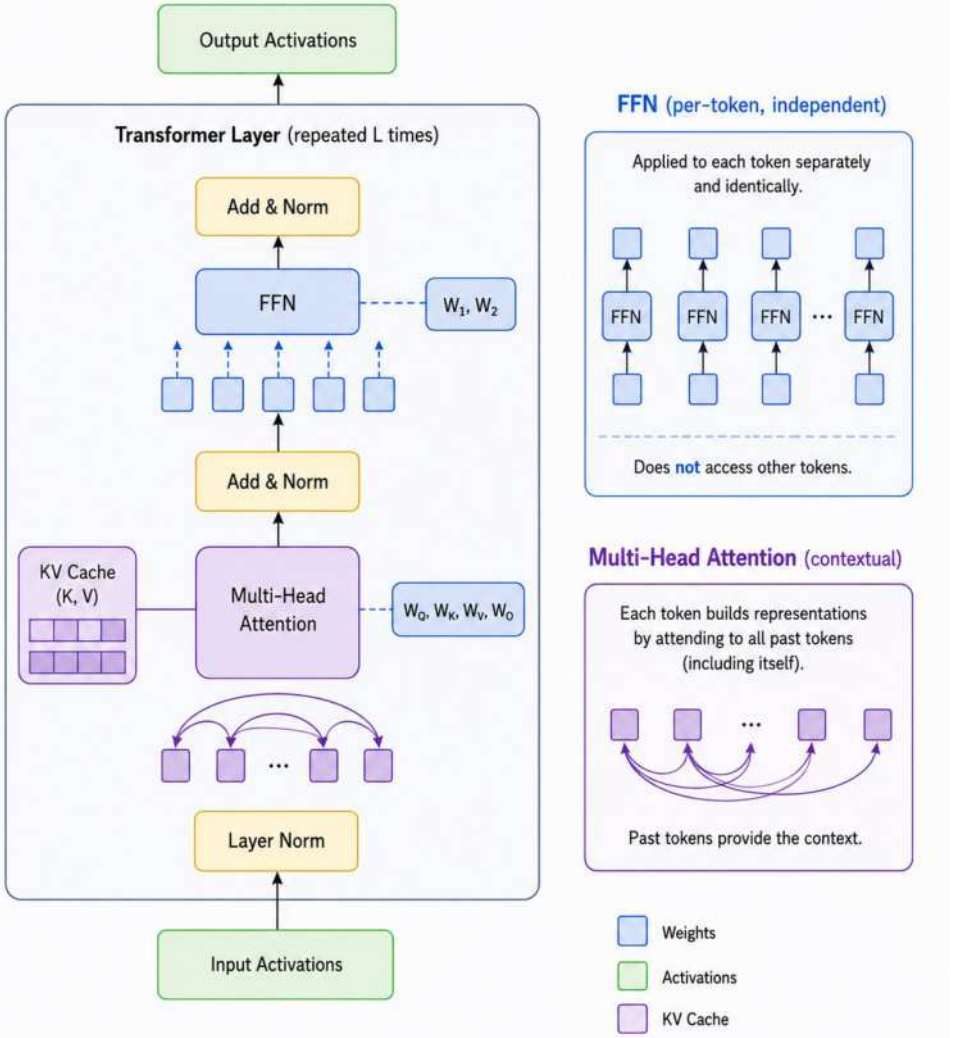
Attention Reminder

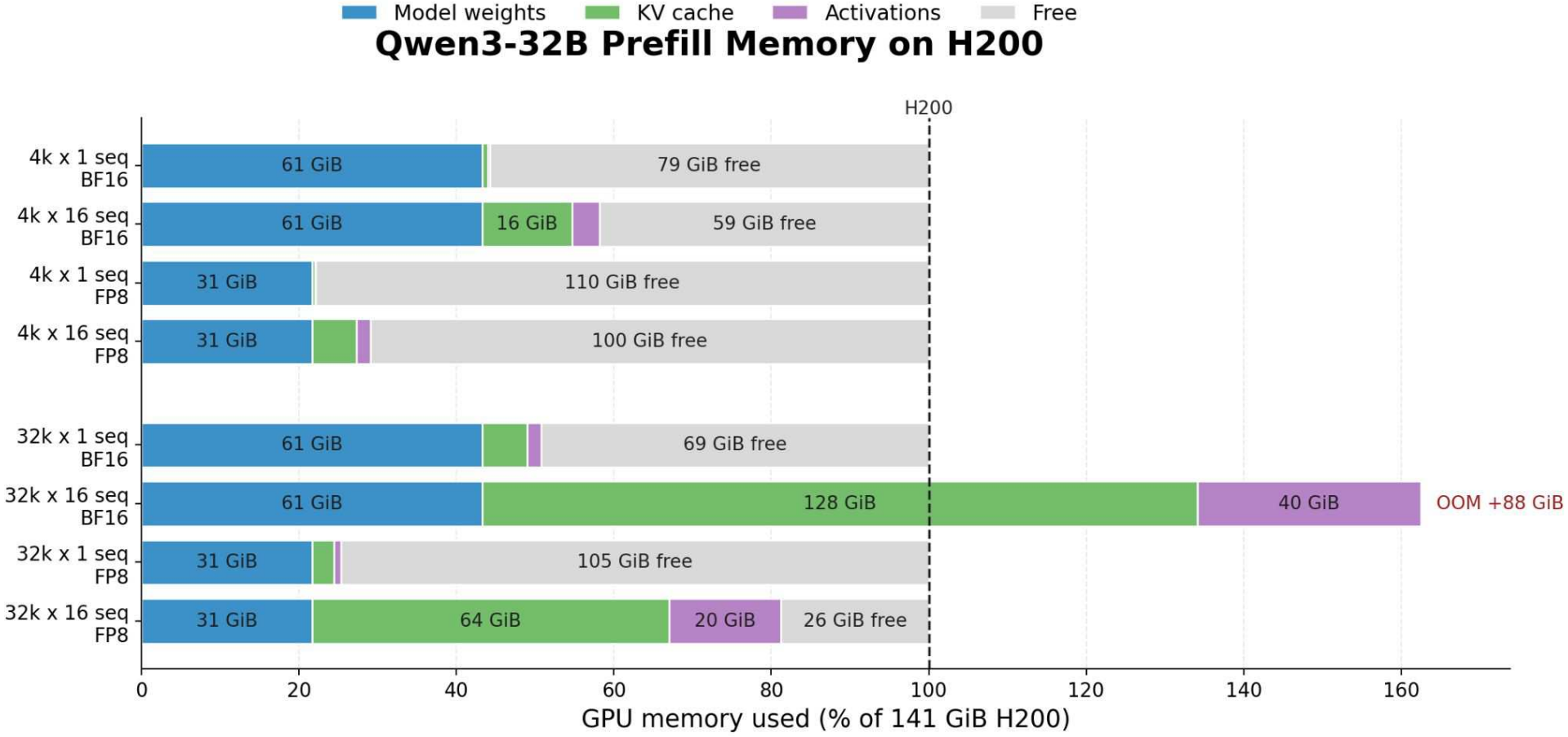


https://d2l.ai/chapter_attention-mechanisms-and-transformers/queries-keys-values.html

Important points about inference

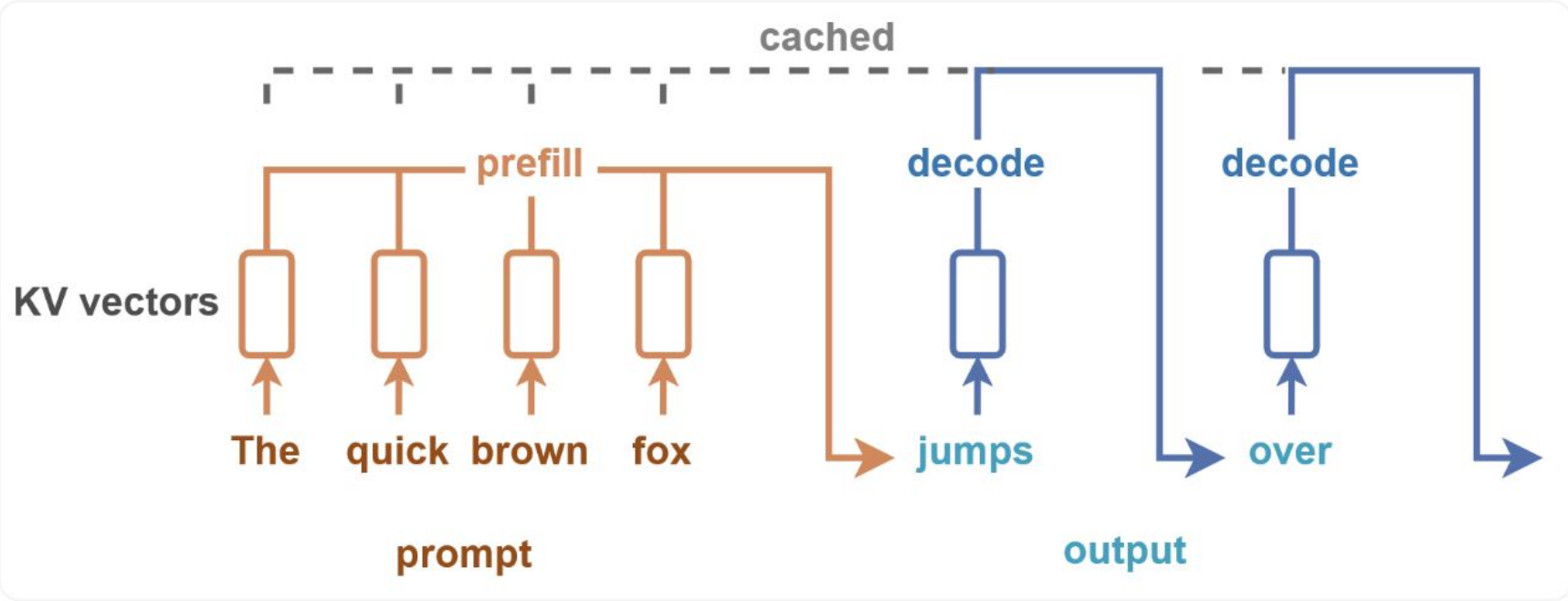
- Autoregressive: tokens are generated one by one.
- Stateful: each request carries a growing KV cache.
- Attention uses the KV cache to look back at previous tokens.
- FFN is applied independently to each token.




Model weights
KV cache
Activations
Free
Qwen3-32B Prefill Memory on H200
H200
4k x 1 seq BF16
61 GiB
79 GiB free
4k x 16 seq BF16
61 GiB
16 GiB
59 GiB free
4k x 1 seq FP8
31 GiB
110 GiB free
4k x 16 seq FP8
31 GiB
100 GiB free
32k x 1 seq BF16
61 GiB
69 GiB free
32k x 16 seq BF16
61 GiB
128 GiB
40 GiB
OOM +88 GiB
32k x 1 seq FP8
31 GiB
105 GiB free
32k x 16 seq FP8
31 GiB
64 GiB
20 GiB
26 GiB free
0
20
40
60
80
100
120
140
160
GPU memory used (% of 141 GiB H200)

Prefill is a single forward pass over the full prompt that builds the KV-cache and is typically compute-bound

Decode generates tokens one by one by reusing the KV-cache (and appending new KV values), and is typically memory-bound.

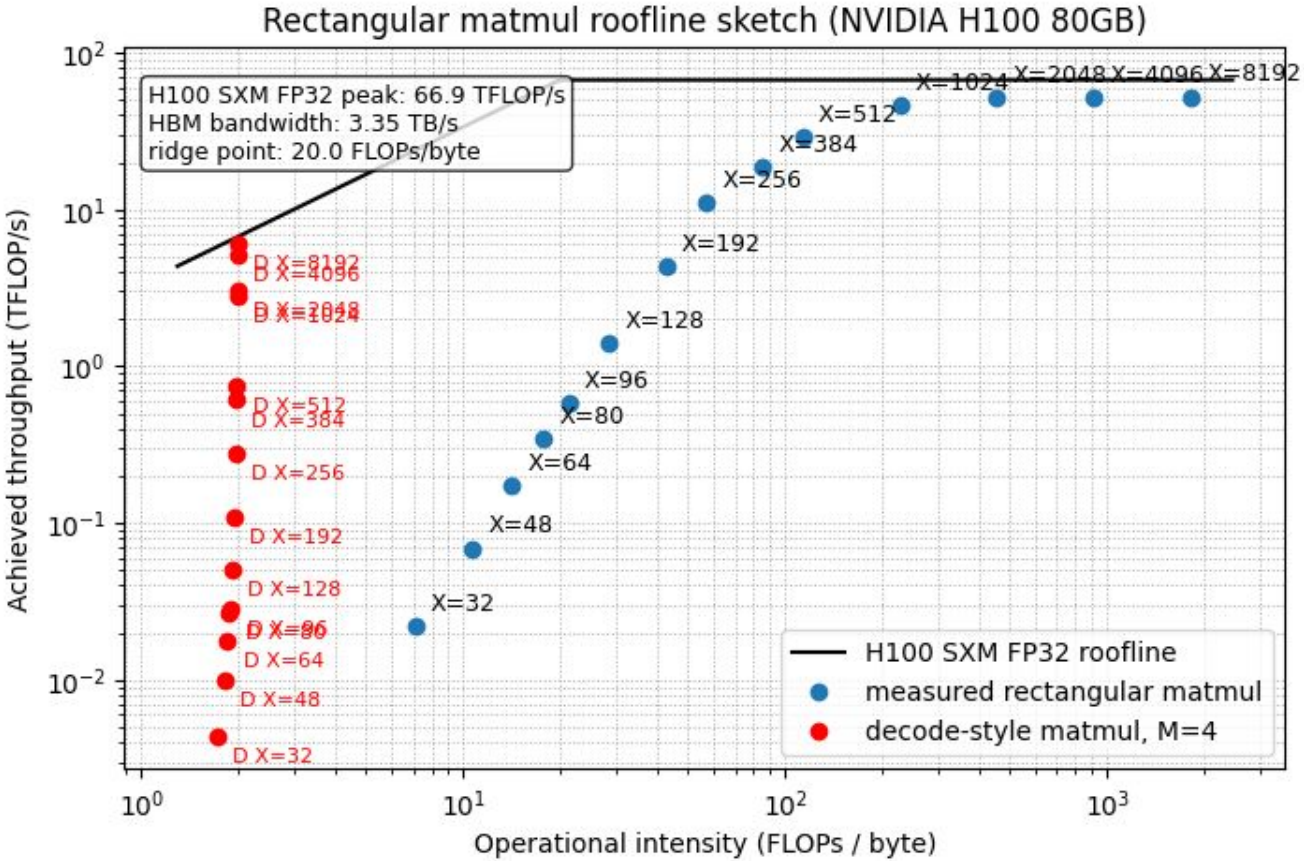


https://huggingface.co/blog/tngtech/llm-performance-prefill-decode-concurrent-requests

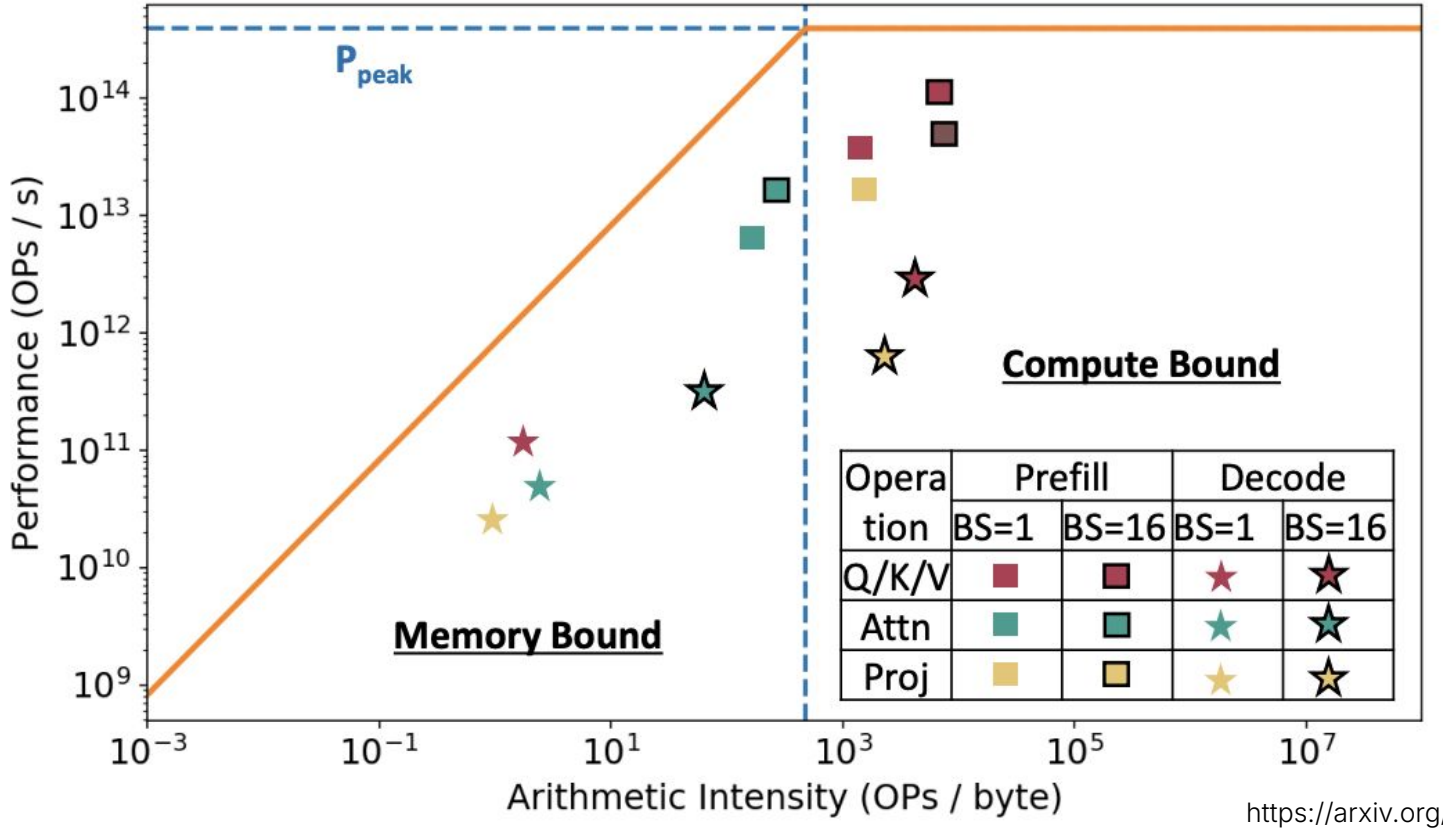
Feel the difference

Decode → Low Arithmetic Intensity → Memory Bound

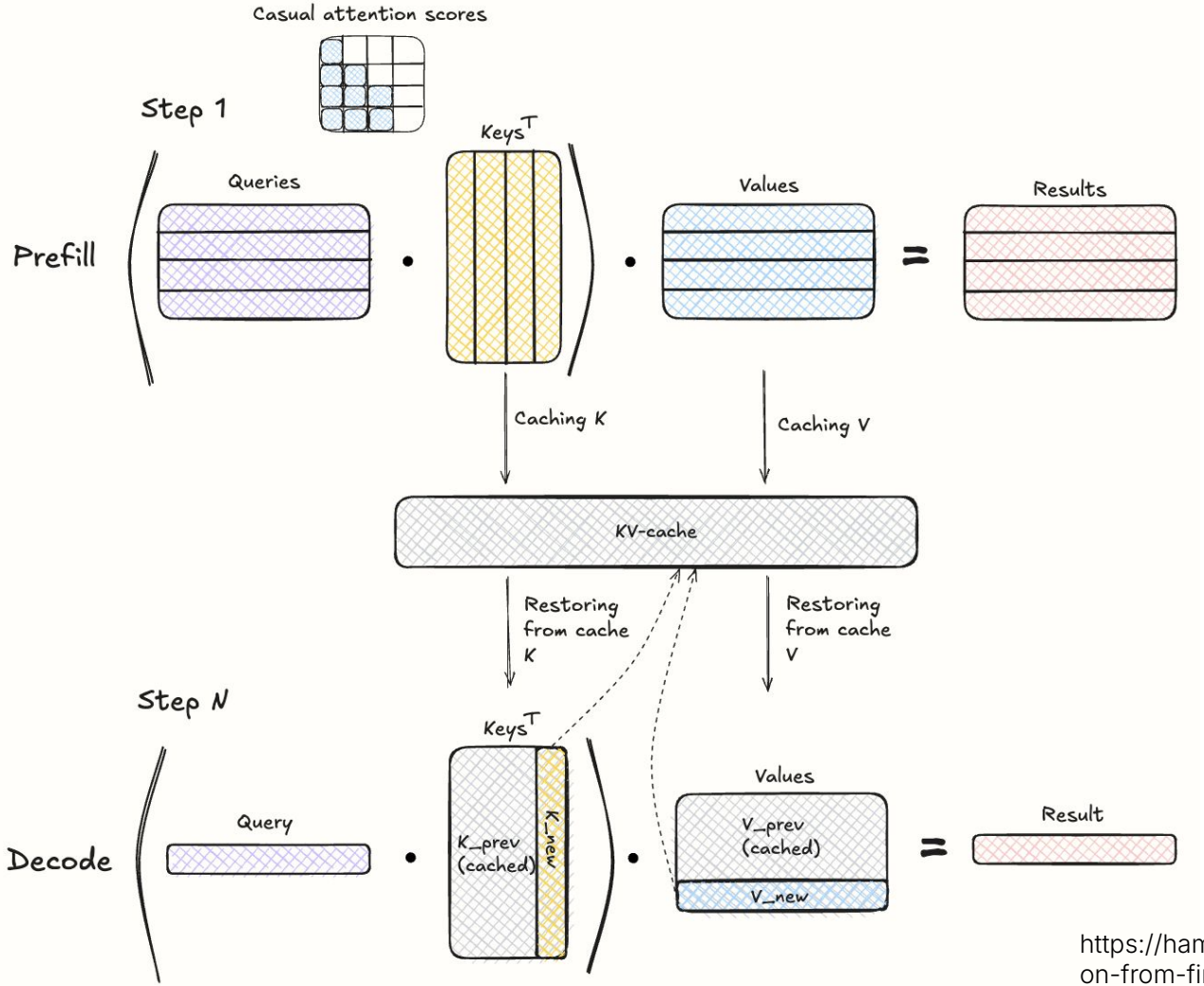
Prefill → High Arithmetic Intensity → Compute Bound



Prefill vs Decode



https://arxiv.org/pdf/2510.02675



https://hamzaelshafie.bearblog.dev/paged-attention-from-first-principles-a-view-inside-vllm/

How big is the KV-cache?
What does it depend on?

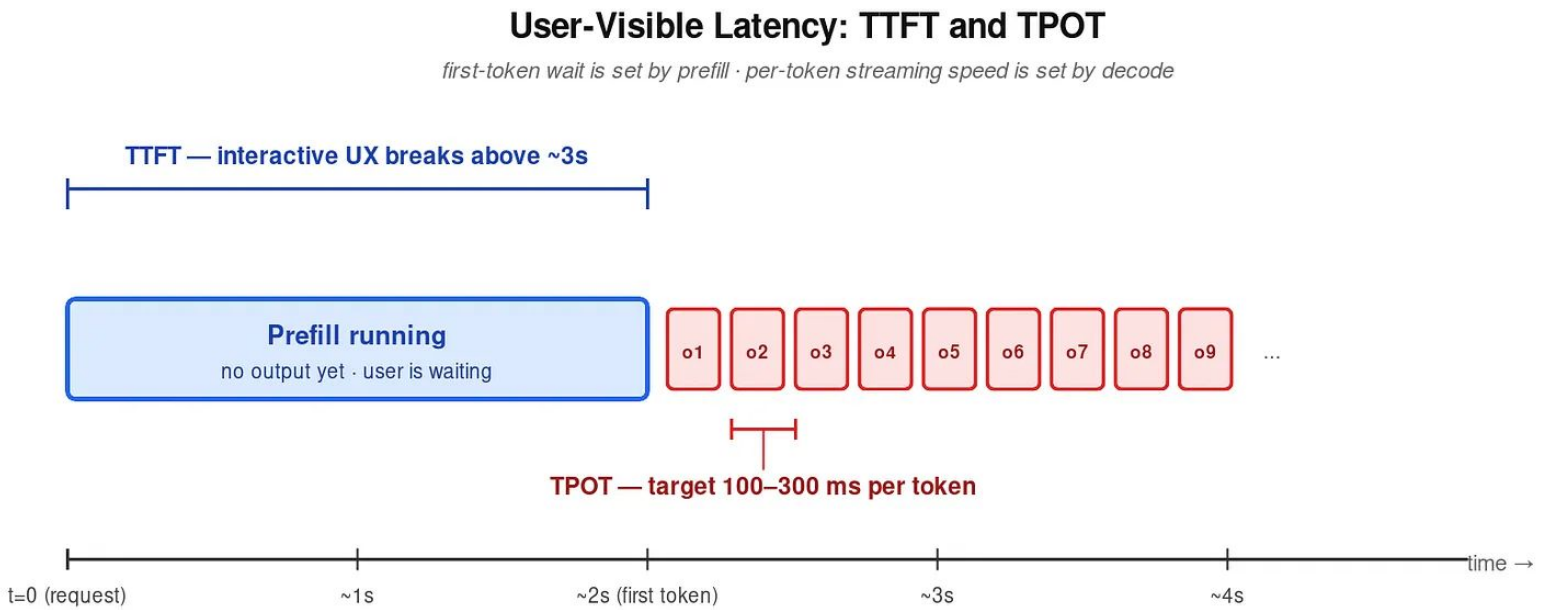
For Llama 3.3 70b, with 32k sequence length:

$$
\begin{aligned}
\text{KV cache size} = 2 & \\
&\times \text{num\_layers} \\
&\times \text{seq\_length} \\
&\times \text{num\_kv\_heads} \\
&\times \text{head\_dim} \\
&\times \text{bytes\_per\_value}
\end{aligned}
$$

$$
\begin{aligned}
& 2 \times 80 \times 32{,}000 \times 8 \times 128 \times 2 \\
&= 10{,}485{,}760{,}000 \text{ bytes} \\
&= \frac{10{,}485{,}760{,}000}{1024^3} \text{ GiB} \\
&\approx 9.77 \text{ GiB}
\end{aligned}
$$

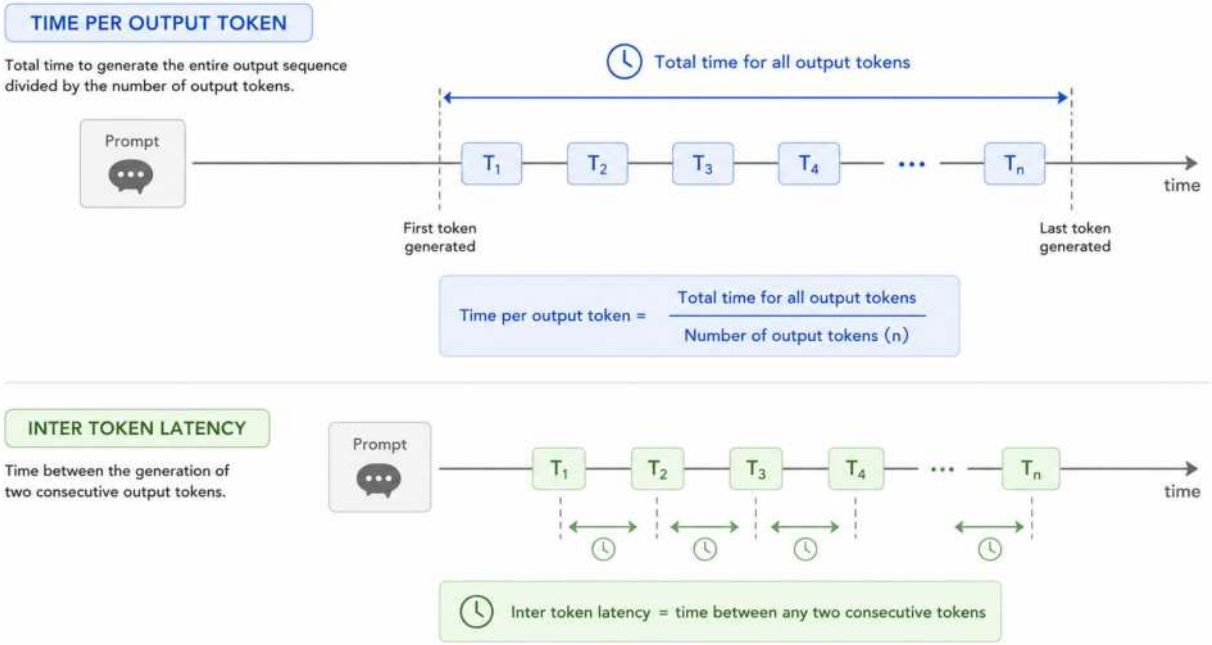
Inference Metrics

TTFT, TPOT, ITL E2E latency Throughput (tok/sec) Cost



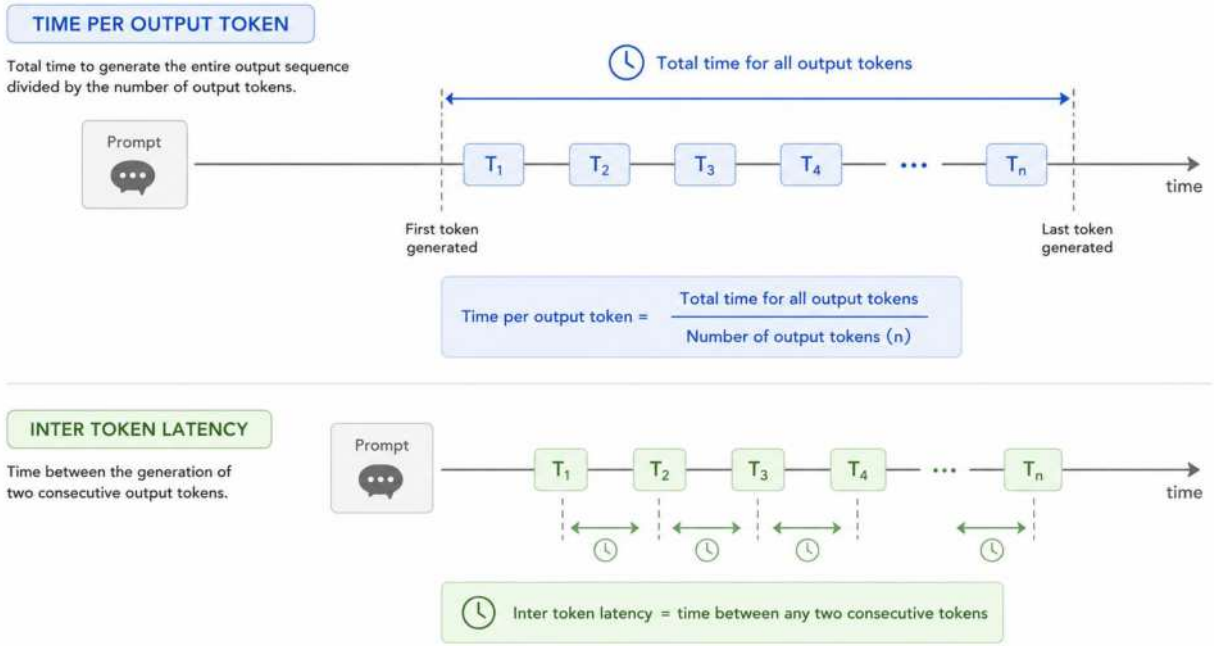
TPOT vs ITL

- For a single request, TPOT is the average ITL
- Across many requests, TPOT is request-weighted, while ITL is token-weighted - long outputs matter more for ITL



TPOT vs ITL

- For a single request, TPOT is the average ITL
- Across many requests, TPOT is request-weighted, while ITL is token-weighted - long outputs matter more for ITL



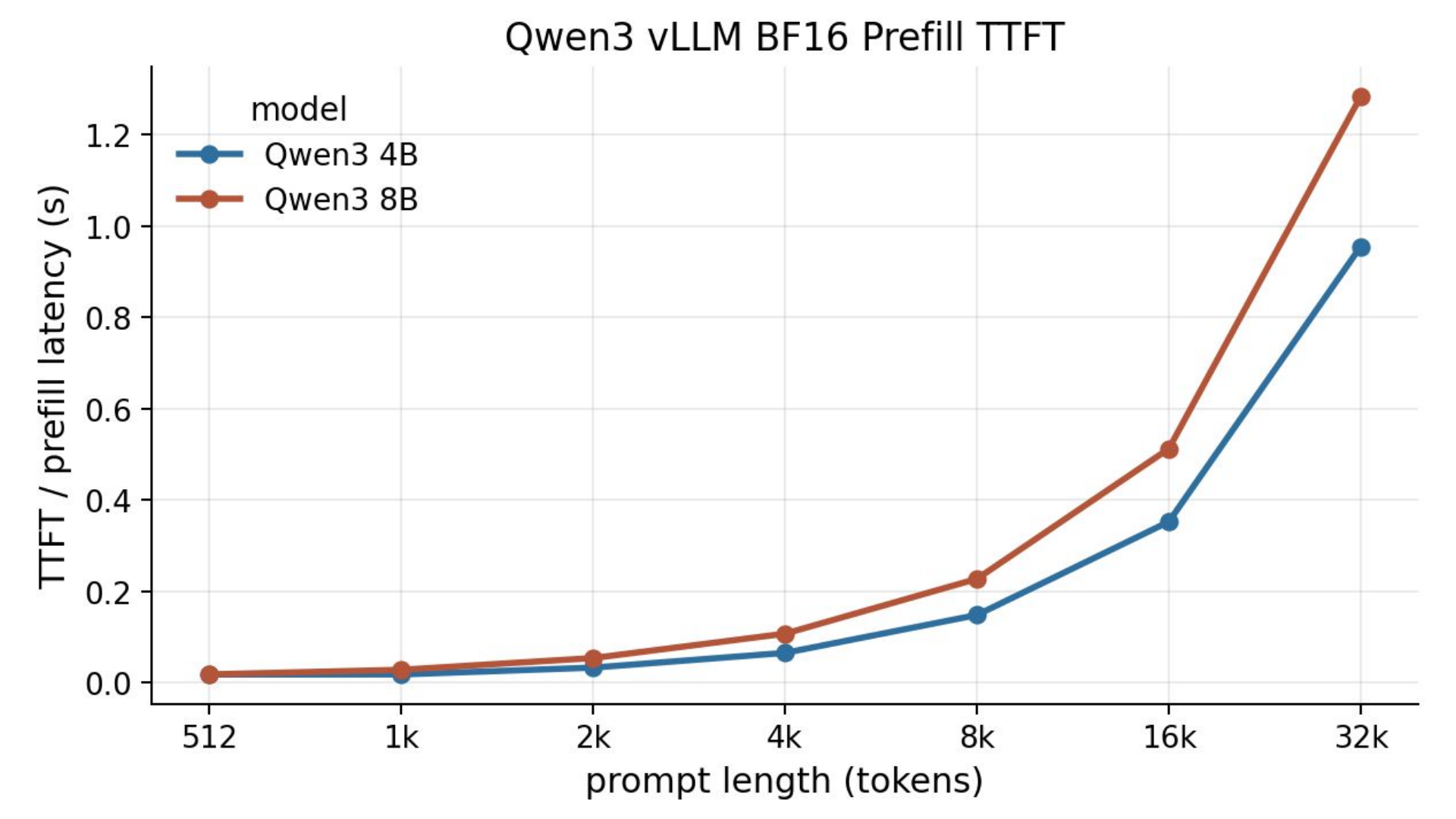
TPOT: “How fast does the typical request stream?”
ITL: “How fast is the typical token gap across the whole workload?”

E2E latency ~= TTFT + num_output_tokens * TPOT

- Long prompts ...
- Long output seq ...

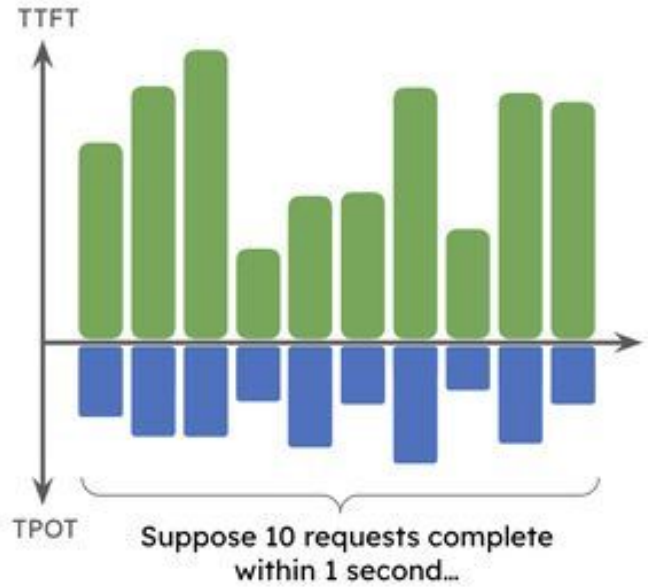
E2E latency ~= TTFT + num_output_tokens * TPOT

- Long prompts usually hurt TTFT first
- Long output seq hurts E2E latency

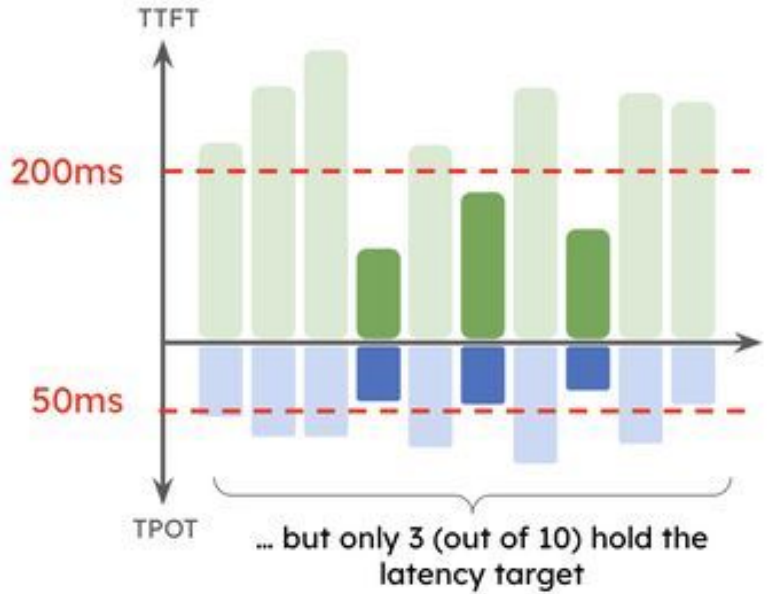

Qwen3 vLLM BF16 Prefill TTFT
model
Qwen3 4B
Qwen3 8B
TTFT / prefill latency (s)
0.0
0.2
0.4
0.6
0.8
1.0
1.2
512
1k
2k
4k
8k
16k
32k
prompt length (tokens)

High Throughput ≠ High Goodput

Throughput = completed request / time
= **10** req / s



Goodput = completed requests **within SLO** / time
= **3** req / s



https://www.linkedin.com/posts/shamsheransari_ai-genai-gpu-activity-7397223890206330880-orLu/

	Training	Inference
Lifecycle	Expensive offline job, run during model development	Online service, runs continuously for users, for months / years
Batch shape	Large, regular, fixed	Small, dynamic, irregular
Dominant work	Forward + Backward	Prefill + many decode steps
Memory pressure	Activations + optimizer state	Weights + KV-cache
Common bottleneck	Compute / communication	Often decode-time memory bandwidth

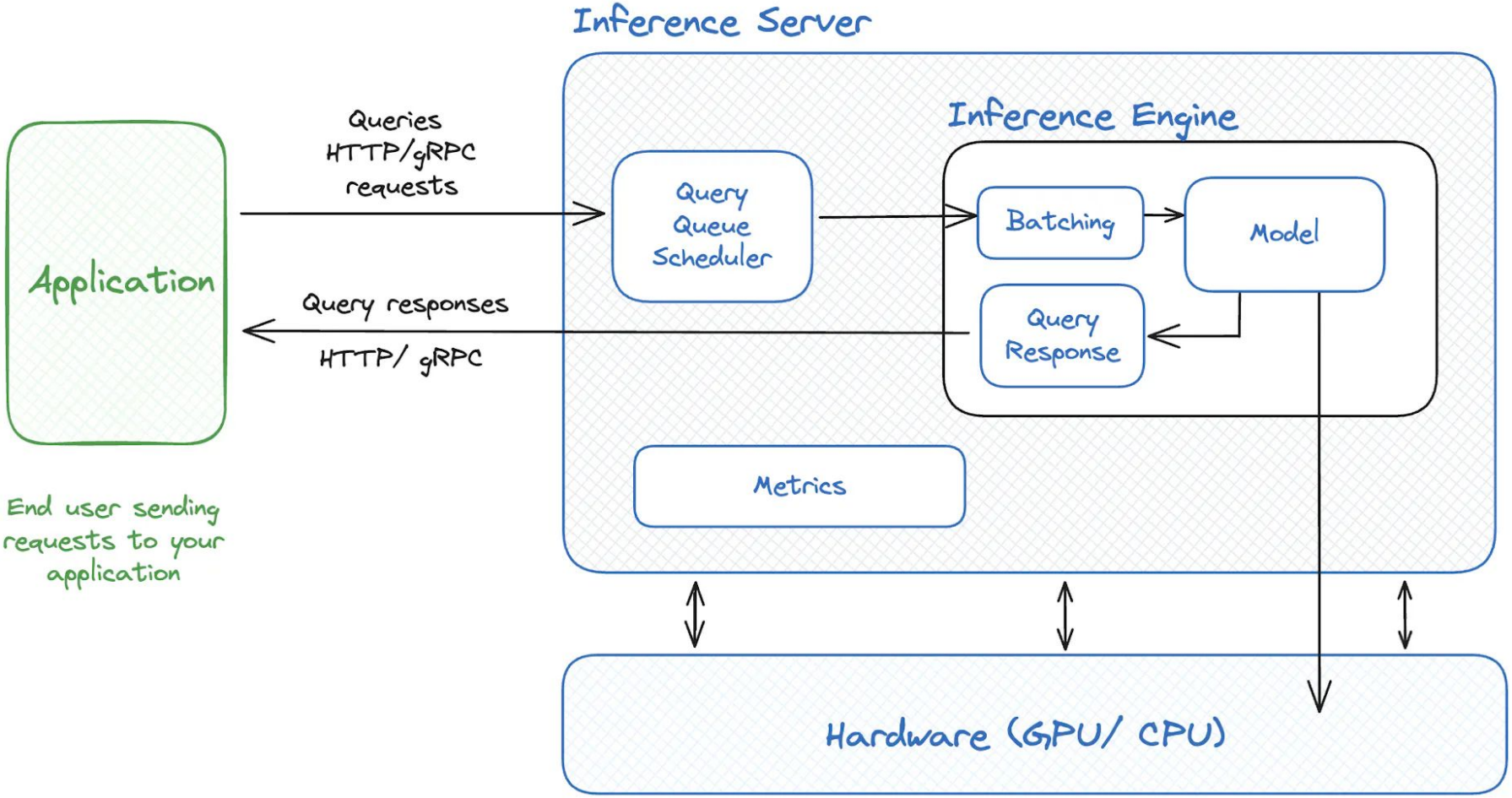
Inference Engine

1. Black box that takes input text, generation parameters and outputs the generated text.

2. Fast and cost-effective.

3. Production ready.

4. A microservice, that can be optimized and scaled separately.


Inference Server
Inference Engine
Application
Queries
HTTP/gRPC
requests
Query
Queue
Scheduler
Batching
Model
Query responses
HTTP/ gRPC
Query
Response
Metrics
End user sending
requests to your
application
Hardware (GPU/ CPU)

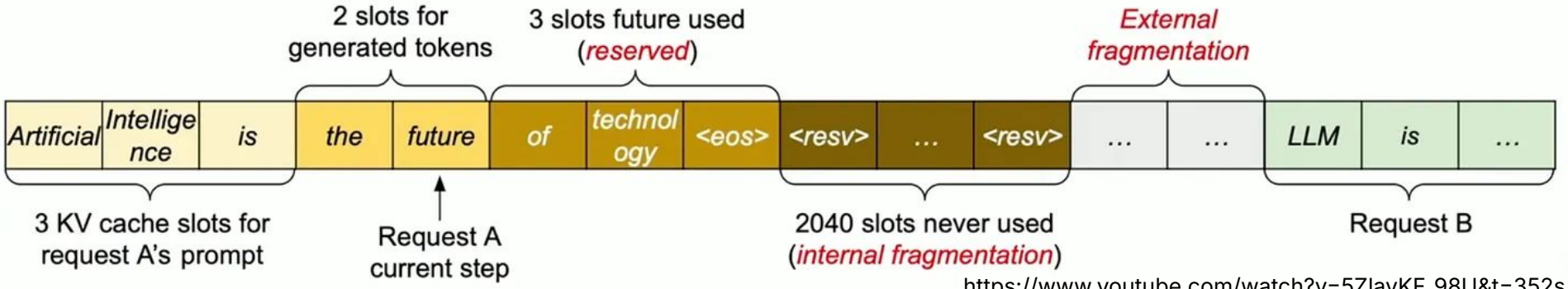
Takeaways

- Inference has two phases: prefill is mostly compute-bound, decode is mostly memory-bound.
- KV-cache makes decoding fast enough to be practical, but turns serving into a memory-management problem.
- The serving metrics to watch are TTFT, TPOT, and throughput.
- Inference engines exist because production serving needs scheduling, batching, KV-cache management, and observability on top of the model itself.


LLM

Memory Waste in Naive KV Cache

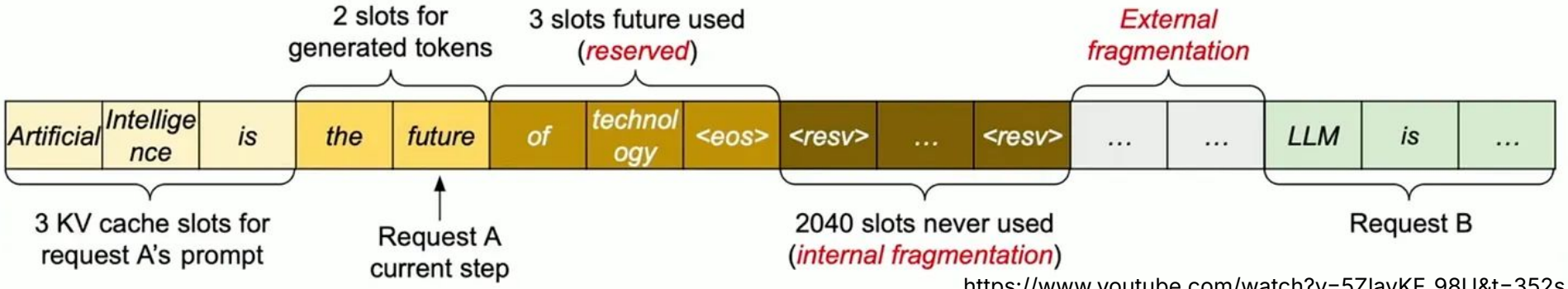
- **Internal Fragmentation**: over-allocated due to the unknown output length



https://www.youtube.com/watch?v=5ZlavKF_98U&t=352s

Memory Waste in Naive KV Cache

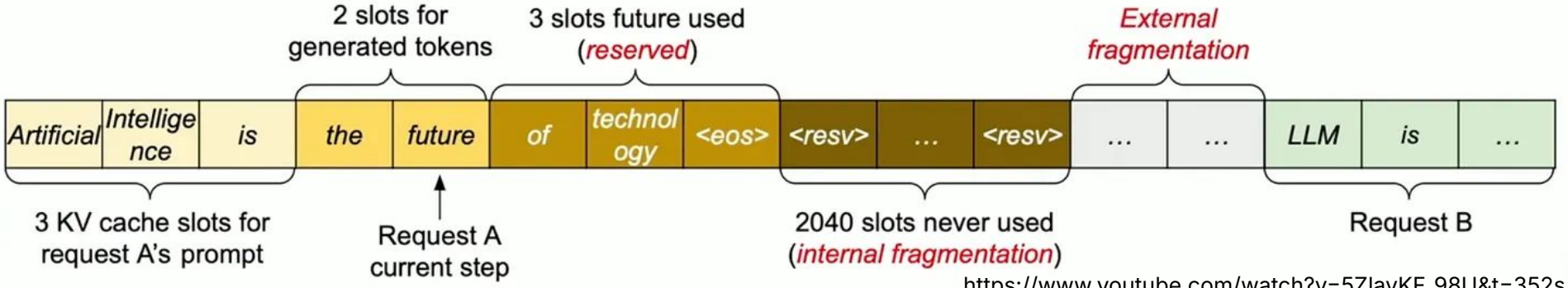
- **Internal Fragmentation**: over-allocated due to the unknown output length
- **Reservation**: not used at the current step, but will be used in the future



https://www.youtube.com/watch?v=5ZlavKF_98U&t=352s

Memory Waste in Naive KV Cache

- **Internal Fragmentation**: over-allocated due to the unknown output length
- **Reservation**: not used at the current step, but will be used in the future
- **External fragmentation**: memory is free but scattered into gaps that can't fit new sequences



https://www.youtube.com/watch?v=5ZlavKF_98U&t=352s

Memory Waste in Naive KV Cache

- **60-80%** was wasted
- smaller batch size → lower intensity → lower throughput

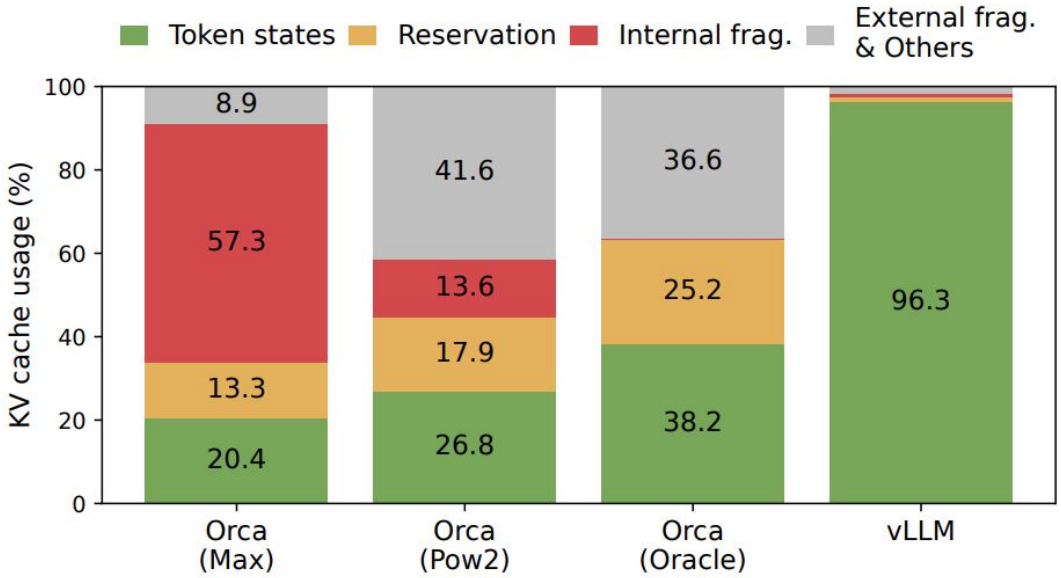
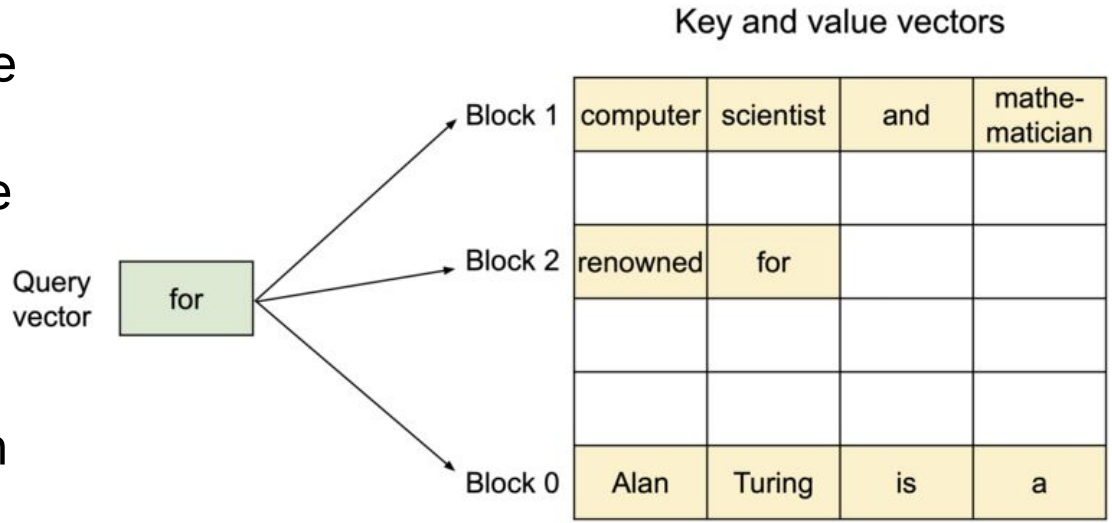


Figure 2. Average percentage of memory wastes in different LLM serving systems during the experiment in §6.2.

https://arxiv.org/pdf/2309.06180

Paged Attention Idea

- Allows for storing attention key and values vectors in non-contiguous blocks in the memory
- Each KV block is a fixed-size contiguous chunk of memory that can store KV cache from left to right
- Attention for each new token is computed across all tokens in the non-contiguous KV blocks



https://vllm.ai/blog/vllm

Paged Attention

0. Before generation.

Seq A

Prompt: “Alan Turing is a computer scientist”
Completion: “”

Logical KV cache blocks

Block 0				
Block 1				
Block 2				
Block 3				

Block table

Physical block no.	# Filled slots
–	–
–	–
–	–
–	–

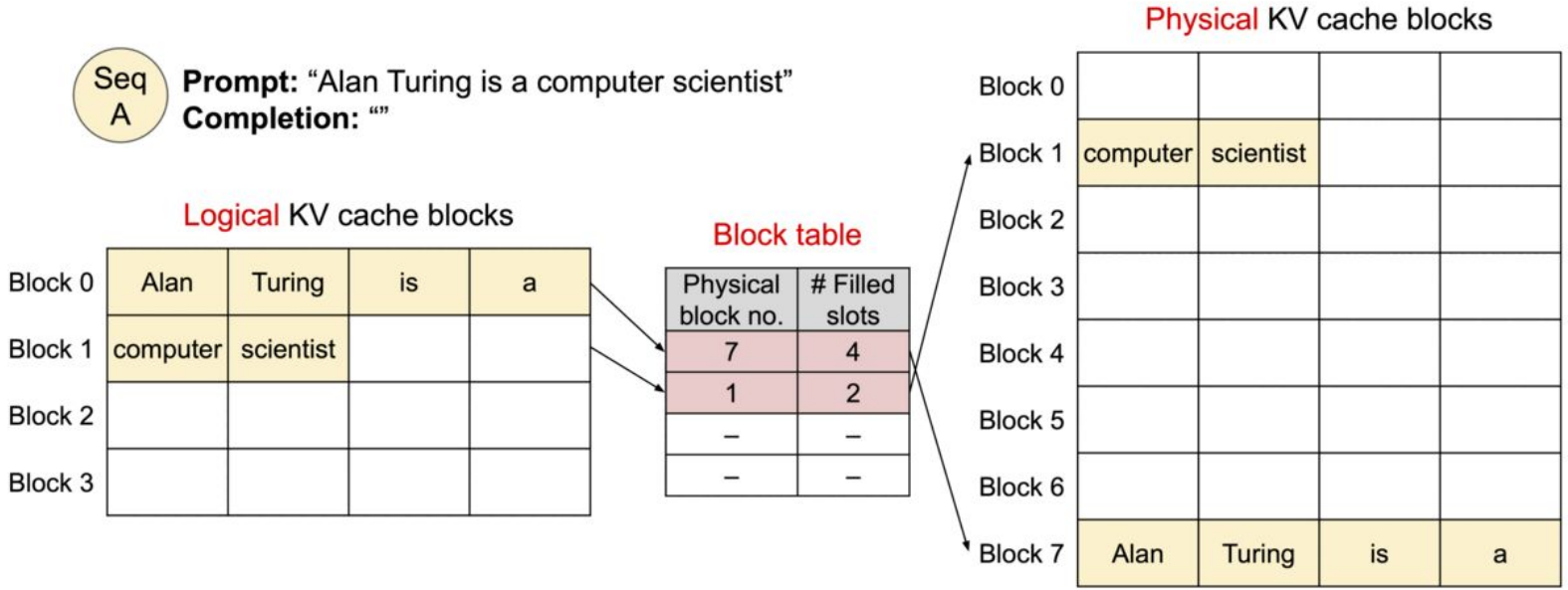
Physical KV cache blocks

Block 0				
Block 1				
Block 2				
Block 3				
Block 4				
Block 5				
Block 6				
Block 7				

https://vllm.ai/blog/vllm

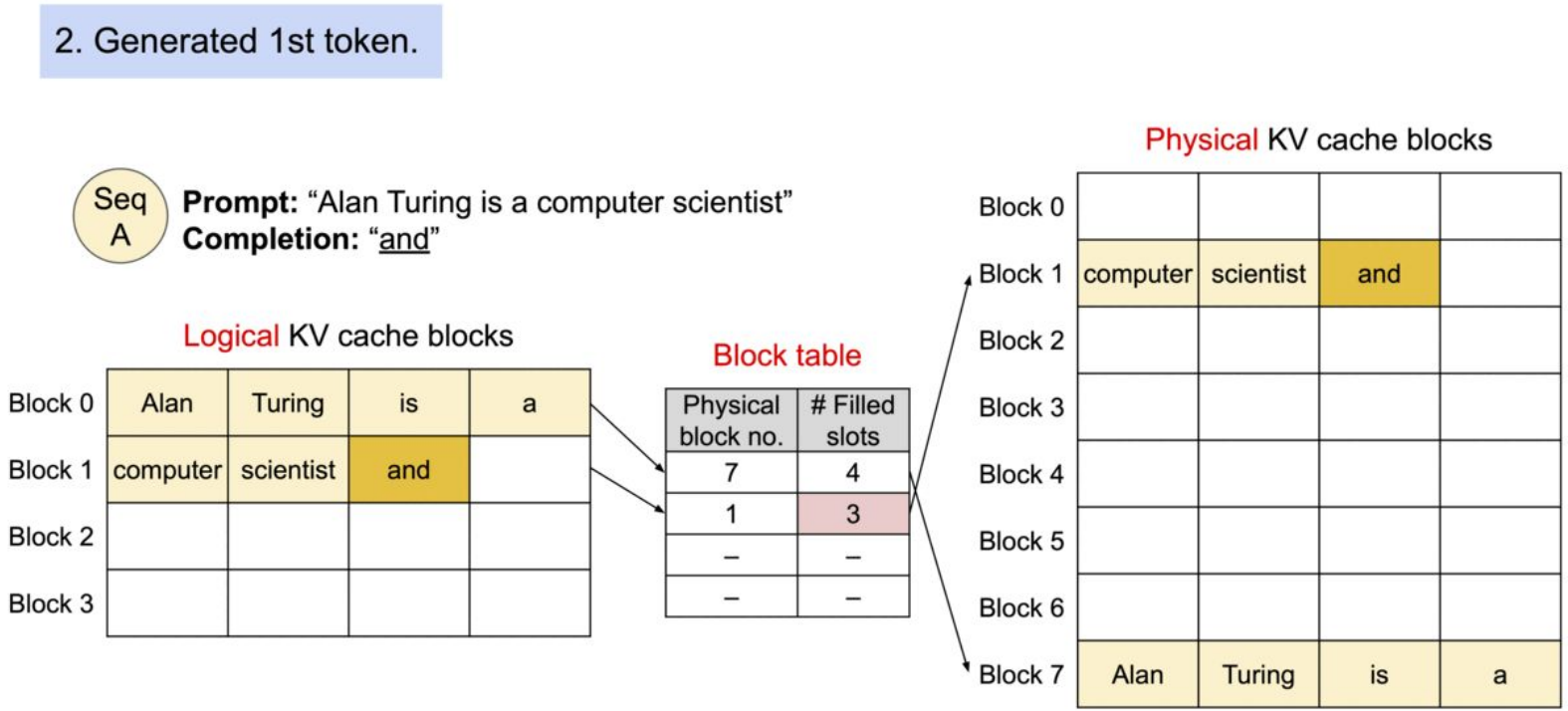
Paged Attention

1. Allocate space and store the prompt's KV cache.



https://vllm.ai/blog/vllm

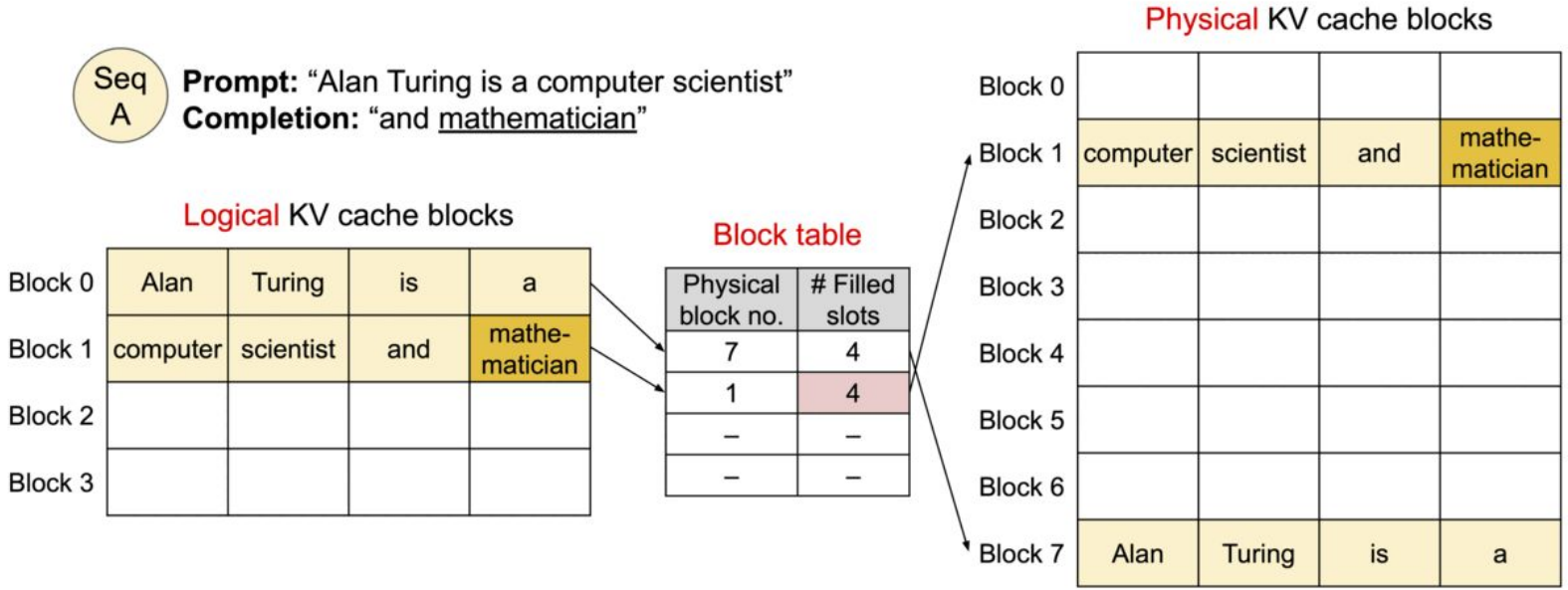
Paged Attention



https://vllm.ai/blog/vllm

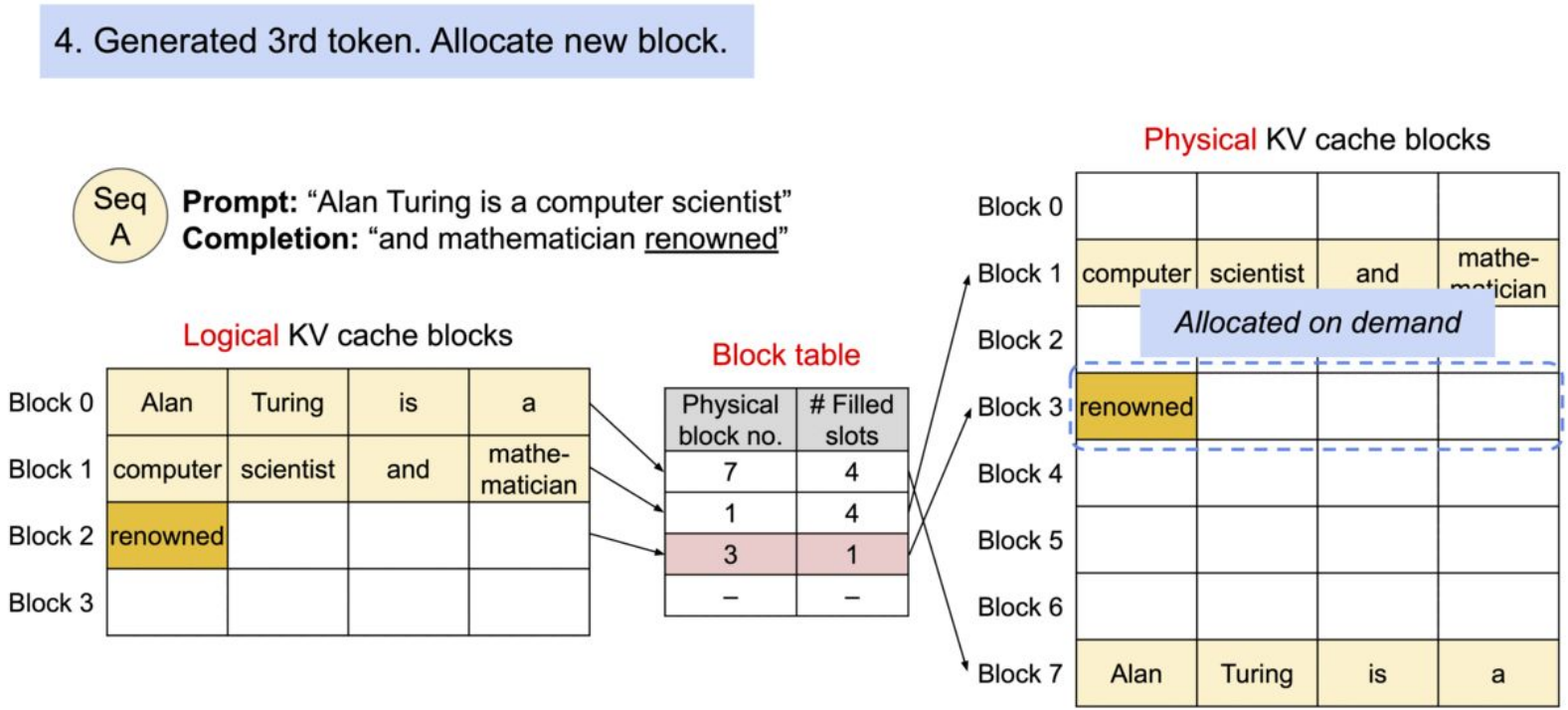
Paged Attention

3. Generated 2nd token.



https://vllm.ai/blog/vllm

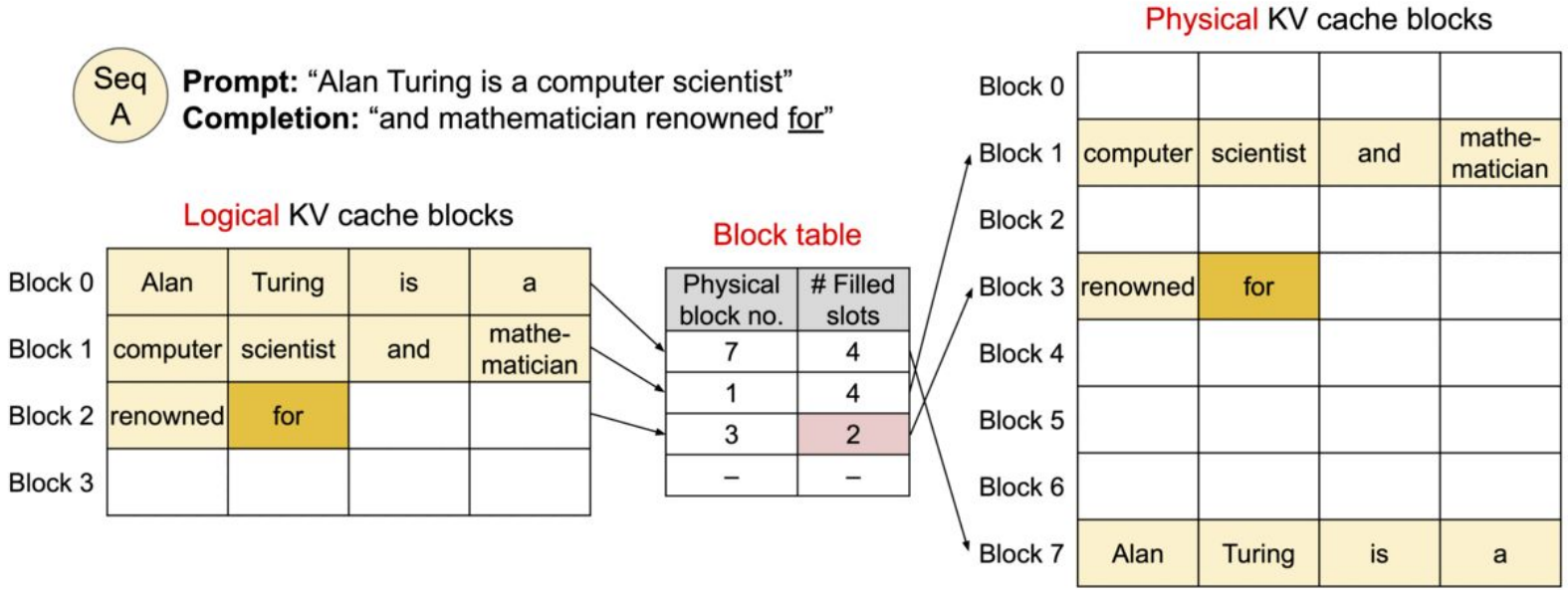
Paged Attention



https://vllm.ai/blog/vllm

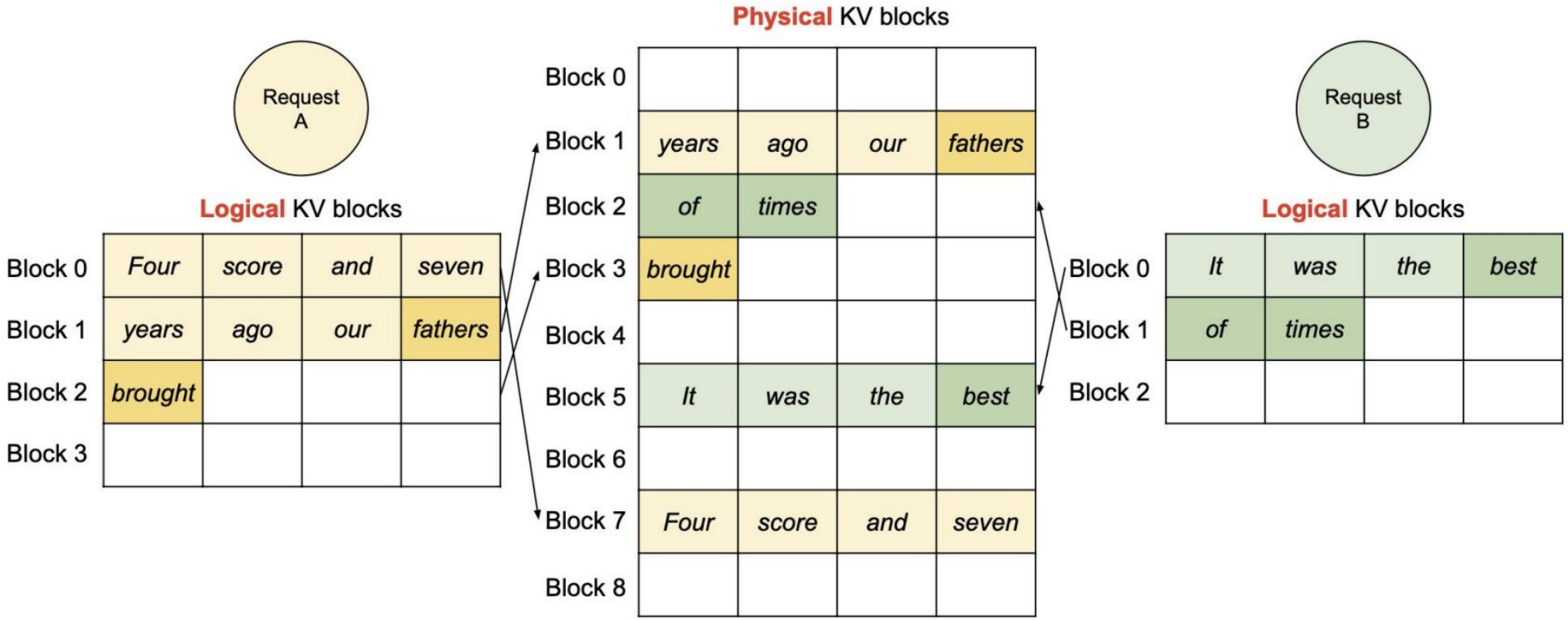
Paged Attention

5. Generated 4th token.



https://vllm.ai/blog/vllm

Handling Multiple Requests



Paged Attention Recap

- Split the KV cache into fixed-size blocks (“pages”) instead of one contiguous buffer
- Each sequence maintains a block table: logical → physical blocks
- Allocate KV blocks on demand as tokens are generated
- During attention, the kernel follows the block table to gather K/V tensors directly from scattered physical blocks in GPU memory

What Paged Attention gives us

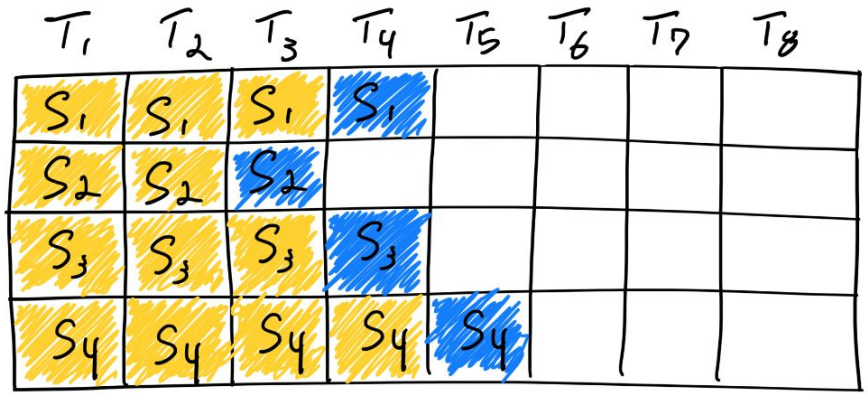
- Near-zero fragmentation: only the last block of each sequence is partially filled; zero external fragmentation
- More sequences fit in HBM → bigger batches → higher throughput
- Easy KV sharing for prefix caching and parallel sampling
- Faster request scheduling/admission: requests can start immediately without reserving full max-length buffers
- Enables efficient eviction of KV blocks across requests

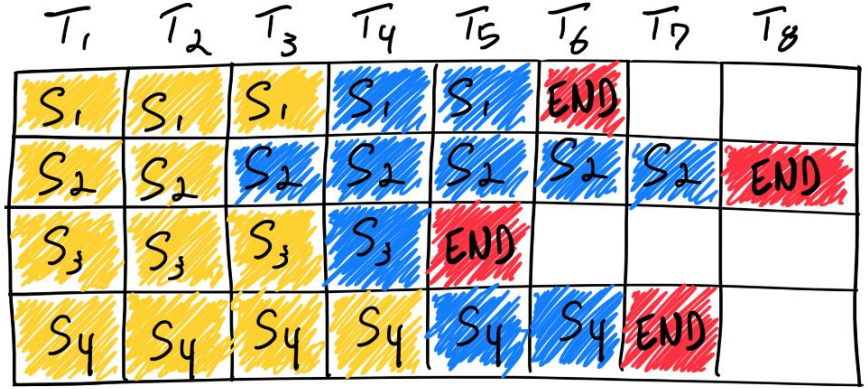
Paged attention gives us the memory flexibility to hold many active sequences

Now: how do we schedule them efficiently during decode?

Naive batching

- Classic static batching: group N requests into a **fixed** batch, run them together until all of them finish
- Shorter requests leave idle GPU slots while waiting for the longest one

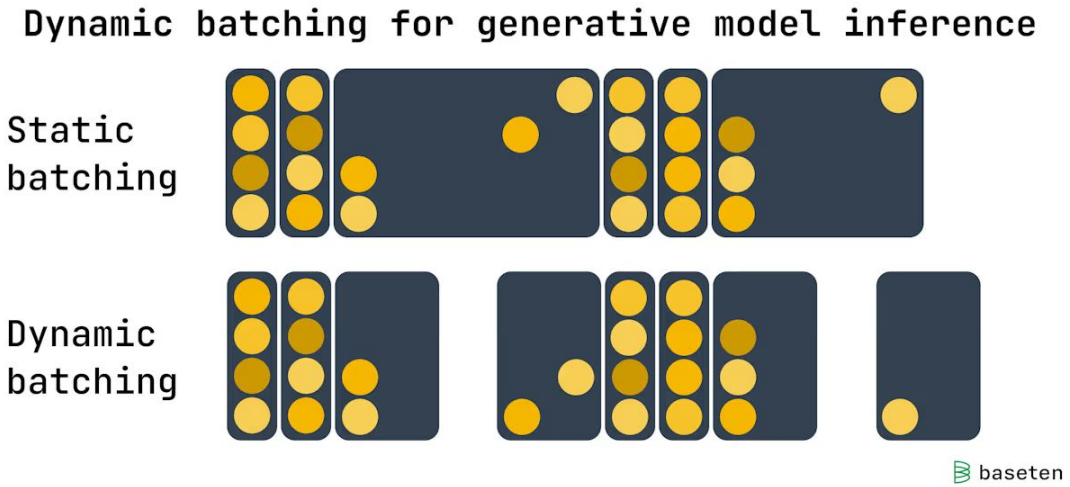




https://www.anyscale.com/blog/continuous-batching-llm-inference

Dynamic batching

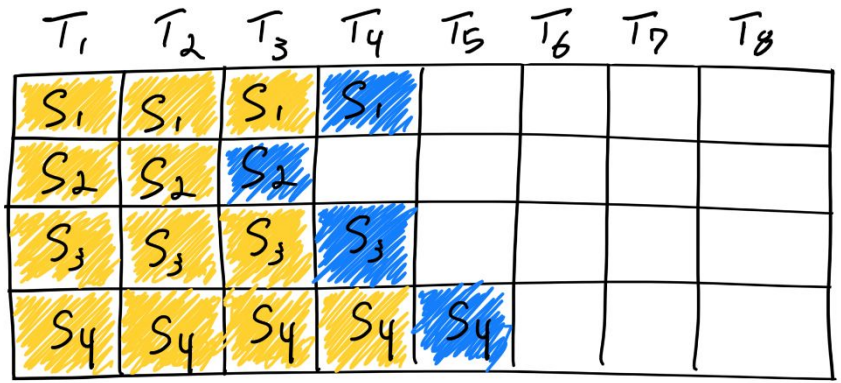
- Collect requests for a short time window before launching the batch
- Launch earlier if the batch becomes full
- Better GPU utilization than static batching
  - static batching may leave the GPU underutilized while waiting for enough requests
- But the batch is still fixed after launch

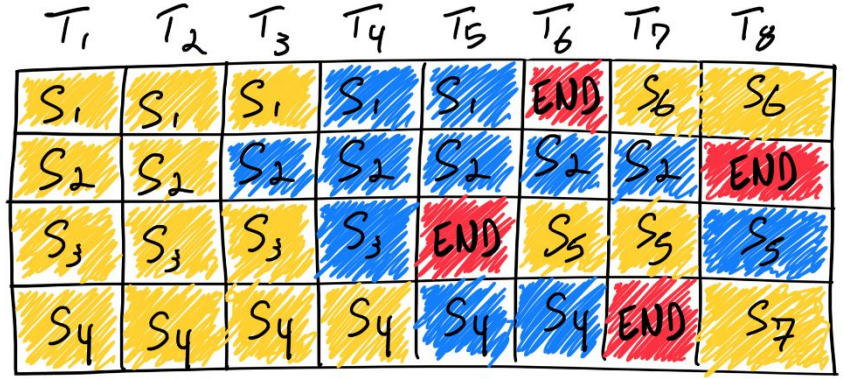


https://www.baseten.co/blog/continuous-vs-dynamic-batching-for-ai-inference/#dynamic-batching-for-ai-inference-in-production

Continuous batching / Iterative Scheduling

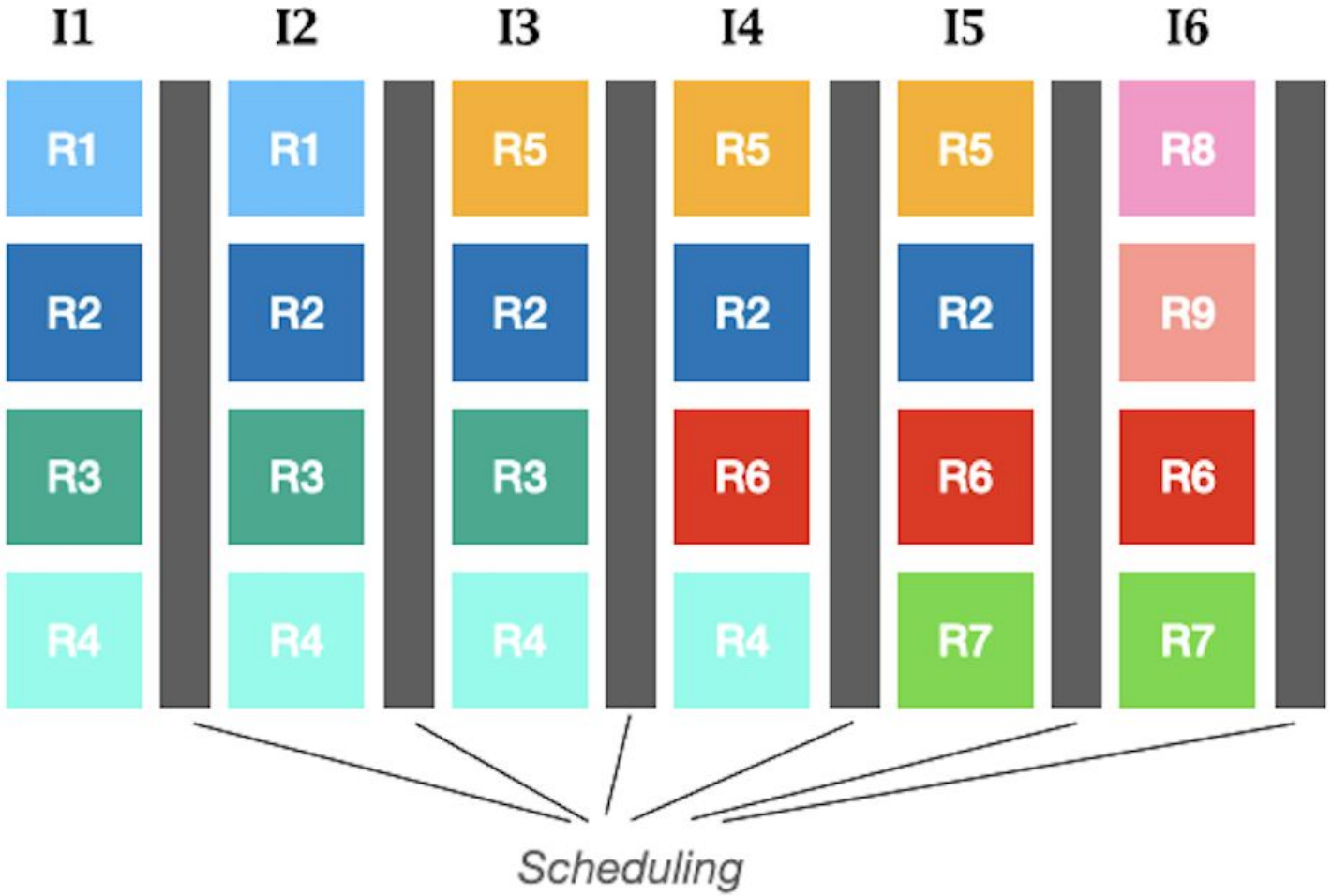
- Scheduler runs **every** decode step
- When a request finishes, a new one immediately takes its slot
- The batch becomes a moving window, not a fixed group
- Huge throughput win on real workloads





https://www.anyscale.com/blog/continuous-batching-llm-inference

Decides the batch composition at every step of model forwarding



https://mlsys.wuklab.io/posts/scheduling_overhead/

The Scheduler’s Job

Every engine has a scheduler loop on the CPU that, each step, decides:

- Admission: which waiting requests enter the running batch

The Scheduler’s Job

Every engine has a scheduler loop on the CPU that, each step, decides:

- Admission: which waiting requests enter the running batch
- Batch composition: which active sequences run this step

The Scheduler’s Job

Every engine has a scheduler loop on the CPU that, each step, decides:

- Admission: which waiting requests enter the running batch
- Batch composition: which active sequences run this step
- Prefill vs decode budget: how much GPU work goes to prompt processing vs token generation

The Scheduler’s Job

Every engine has a scheduler loop on the CPU that, each step, decides:

- Admission: which waiting requests enter the running batch
- Batch composition: which active sequences run this step
- Prefill vs decode budget: how much GPU work goes to prompt processing vs token generation
- Preemption: which running sequence gets paused if memory is tight

The Scheduler’s Job

Every engine has a scheduler loop on the CPU that, each step, decides:

- Admission: which waiting requests enter the running batch
- Batch composition: which active sequences run this step
- Prefill vs decode budget: how much GPU work goes to prompt processing vs token generation
- Preemption: which running sequence gets paused if memory is tight
- Priority / fairness / SLA policy: FIFO, priority queues, per-user fairness, latency targets

CPU Bottleneck

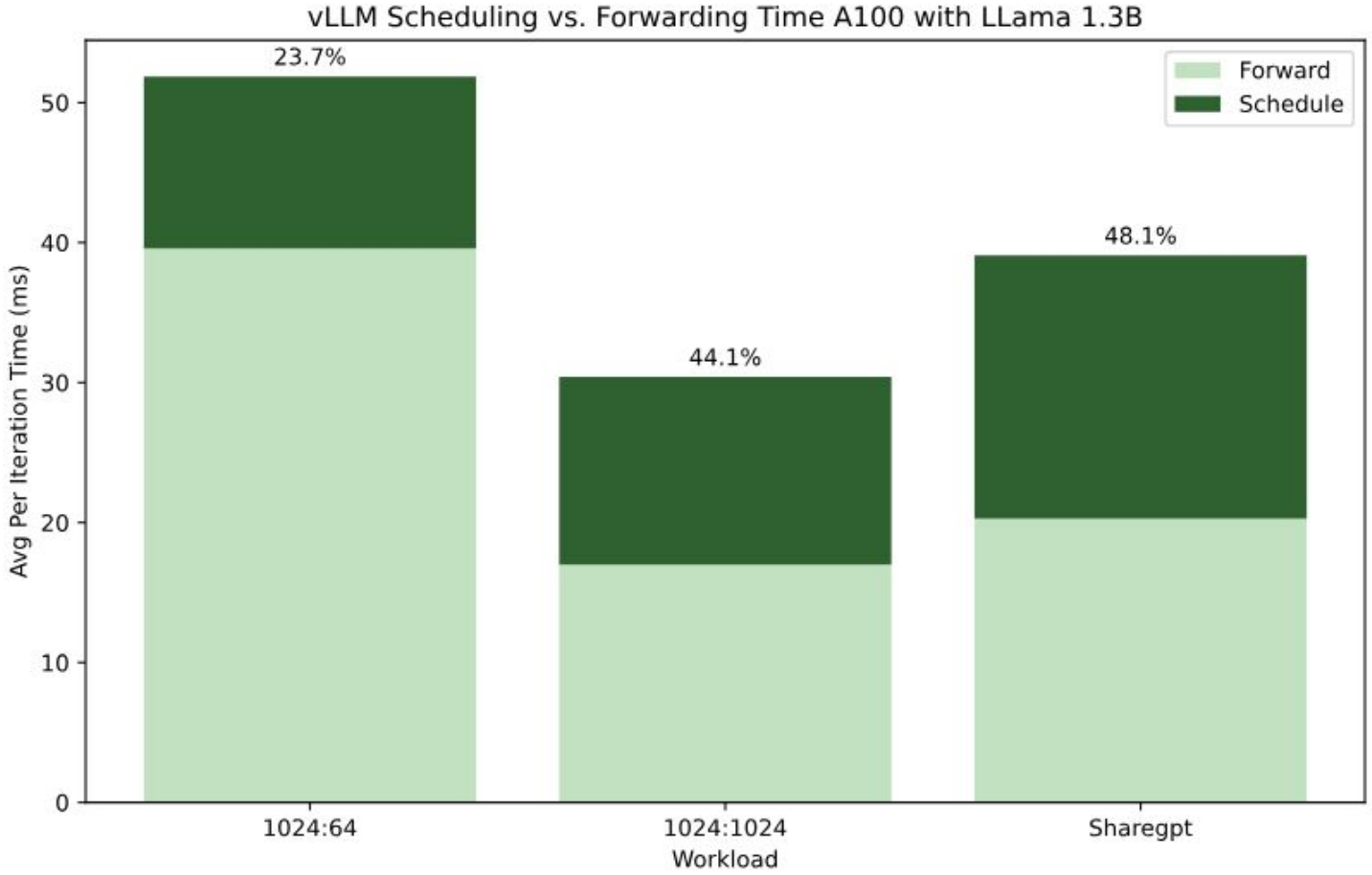
Continuous batching schedules work at fine granularity.

That improves GPU utilization.

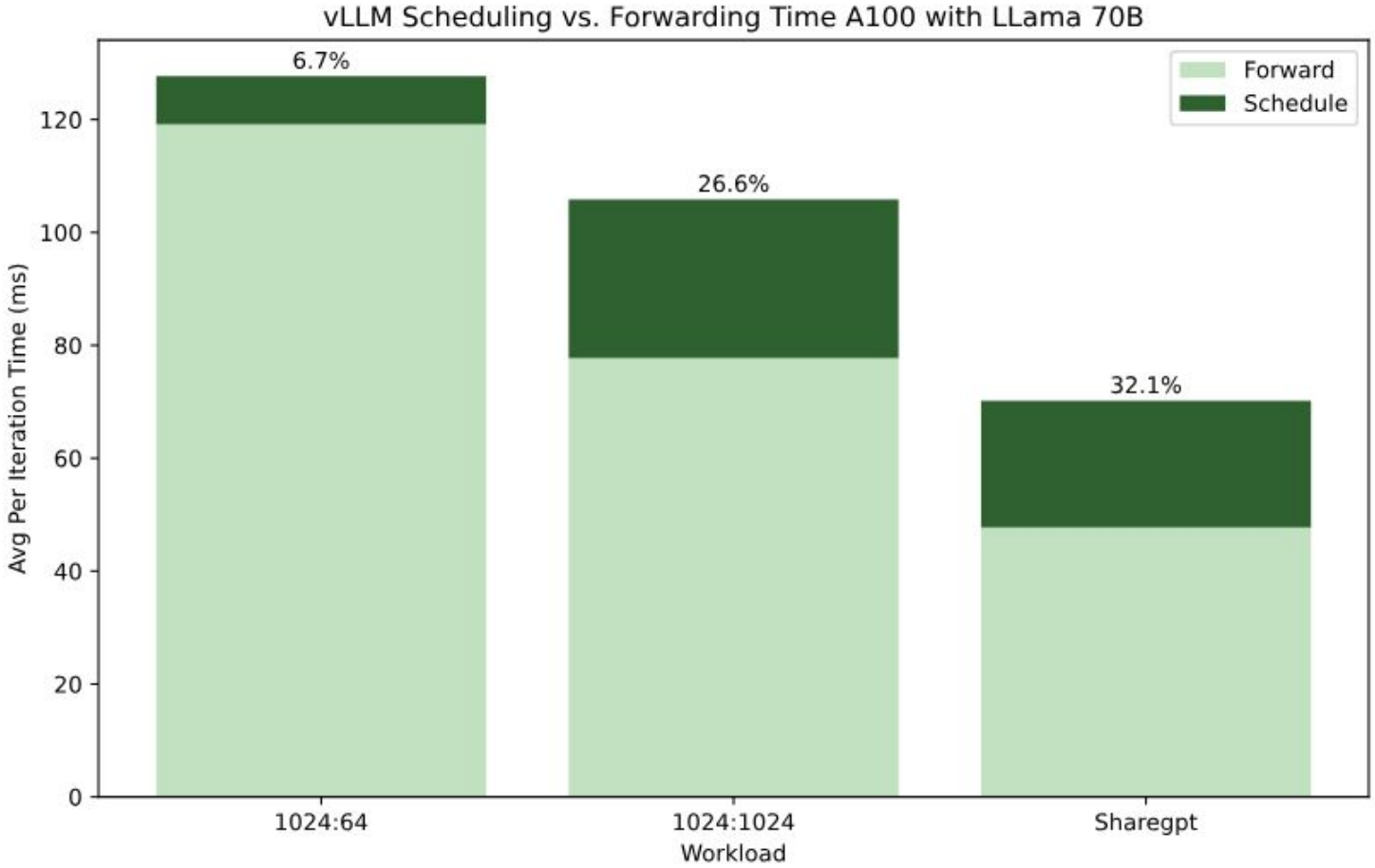
But CPU-side serving work can enter the critical path.

Why can CPU matter in GPU inference?

- Decode steps are small and frequent
- The scheduler runs between steps
- API formatting, input and output processing happen on CPU
- If CPU work is on the critical path, the GPU waits

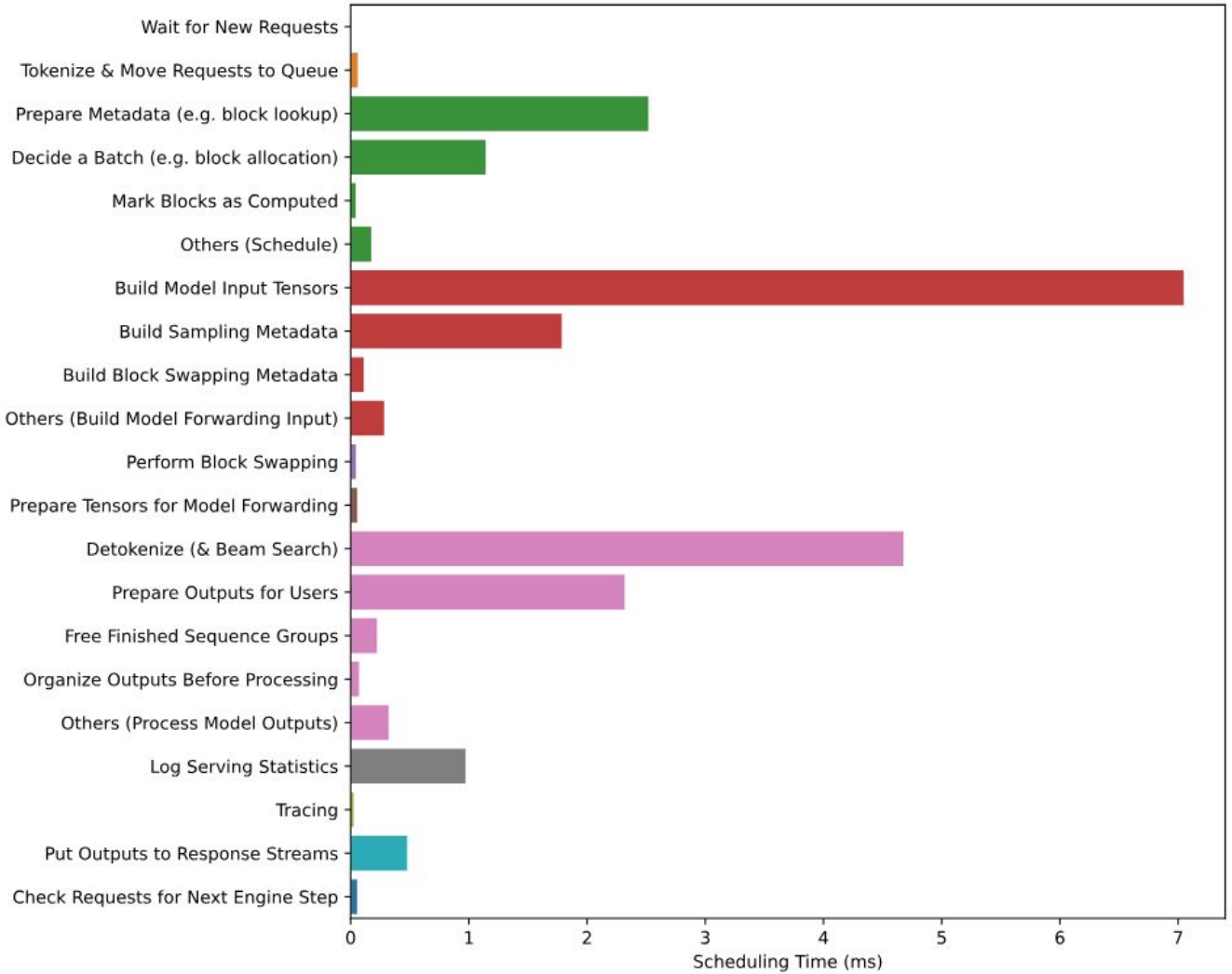


https://mlsys.wuklab.io/posts/scheduling_overhead/



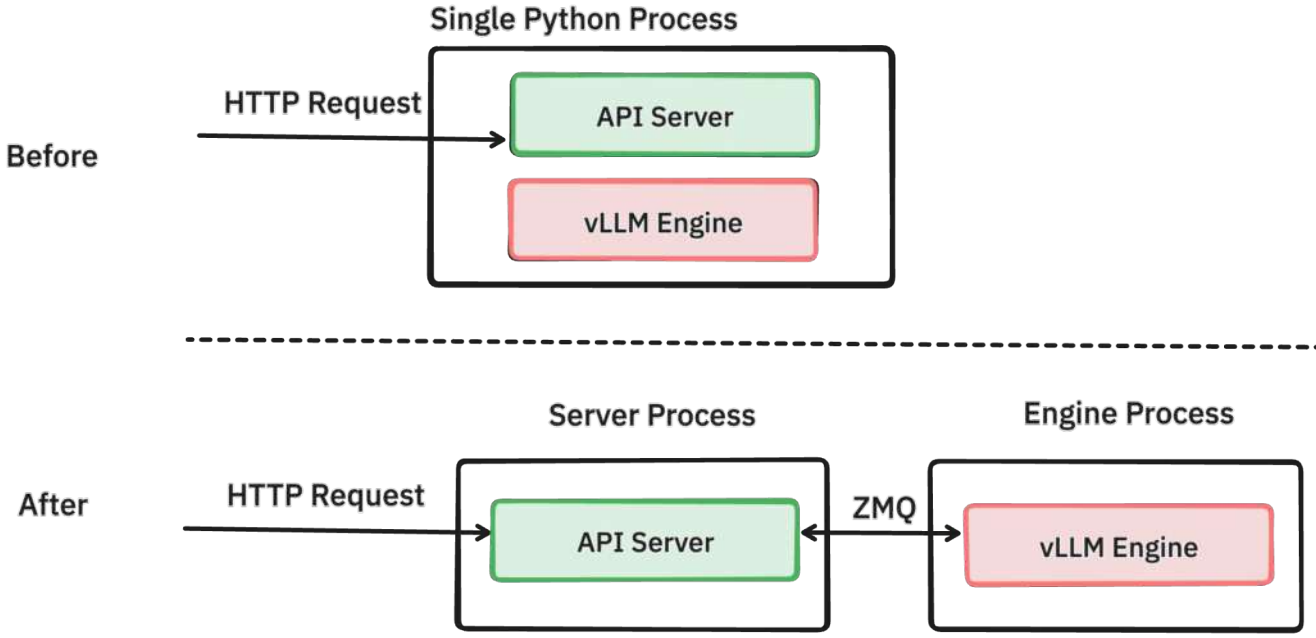
https://mlsys.wuklab.io/posts/scheduling_overhead/

Where did the time go?



Separating API server and inference engine into different processes

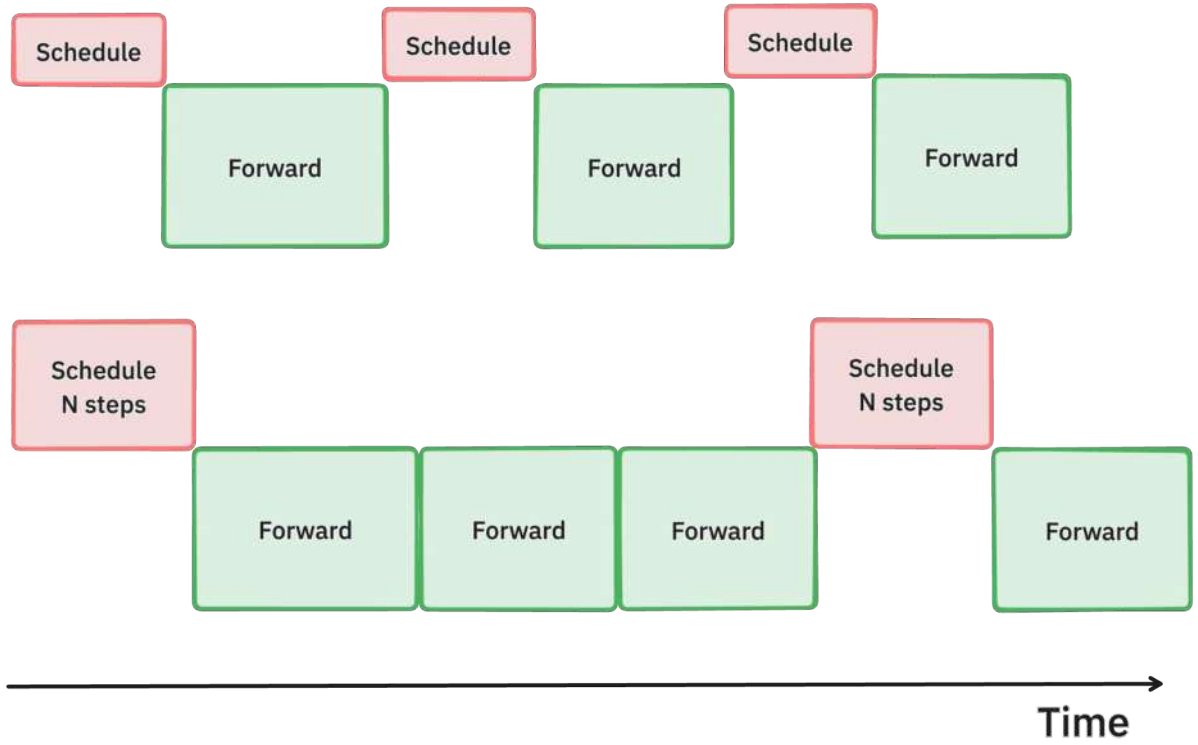
- Request validation, tokenization, JSON formatting, and streaming, ...
- Competition for CPU time / GIL



https://vllm.ai/blog/2024-09-05-perf-update

Batch scheduling multiple steps ahead

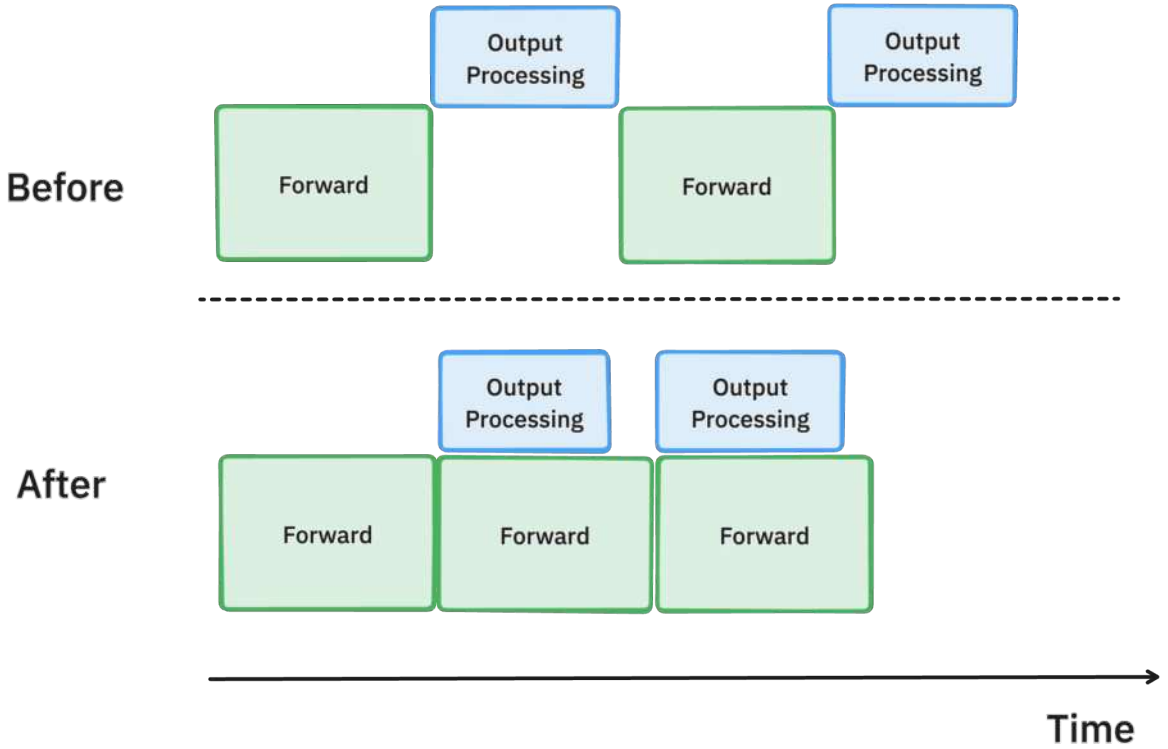
Decode steps are short and frequent, so scheduling every step adds repeated CPU overhead.



https://vllm.ai/blog/2024-09-05-perf-update

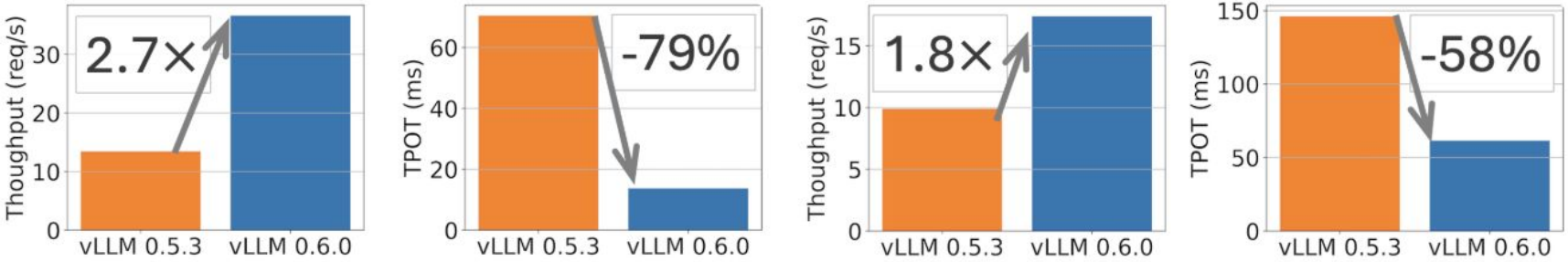
Asynchronous output processing

After each step, the engine must move outputs to CPU, process token IDs, detokenize, and check stop conditions.



https://vllm.ai/blog/2024-09-05-perf-update

Results



Performance comparison between vLLM v0.5.3 and v0.6.0 for Llama 8B on 1xH100 and 70B on 4xH100 on ShareGPT dataset (500 prompts). TPOT measured at 32 QPS.

https://vllm.ai/blog/2024-09-05-perf-update

Another scheduling contention

The GPU may be busy, but not with the latency-critical work.

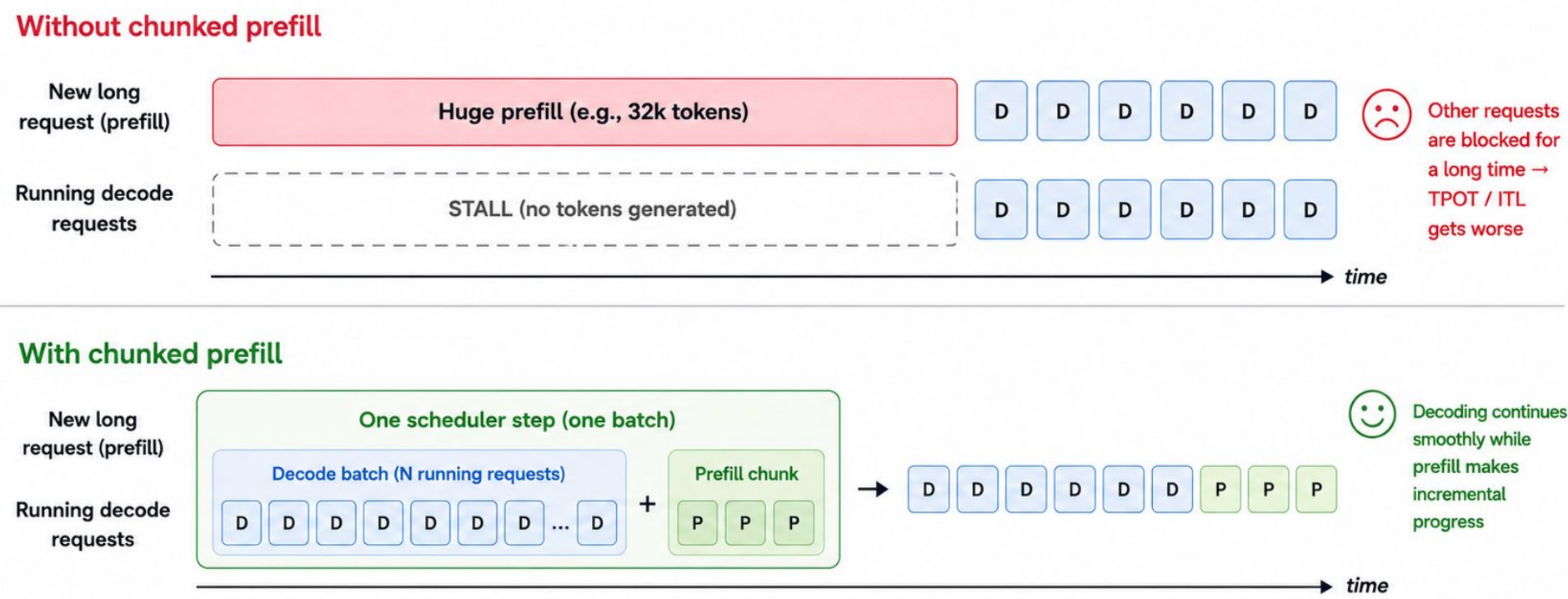
Long prefills create a scheduling conflict

- Prefill is compute-heavy and determines TTFT
- Decode is memory-bound and determines TPOT
- A large prefill can occupy the GPU for a long time
- During that time, running generations stop producing tokens

So, optimizing TTFT for one request can hurt TPOT for everyone else

Chunked Prefill: split large prefills across scheduler steps

Large prefills are split into chunks and scheduled incrementally alongside decode.



D **Decode step**
Generate next token(s) for running requests

P **Prefill chunk**
Process a fixed number of prompt tokens (chunk size)

Key idea
Large prefills are broken into chunks and scheduled across multiple steps.

Chunked Prefill

- Limit the amount of prefill work per scheduler step
- Decode latency becomes more predictable under load
- Long prompts stop monopolizing the GPU
- Main trade-off: TTFT vs TPOT

Performance Tuning (1)

max_num_batched_tokens

- Total token budget per scheduler step
- Decode usually spends ~1 token per running request
- Prefill spends one token per prompt token processed now
- Lower budget: smoother ITL / TPOT
- Higher budget: faster prefill progress and often higher throughput

max_num_seqs

- Max sequences in one scheduler step
- More active decodes → less remaining budget for prefill
- Too high can increase memory pressure

Performance Tuning (2)

max_num_partial_prefills

- Max number of chunked prefills in progress
- Higher: more prompts make progress concurrently
- Lower: less prefill contention with decode

long_prefill_token_threshold

- Defines which prompts count as “long”

max_long_partial_prefills

- Caps how many long prompts can prefill at once
- Lets shorter prompts avoid waiting behind long prefills

Compilation

From scheduling to execution

So far: we optimized which requests run in each scheduler step

But once the scheduler picks a batch, the engine still has to execute the model forward pass

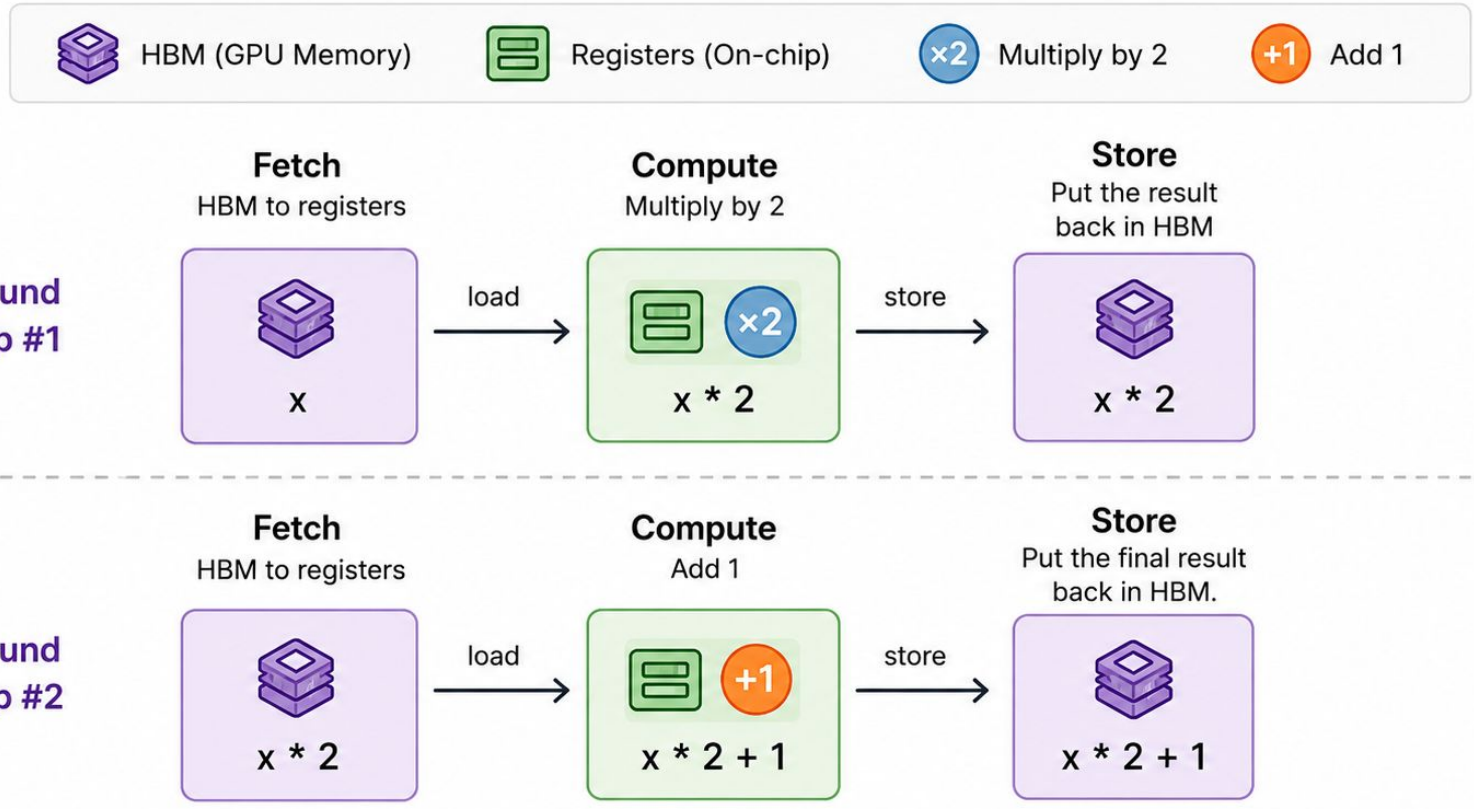
PyTorch Default Mode: Eager Mode

- Immediate line by line execution
- Each op on GPU requires a kernel launch
- Great for research and dynamic code
- But many small kernels mean
  - kernel launch overhead
  - memory transfer overhead

```
x = torch.randn(100)
y = x * 2  # <--- GPU executes this immediately!
print(y)   # <--- Value is ready to inspect instantly
```

PyTorch Eager Mode

x * 2 + 1



What compilation tries to do

- Capture tensor operations into a graph
- Optimize the graph
- Generate faster kernel(s)
- Reuse the compiled version on later calls
- Main wins: less memory traffic, fewer kernel launches, less Python overhead

Example 1

```python
import time
import torch

dev = torch.device("cuda")


def f(x):
    a = torch.sin(x)
    b = torch.cos(x)
    return (a * b + a / (b + 1e-3)).tanh().exp().log1p()


def bench(fn, x, iters=1000, warmup=20):
    for _ in range(warmup):
        fn(x)
    torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(iters):
        fn(x)
    torch.cuda.synchronize()
    return (time.perf_counter() - t0) * 1e6 / iters  # us
```

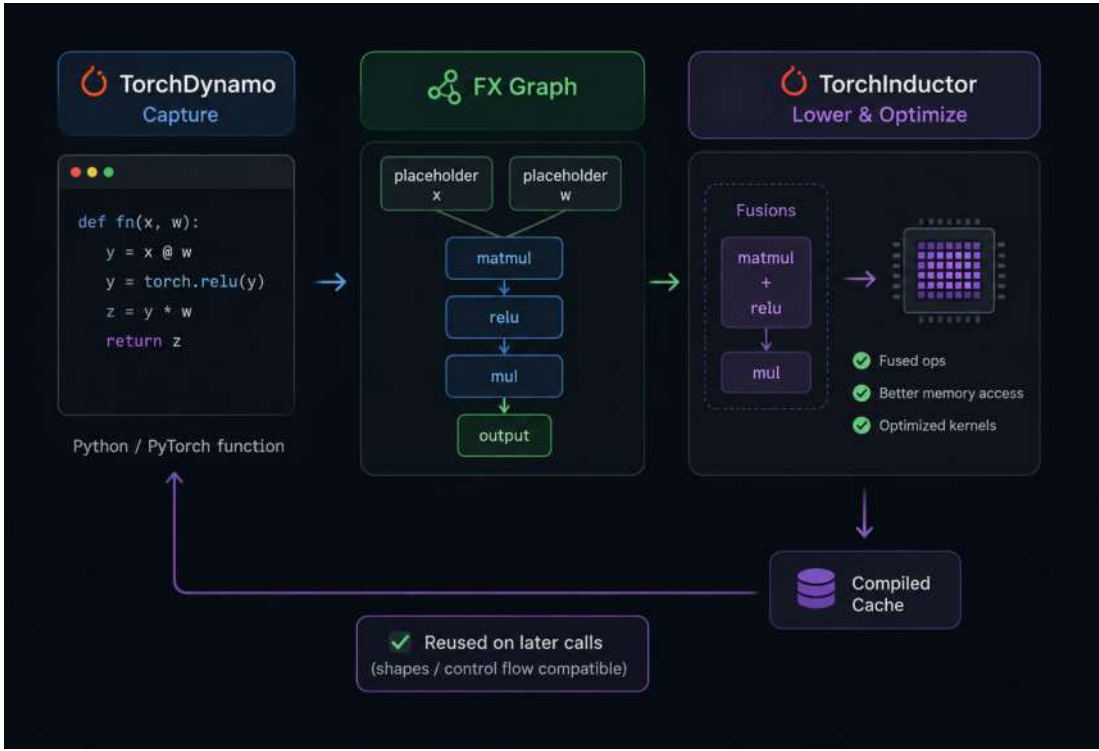
```
eager = f
compiled = torch.compile(f)

print(f"GPU: {torch.cuda.get_device_name(0)}, torch {torch.__version__}\n")
print(f"{'shape':<22}{'eager':>10}{'compiled':>12}{'speedup':>10}")

for shape in [(64, 64), (1024, 1024), (4096, 4096), (8192, 8192)]:
    x = torch.randn(*shape, device=dev)
    t_e = bench(eager, x)
    t_c = bench(compiled, x)
    print(f"{str(shape):<22}{t_e:8.1f}us{t_c:10.1f}us{t_e/t_c:9.2f}x")
```

```python
eager = f
compiled = torch.compile(f)

print(f"GPU: {torch.cuda.get_device_name(0)}, torch {torch.__version__}\n")
print(f"{'shape':<22}{'eager':>10}{'compiled':>12}{'speedup':>10}")

for shape in [(64, 64), (1024, 1024), (4096, 4096), (8192, 8192)]:
    x = torch.randn(*shape, device=dev)
    t_e = bench(eager, x)
    t_c = bench(compiled, x)
    print(f"{str(shape):<22}{t_e:8.1f}us{t_c:10.1f}us{t_e/t_c:9.2f}x")
```

```
GPU: NVIDIA H100 80GB HBM3, torch 2.11.0+cu128

shape                      eager    compiled   speedup
(64, 64)                   47.5us      32.5us     1.46x
(1024, 1024)               47.5us      29.0us     1.64x
(4096, 4096)              495.6us      78.5us     6.31x
(8192, 8192)             1870.8us     302.1us     6.19x
```

How torch.compile works

1. TorchDynamo captures your Python/PyTorch function into an FX graph (PyTorch's intermediate representation)
2. TorchInductor lowers that graph: fuses suitable ops, improves memory access, and generates optimized kernels.
3. The compiled version is cached and reused on later calls, as long as shapes/control flow stay compatible



Pros and Cons

Pros:

- fewer, bigger kernels
- better arithmetic intensity
- fewer round-trips through HBM
- Python-side overhead largely eliminated

Cons:

- compile time
- first call is slower
- recompilation when shapes/branches change
- graph breaks can fall back to eager execution

TorchDynamo output example

```python
def f(x):
    a = torch.sin(x)
    b = torch.cos(x)
    return (a * b + a / (b + 1e-3)).tanh().exp().log1p()
```

```
================================================================================
Stage 1: FX graph (what Dynamo captured from Python)
================================================================================
class GraphModule(torch.nn.Module):
    def forward(self, L_x_: "f32[8, 8]"):
        l_x_ = L_x_

        # File: /home/lex/misc/torch-compile-demo/02_graph.py:19 in f, code: a = torch.sin(x)
        a: "f32[8, 8]" = torch.sin(l_x_)

        # File: /home/lex/misc/torch-compile-demo/02_graph.py:20 in f, code: b = torch.cos(x)
        b: "f32[8, 8]" = torch.cos(l_x_);  l_x_ = None

        # File: /home/lex/misc/torch-compile-demo/02_graph.py:21 in f, code: return (a * b + a / (b + 1e-3)).tanh().exp().log1p()
        mul: "f32[8, 8]" = a * b
        add: "f32[8, 8]" = b + 0.001;  b = None
        truediv: "f32[8, 8]" = a / add;  a = add = None
        add_1: "f32[8, 8]" = mul + truediv;  mul = truediv = None
        tanh: "f32[8, 8]" = add_1.tanh();  add_1 = None
        exp: "f32[8, 8]" = tanh.exp();  tanh = None
        log1p: "f32[8, 8]" = exp.log1p();  exp = None
        return (log1p,)
```

TorchInductor output example

```
def f(x):
    a = torch.sin(x)
    b = torch.cos(x)
    return (a * b + a / (b + 1e-3)).tanh().exp().log1p()
```

```
============================================================
Stage 2: Triton kernel (what Inductor generated)
============================================================
@triton.jit
def triton_poi_fused_add_cos_div_exp_log1p_mul_sin_tanh_0(in_ptr0, out_ptr0, xnumel, XBLOCK : tl.constexpr):
xnumel = 64
xoffset = tl.program_id(0) * XBLOCK
xindex = xoffset + tl.arange(0, XBLOCK)[:]
xmask = xindex < xnumel
x0 = xindex
tmp0 = tl.load(in_ptr0 + (x0), xmask)
tmp1 = tl_math.sin(tmp0)
tmp2 = tl_math.cos(tmp0)
tmp3 = tmp1 * tmp2
tmp4 = tl.full([1], 0.001, tl.float32)
tmp5 = tmp2 + tmp4
tmp6 = (tmp1 / tmp5)
tmp7 = tmp3 + tmp6
tmp8 = libdevice.tanh(tmp7)
tmp9 = libdevice.exp(tmp8)
tmp10 = libdevice.log1p(tmp9)
tl.store(out_ptr0 + (x0), tmp10, xmask)
```

Graph Break Example

```python
def f_clean(x):
    return (x.sin() * x.cos() + x.tanh()).exp().log1p()


def f_breaky(x):
    y = x.sin() * x.cos()
    if y.abs().sum().item() > 0:
        y = y + x.tanh()
    else:
        y = y + x.tanh()
    print(f"  [side effect] y.shape={tuple(y.shape)}")
    return y.exp().log1p()
```

Graph Break Example

```python
def f_clean(x):
    y = x.sin() * x.cos()
    y = y + x.tanh()
    return y.exp().log1p()
```

```python
def f_clean(x):
    return (x.sin() * x.cos() + x.tanh()).exp().log1p()


def f_breaky(x):
    y = x.sin() * x.cos()
    if y.abs().sum().item() > 0:
        y = y + x.tanh()
    else:
        y = y + x.tanh()
    print(f"  [side effect] y.shape={tuple(y.shape)}")
    return y.exp().log1p()
```

Same?

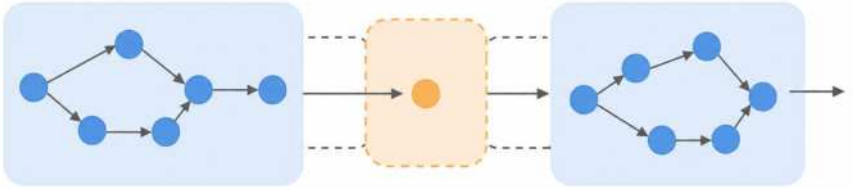
```
torch.manual_seed(0)
x = torch.randn(1024, 1024, device=dev)

print("=" * 70)
print("torch._dynamo.explain() on f_breaky")
print("=" * 70)
exp = torch._dynamo.explain(f_breaky)(x)
print(f"graphs={exp.graph_count}  breaks={exp.graph_break_count}  "
      f"captured ops={exp.op_count}")
for i, reason in enumerate(exp.break_reasons, 1):
    print(f"\nbreak #{i}: {reason.reason}")
    for line in reason.user_stack:
        print(f"  at {line.filename.split('/')[-1]}:{line.lineno}  {line.line}")
```

```
======================================================================
torch._dynamo.explain() on f_breaky
======================================================================
  [side effect] y.shape=(1024, 1024)
graphs=3  breaks=2  captured ops=2

break #1: Unsupported Tensor.item() call with capture_scalar_outputs=False
  Explanation: Dynamo does not support tracing `Tensor.item()` with config.capture_scalar_outputs=False.
  Hint: Set `torch._dynamo.config.capture_scalar_outputs = True` or `export TORCHDYNAMO_CAPTURE_SCALAR_OUTPUTS=1` to include these operations in the captured graph.

  Developer debug context: call_method TensorVariable() item () {}

 For more details about this graph break, please visit: https://meta-pytorch.github.io/compile-graph-break-site/gb/gb0124.html
  at 03_graph_breaks.py:27  if y.abs().sum().item() > 0:

break #2: Failed to trace builtin operator
  Explanation: Dynamo does not know how to trace builtin operator `print` with argument types ['str'] (has_kwargs False)
  Hint: Avoid calling builtin `print` with argument types ['str']. Consider using an equivalent alternative function/method to `print`.
  Hint: If you are attempting to call a logging function (e.g. `print`), you can try adding it to `torch._dynamo.config.reorderable_logging_functions`.
  Hint: Please report an issue to PyTorch.

  Developer debug context: builtin print [<class 'torch._dynamo.variables.constant.ConstantVariable'>] False

 For more details about this graph break, please visit: https://meta-pytorch.github.io/compile-graph-break-site/gb/gb0059.html
  at 03_graph_breaks.py:31  print(f"  [side effect] y.shape={tuple(y.shape)}")
```

```python
def bench(fn, x, iters=200, warmup=20):
    import contextlib, io, sys
    with contextlib.redirect_stdout(io.StringIO()):  # mute the print() side effect
        for _ in range(warmup):
            fn(x)
        torch.cuda.synchronize()
        t0 = time.perf_counter()
        for _ in range(iters):
            fn(x)
        torch.cuda.synchronize()
    return (time.perf_counter() - t0) * 1e6 / iters


print("\n" + "=" * 70)
print("Cost of graph breaks (same math, two structures)")
print("=" * 70)
torch._dynamo.reset()
clean_c = torch.compile(f_clean)
breaky_c = torch.compile(f_breaky)
t_clean = bench(clean_c, x)
t_breaky = bench(breaky_c, x)
print(f"compiled f_clean  (1 subgraph) : {t_clean:7.1f} us")
print(f"compiled f_breaky ({exp.graph_count} subgraphs): {t_breaky:7.1f} us")
print(f"breaks cost ~{t_breaky - t_clean:.1f}us / call ({t_breaky/t_clean:.1f}x slower)")
```

```
================================================================================
Cost of graph breaks (same math, two structures)
================================================================================
compiled f_clean  (1 subgraph) :    25.2 us
compiled f_breaky (3 subgraphs):   110.7 us
breaks cost ~85.5us / call (4.4x slower)
```

Graph breaks summary

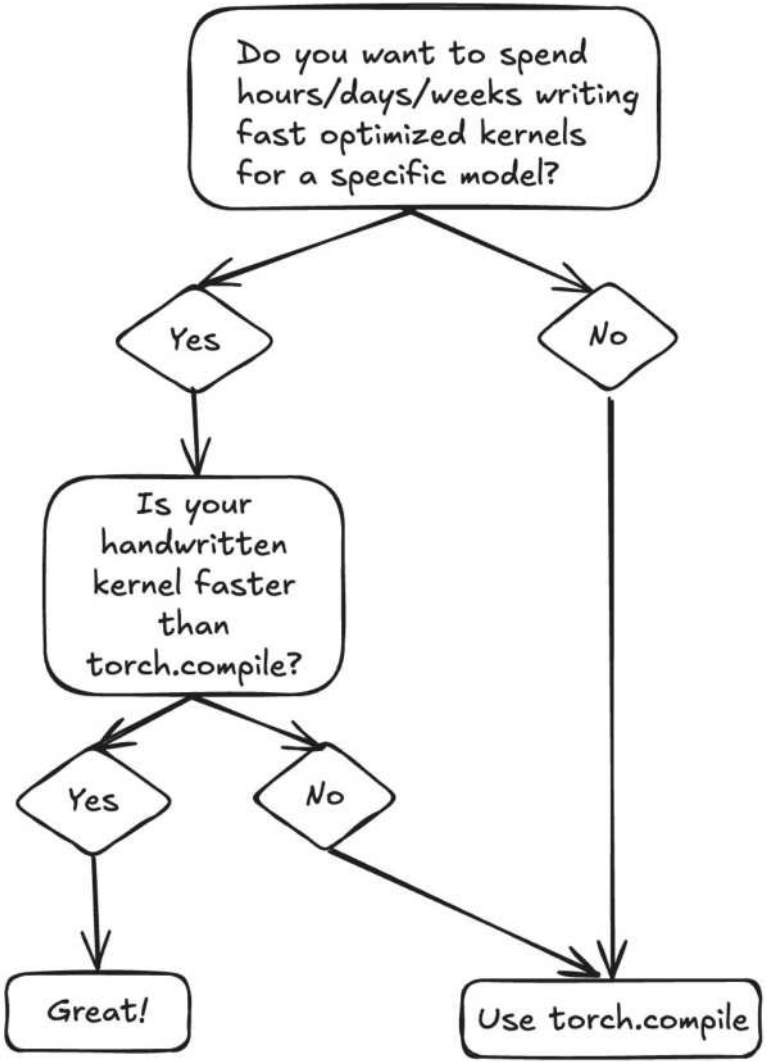


Happen when code is hard to trace:

- data-dependent Python control flow
- calls into non-PyTorch libraries
- converting tensors into Python values
- unsupported custom operations

The function is split into compiled subgraphs with the unsupported op run in Eager mode between them.

Practical note: if you pass `fullgraph=True` to `torch.compile`, breaks become hard errors.


Do you want to spend hours/days/weeks writing fast optimized kernels for a specific model?
Yes
No
Is your handwritten kernel faster than torch.compile?
Yes
No
Great!
Use torch.compile

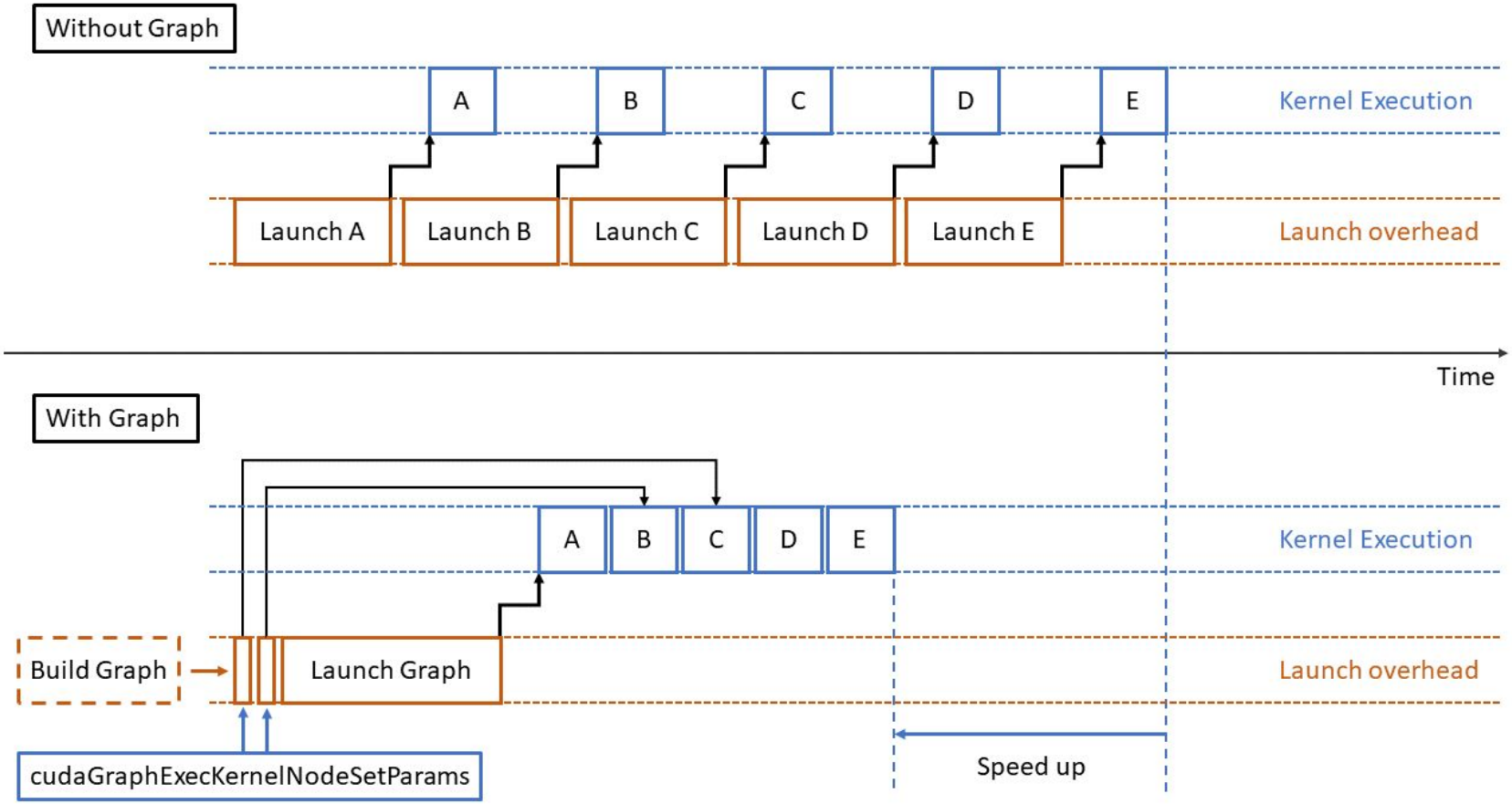
CUDA Graphs

torch.compile gave us fused kernels.

But the CPU still launches them, one by one.

CUDA graphs: record once, replay many times

- Each launch costs ~5–50 µs of CPU overhead
- CUDA graphs record the full launch sequence once
- Replay it later with one CPU call



https://developer.nvidia.com/blog/constructing-cuda-graphs-with-dynamic-parameters/

Note on FX Graphs in torch.compile

FX graph:

- *what computation to do, before kernels are picked*
- graph of PyTorch ops
- compiler's IR (intermediate representation)
- used for codegen, then thrown away

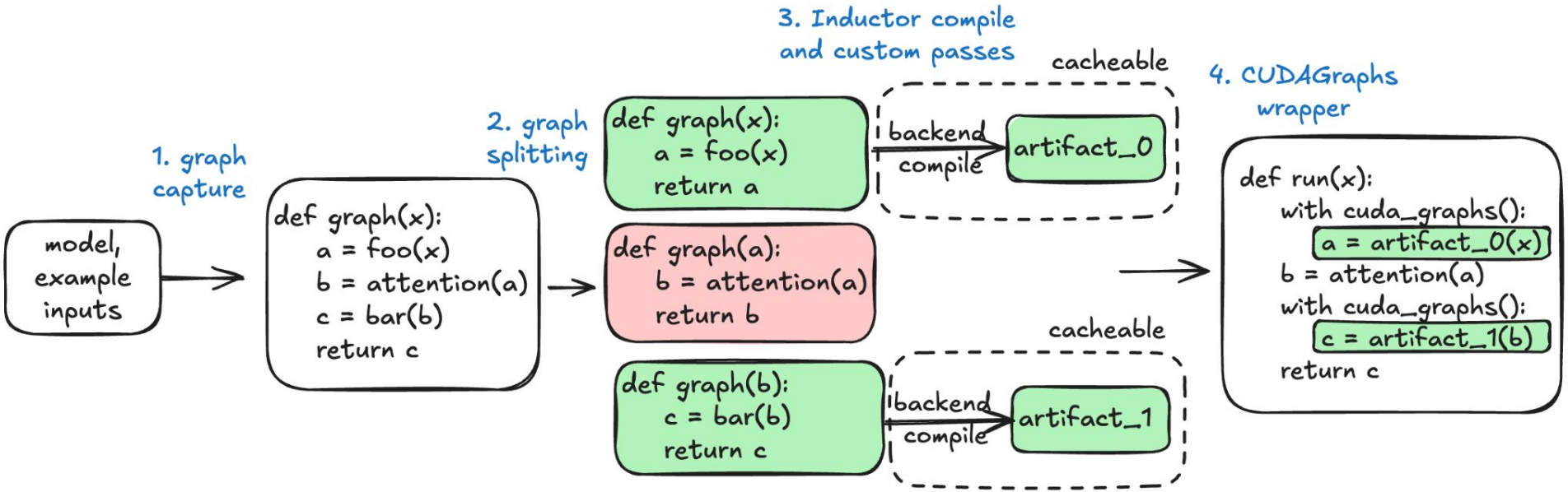
CUDA graph

- what specific GPU calls to make, after kernels are picked
- a graph of kernel launches with concrete memory addresses
- captured once, replayed every step

torch.compile optimizes what runs

CUDA graphs optimize how it gets launched

torch.compile and CUDA Graphs in vLLM



vLLM’s args

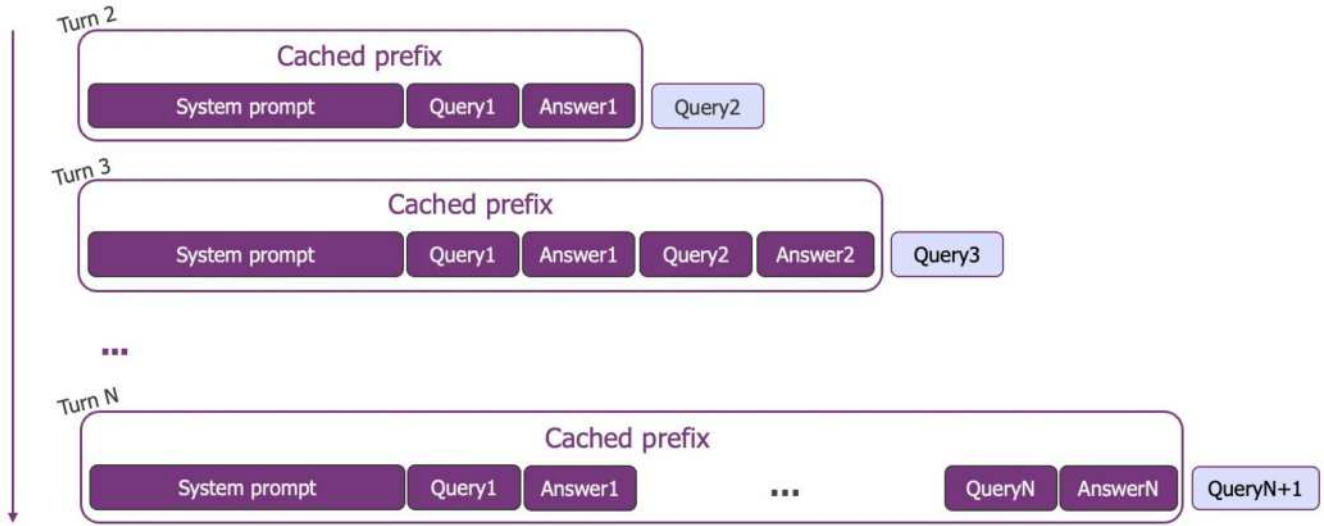
- enforce_eager=True - off switch. Disables both.
- compile_sizes=[1, 2, 4] - specialize compilation at these batch sizes (better kernels, slower start). Cost: slower startup
- cudagraph_capture_sizes=[1, 2, 4, 8, ...] - record CUDA graphs at these batch sizes (skip launch overhead). Cost: slower startup and GPU memory

Prefix Caching

Can we just avoid the computation?

Repeated Prefills

- Recomputing the same prefill wastes FLOPs
- Especially common in:
  - chat
  - agents
  - few-shot prompting

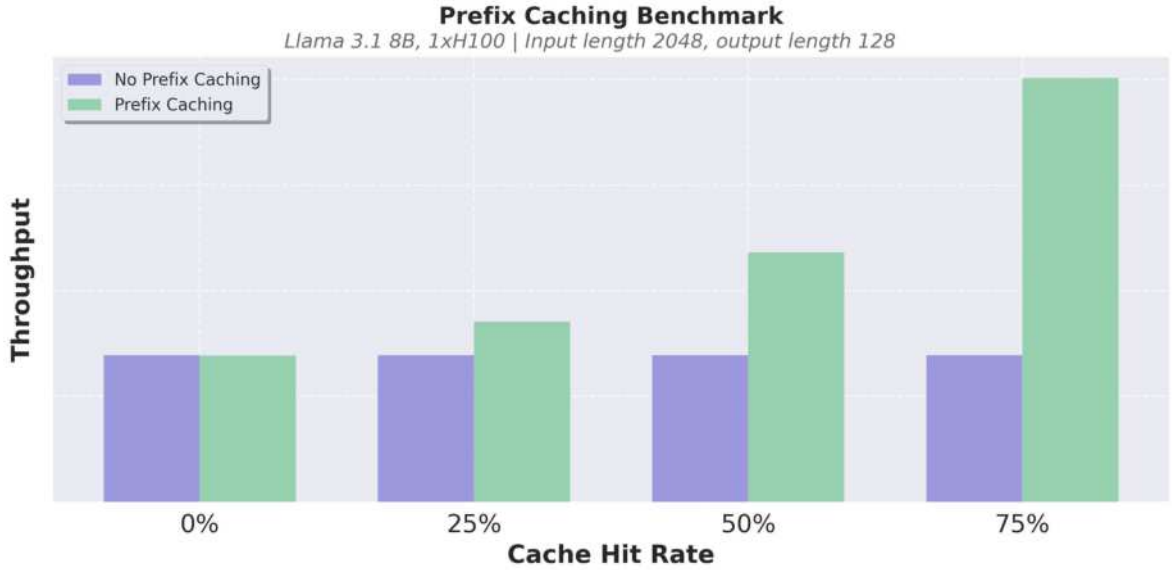


https://llm-d.ai/blog/kvcache-wins-you-can-see

Prefix Caching Idea

- Store KV-cache for previously seen prefixes
- Reuse it instead of recomputing prefill
- Huge TTFT win

Near zero overhead for Prefix Caching in vLLM

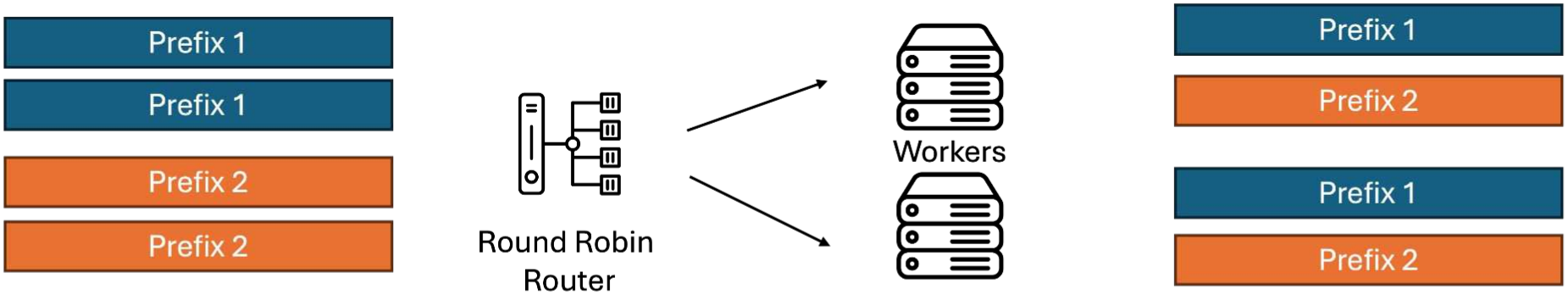


KV-cache - intra-request

Prefix Caching - inter-request

KV cache	Prefix caching
Decode reuses this request's KV cache.	Prefix caching reuses an earlier request's KV cache.
Avoids recomputing previous tokens during generation.	Avoids recomputing shared prompt prefix.
Makes autoregressive decoding practical.	Reduces TTFT for repeated prefixes.

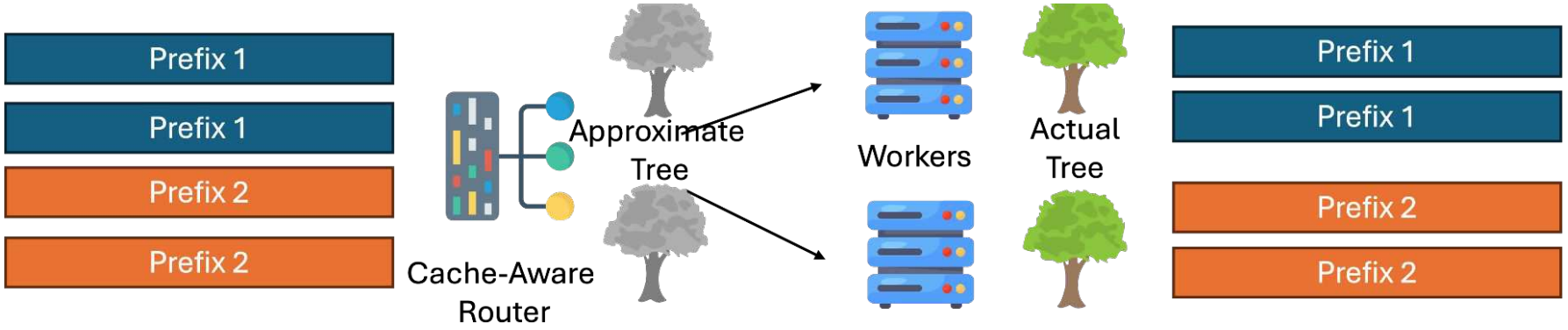
The challenge for scale up



https://www.lmsys.org/blog/2024-12-04-sglang-v0-4/

Prefix-cache-aware routing

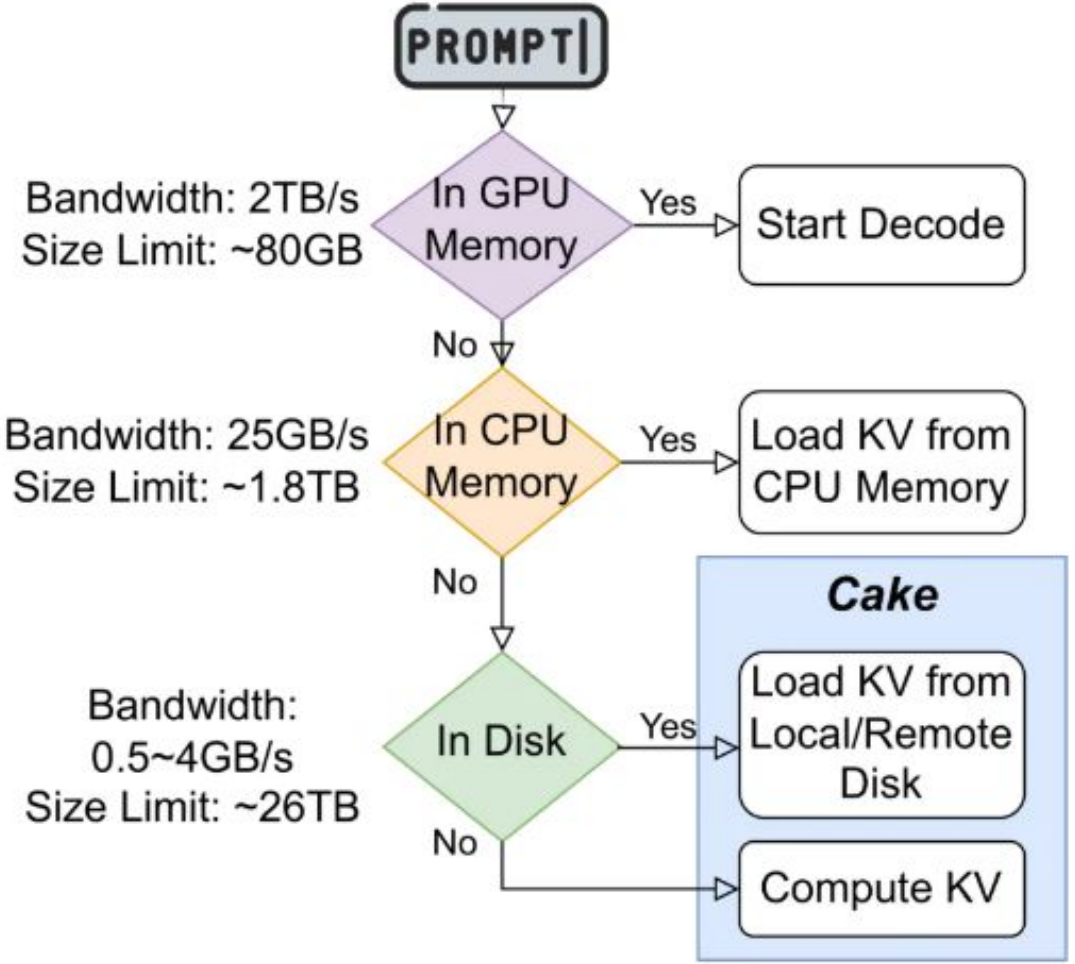
- Route requests to replicas with matching cached KV
- Trade-off cache hits vs load



https://www.lmsys.org/blog/2024-12-04-sglang-v0-4/

Distributed KV-cache hierarchy

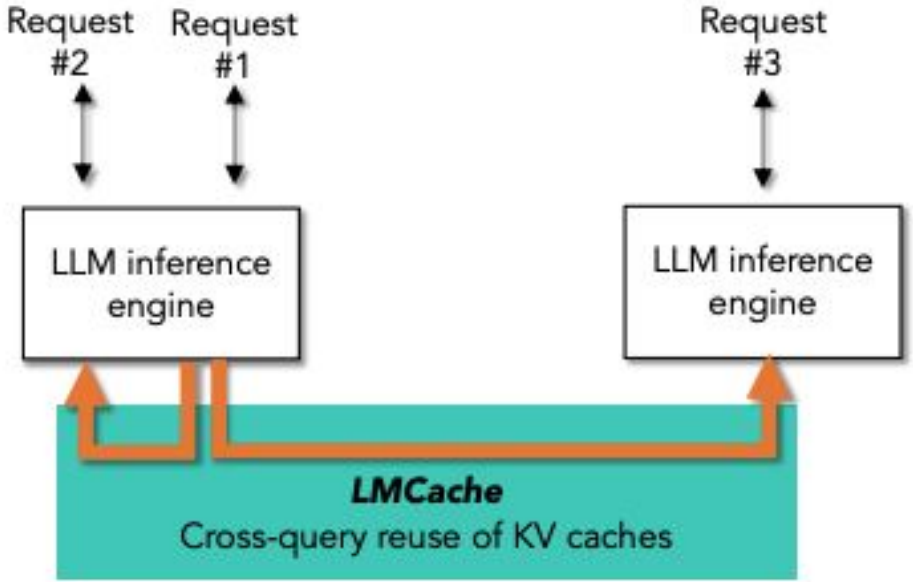
- Make the cache available beyond one GPU/replica
- Helps cache reuse, but adds network traffic and complexity



(Long-context LLM inference workflow with prefix caching)

LMCache

- External KV-cache layer beyond one engine instance
- Configurable storage backends: CPU, disk, remote, ...
- Trade-off: lower prefill cost vs KV transfer + system complexity



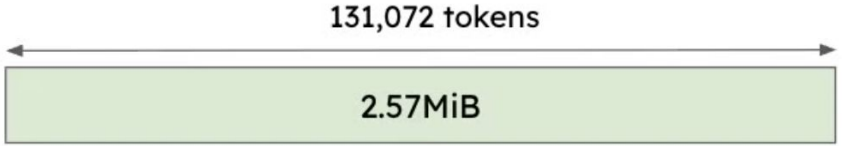
(a) Context caching: Cross-query reusing via CPU/disk offloading of KV cache

Note on hybrid models

Attention layer

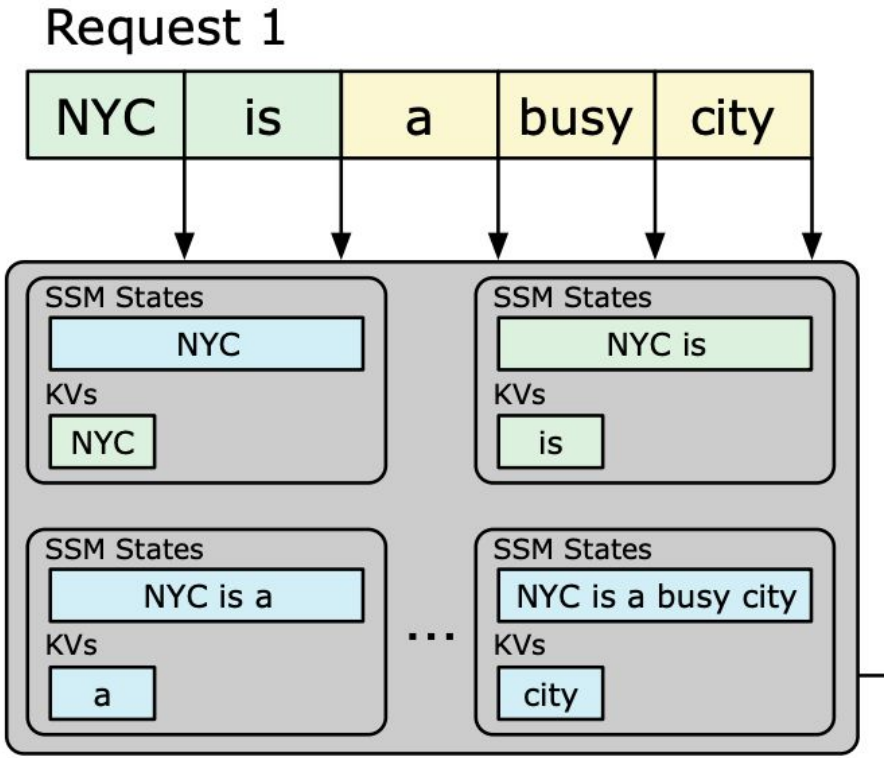
- Attention state = KV Cache
- Every new token → Concatenate
- Each block of 16 tokens = 64KiB

Mamba layer



- Mamba state = Conv + SSM State
- Every new token → Update in place
- State for entire sequence = 2.57 MiB

Note on hybrid models



When using APIs

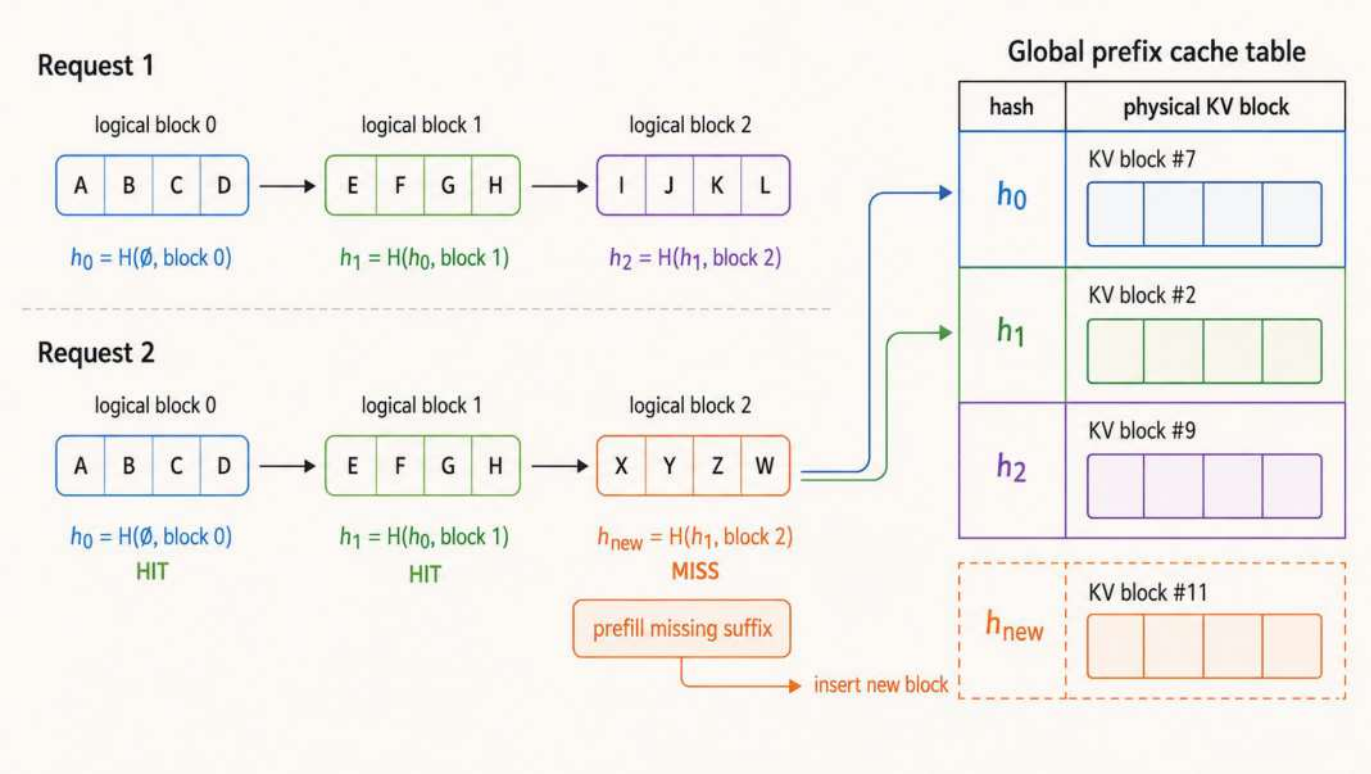
Cached input is often ~10× cheaper than normal input, so it’s worth optimizing for:

- Put stable content first: system prompt, tool schemas, examples, static RAG templates.
- Put dynamic/user-specific content last.
- Keep serialization deterministic: same order, same whitespace, same JSON keys.
- Avoid injecting timestamps, random IDs, or changing metadata into the prefix.
- Monitor and analyze!


SGL

vLLM caches prefix-addressed KV blocks

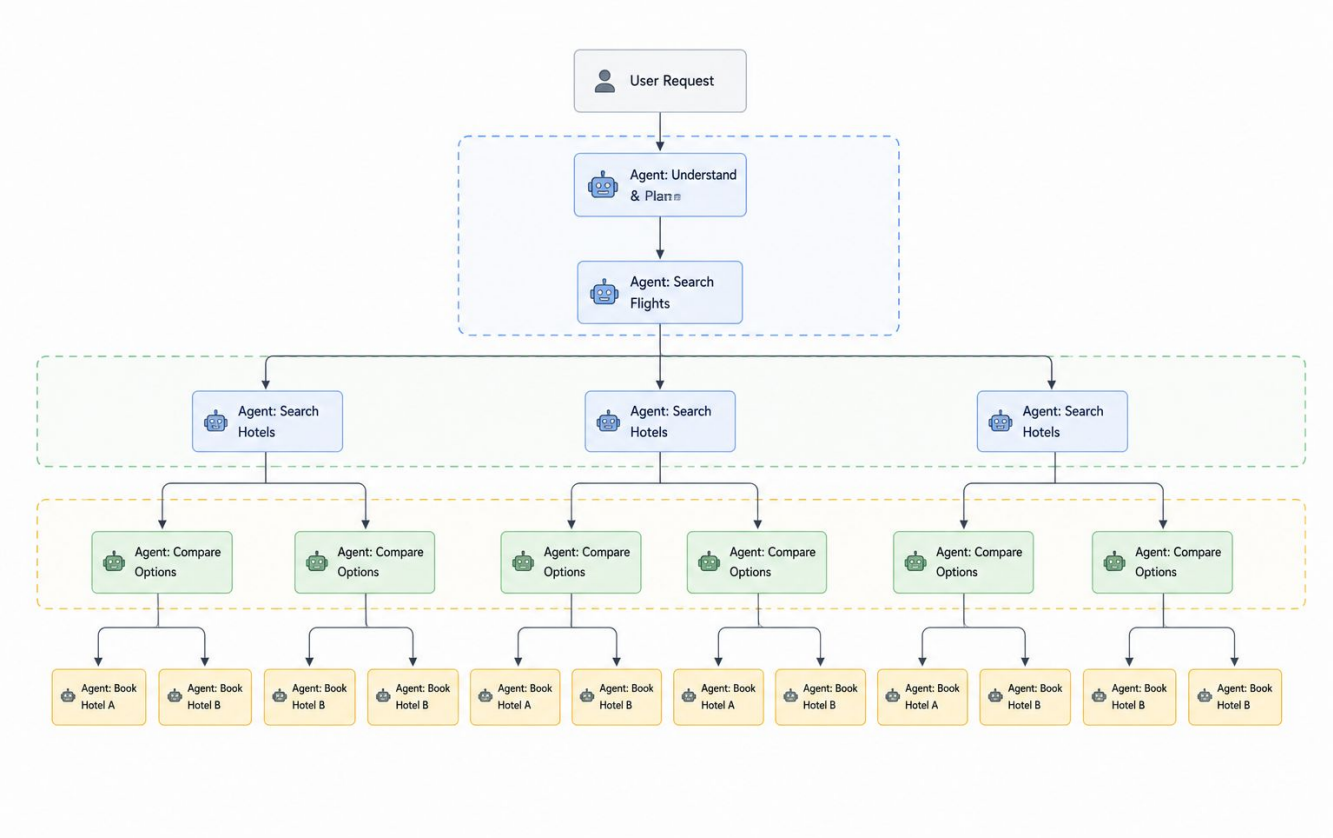
- Split prompt KV into fixed-size blocks
- Hash each full block with its parent prefix hash
- Reuse matching blocks
- Prefill only the missing suffix



Limitations

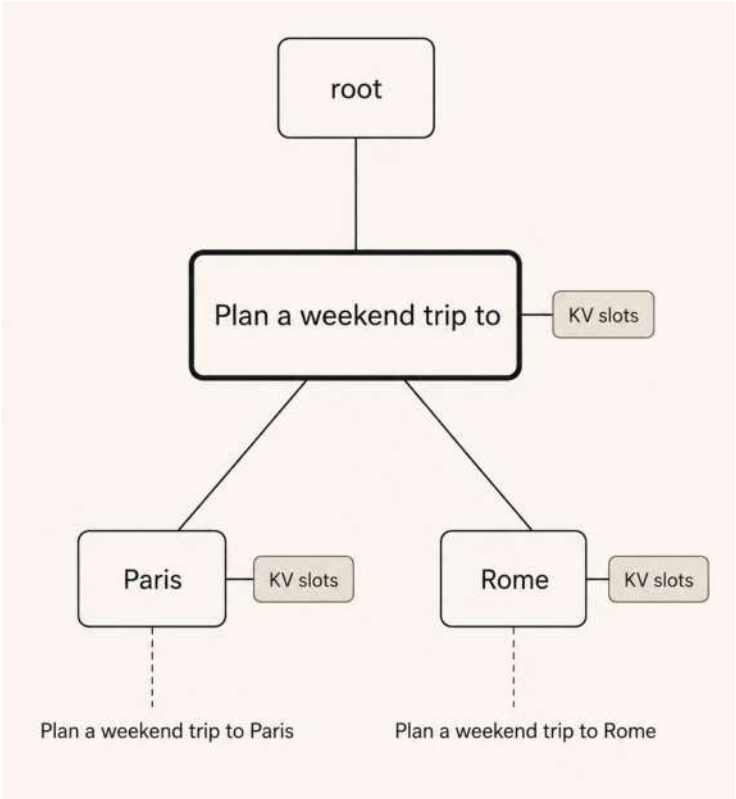
- Block boundaries - the cache is block granular
- Branchy workloads - agents create prefix trees, not just repeated strings
  - The valuable part is the shared trunk
  - The disposable part is often the leaves
- Eviction is per-block LRU: can evict a shared prefix that's about to be reused by many requests.
- Scheduling is cache-blind: a request that would hit a 10k-token cache waits behind one with zero overlap.

The workflow the Sglang was built for

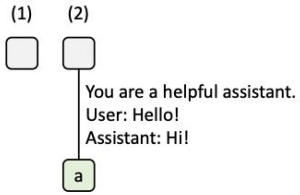


SGLang makes the prefix tree first-class

- Store KV cache as a radix tree over token sequences
- Shared trunks are stored once
- Branches fork exactly where tokens diverge
- The tree guides prefix matching, eviction, and scheduling

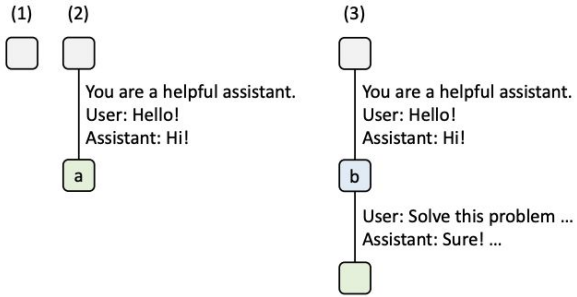


Radix Attention - Prefix Cache as a Tree



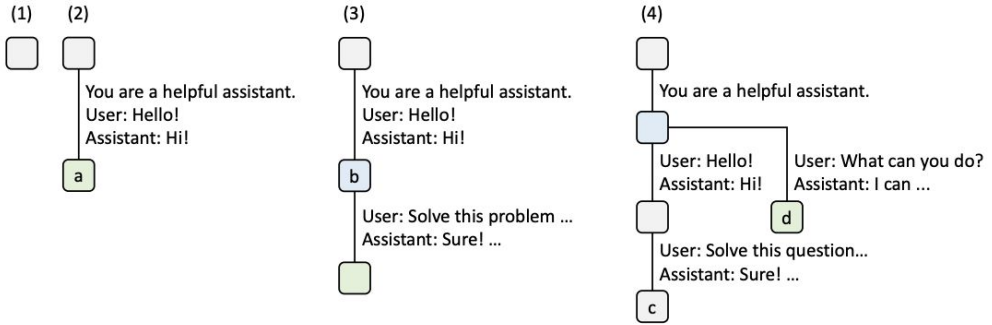
https://arxiv.org/pdf/2312.07104

Radix Attention - Prefix Cache as a Tree



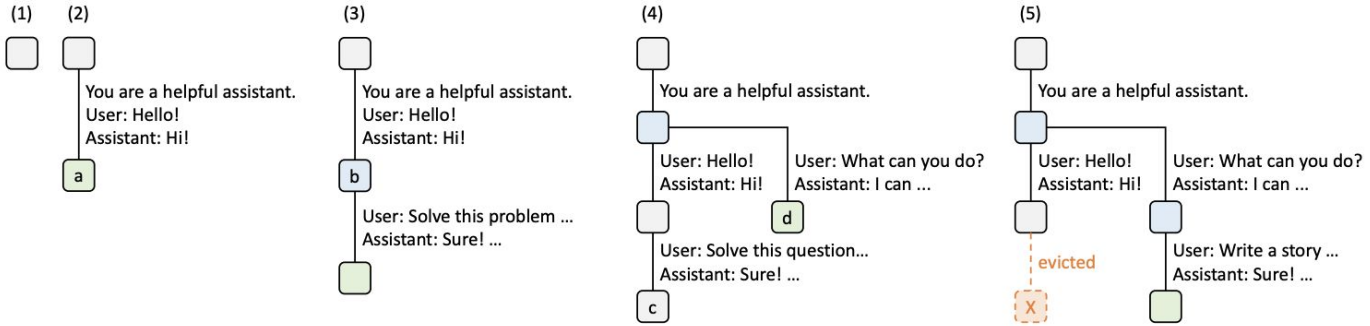
https://arxiv.org/pdf/2312.07104

Radix Attention - Prefix Cache as a Tree



https://arxiv.org/pdf/2312.07104

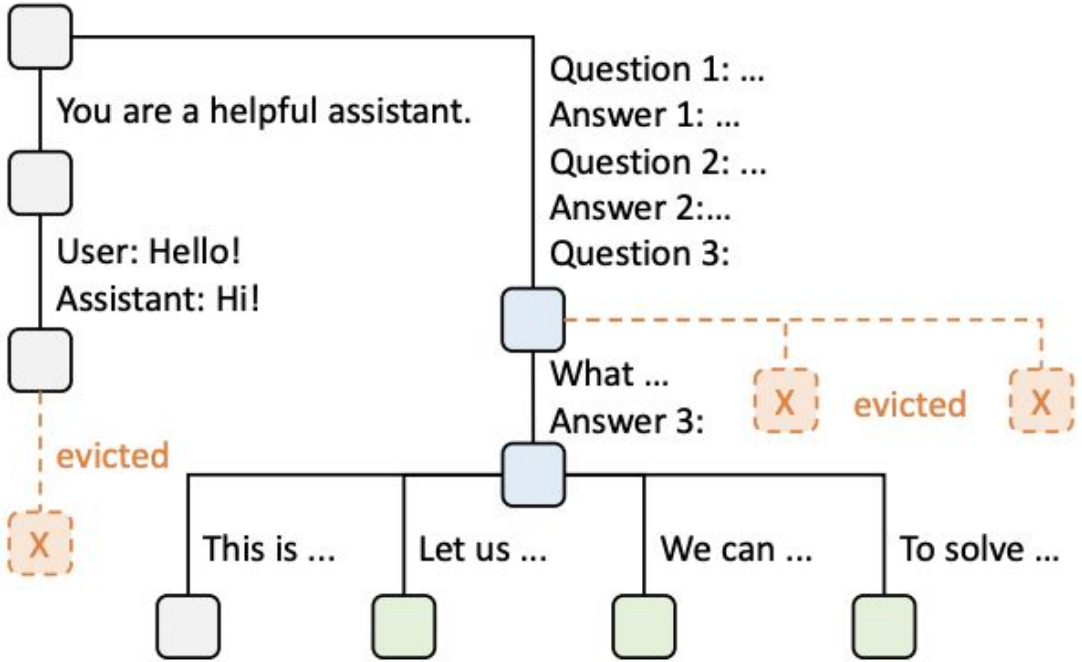
Radix Attention - Prefix Cache as a Tree



https://arxiv.org/pdf/2312.07104

Radix Attention handles complex reuse patterns

Efficient prefix matching, insertion, eviction



https://arxiv.org/pdf/2312.07104

Takeaways

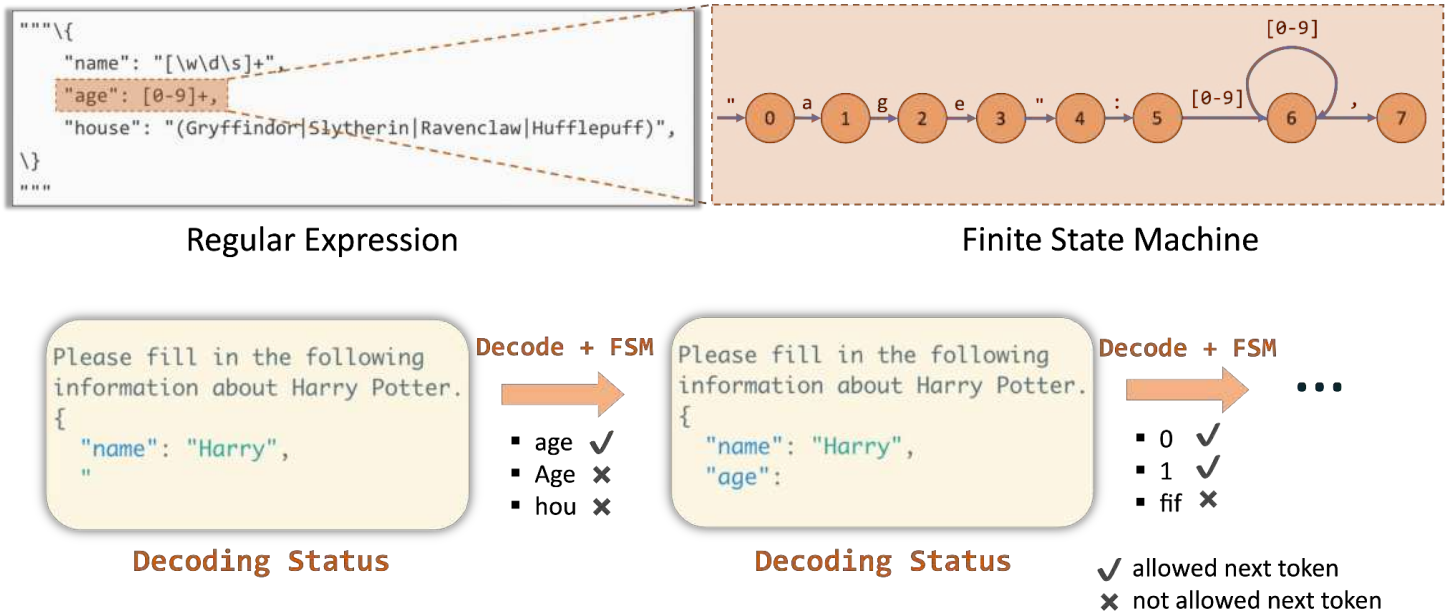
- vLLM caches prefix-addressed KV blocks
- Simple repeated prefixes → block hashing works very well
- SGLang makes the prefix tree explicit
- Branchy workloads → a prefix tree exposes more reuse
- Eviction can favor shared trunks over disposable leaves
- Scheduling can prioritize requests with large cache hits

Constrained Decoding

- Sometimes we need to generate JSON
- Supported both in vllm and sglang

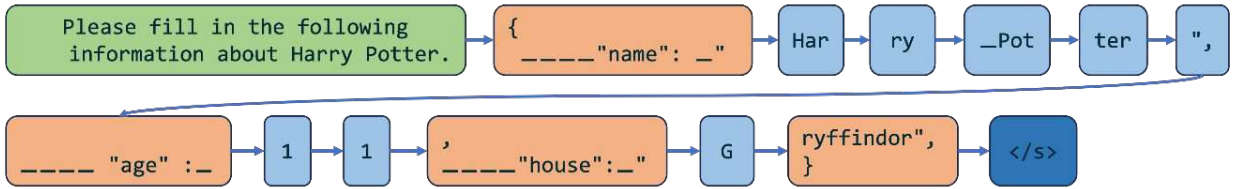
```python
class ClassificationOutput(BaseModel):
    overall_assessment: str
    safety_review: str
    overall_score: int
    safety_score: int
```

JSON Schema → RegExp → Finite State Machine → Logits Mask

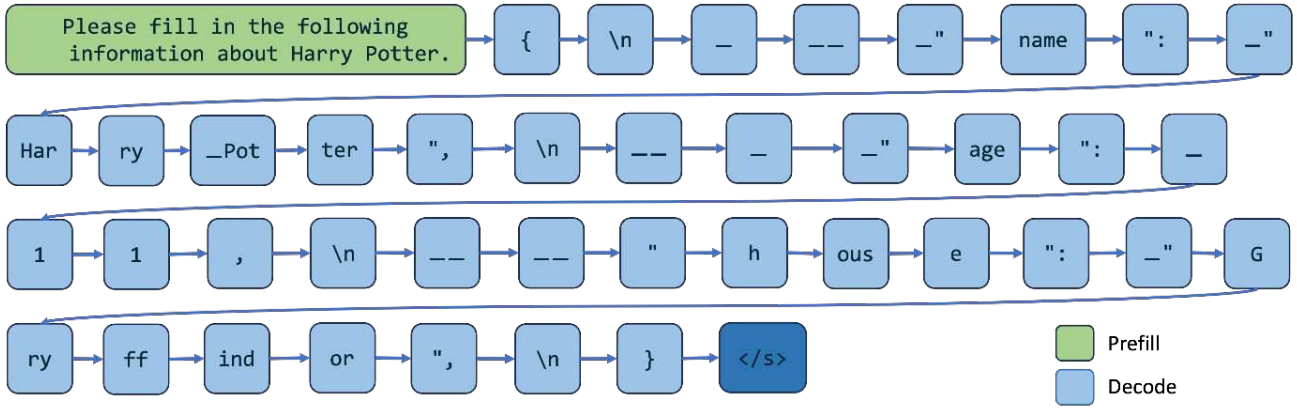


Constrained Decoding With Logits Mask

Compressed FSM for Multi-Token Decoding



Jump-Forward Decoding With Compressed FSM



Normal Decoding With FSM

```
Please fill in the following
information about Harry Potter.
{
  "name": "Harry",
  "age": 15,
  "house": "Gryffindor"
}
```

```
Please fill in the following
information about Harry Potter.
{
  "name": "Harry",
  "age": 15,
  "house": "Gryffindor"
}
```

Generated JSONs

https://www.lmsys.org/blog/2024-02-05-compressed-fsm/

SGLang

Outlines + vLLM

Frontend Language

- Write structured LM programs, not just single API calls
- Compose prompts, generation, branching, and parallel calls

```
@function
def tip_suggestion(s):
    s += assistant(
        "Here are two tips for staying healthy: "
        "1. Balanced Diet. 2. Regular Exercise.\n\n"
    )

    forks = s.fork(2)
    for i, f in enumerate[Any](forks):
        f += assistant(
            f"Now, expand tip {i+1} into a paragraph:\n"
            + gen("detailed_tip", max_tokens=256, stop="\n\n")
        )

    s += assistant("Tip 1:" + forks[0]["detailed_tip"] + "\n")
    s += assistant("Tip 2:" + forks[1]["detailed_tip"] + "\n")
    s += assistant(
        "To summarize the above two tips, I can say:\n" + gen("summary", max_tokens=512)
    )


state = tip_suggestion()
```

Which one to use?

Which one to use?
Benchmark your workflow!

Which one to use?

vLLM:

- Serving dense models
- You need many LoRA adapters or active community plugins.
- Your team wants the most stable, best-documented serving stack.
- Workload prefixes are short, diverse, and don't repeat much.

SGlang:

- Serving wide MoE (DeepSeek-V3, Kimi K2)
- Agents, chat, RAG - workloads where the same prefix is sent dozens of times per second.
- You want the SGL DSL - parallel branches, forks, joins co-scheduled by the runtime.
- Long, branchy prompts where token-level prefix matching pays off.

Alternative Stacks

TensorRT-LLM + Triton Inference Server

Three levels of abstraction in NVIDIA's inference stack:

- **TensorRT-LLM** - optimized LLM compiler and runtime. Turns model weights into a fast NVIDIA-specific engine.
- **Triton Inference Server -** production serving layer. Exposes the engine through HTTP/gRPC, manages model loading, versions, metrics.
- **NIM -** packaged inference microservice. Prebuild container with model + runtime + API.

Typical Workflow

```
# 1. convert HF checkpoint → TRT-LLM format
python convert_checkpoint.py \
  --model_dir ./Llama-3-70B \
  --output_dir ./ckpt \
  --dtype float8 --tp_size 4

# 2. compile → engine binary (~30 min)
trtllm-build --checkpoint_dir ./ckpt \
  --max_batch_size 64 \
  --max_input_len 8192 \
  --max_seq_len   16384 \
  --output_dir    ./engine

# 3. serve (Triton, or wrap as NIM)
tritonserver --model-repo ./engine
```

Change GPU, dtype, TP size, max batch, or max sequence length → rebuild.

Pros & Cons

Pros:

- Often best raw performance on NVIDIA GPUs
- Strong optimization stack: fused kernels, quantization, CUDA graphs, etc
- Integrates well with NVIDIA production tooling: Triton, NIM, metrics

Cons:

- Narrower / slower model support than vLLM or SGLang
- Build + rebuild loop slows iteration
- Engine is tied to deployment assumptions
- More operational complexity: artifacts, containers, Triton config

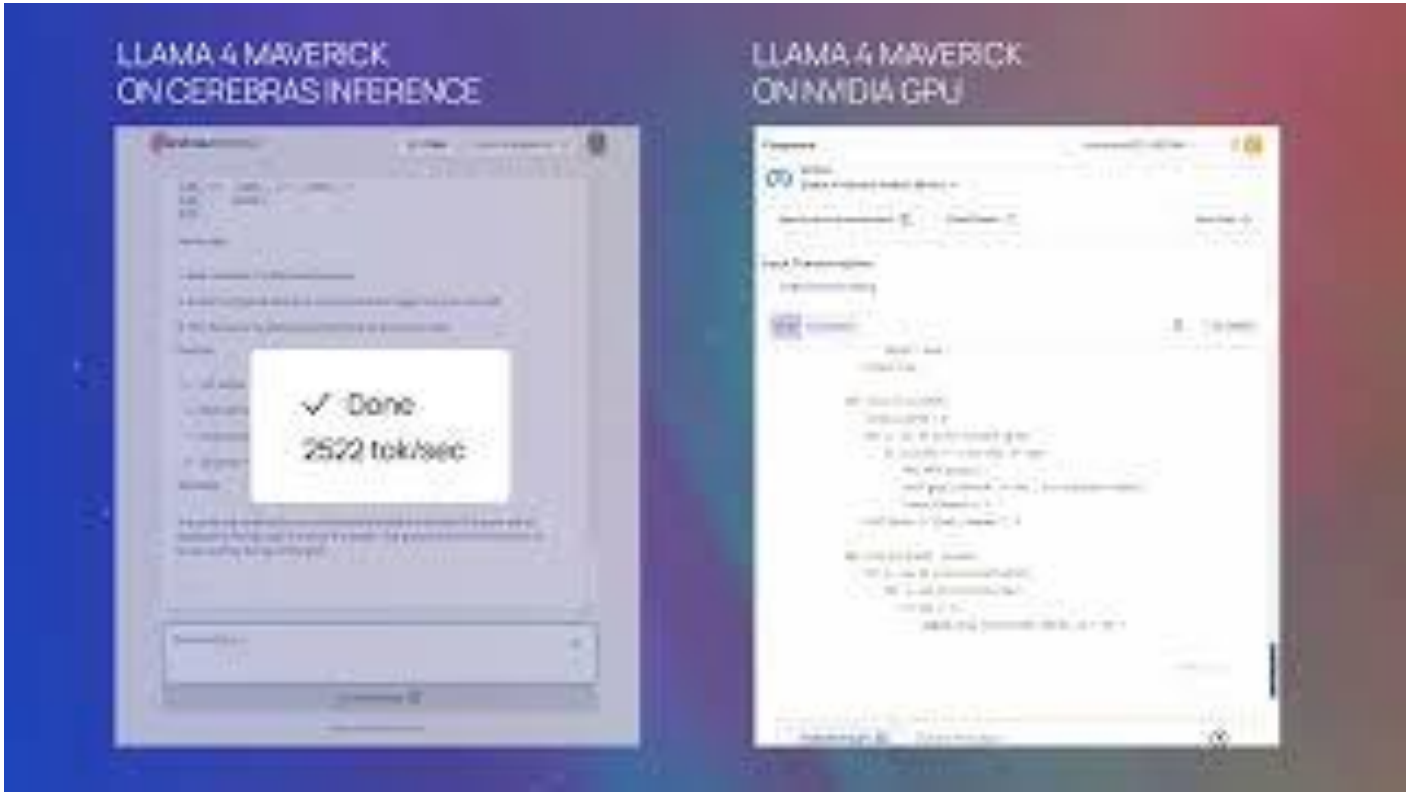
Managed Inference API Providers: Frontier closed models

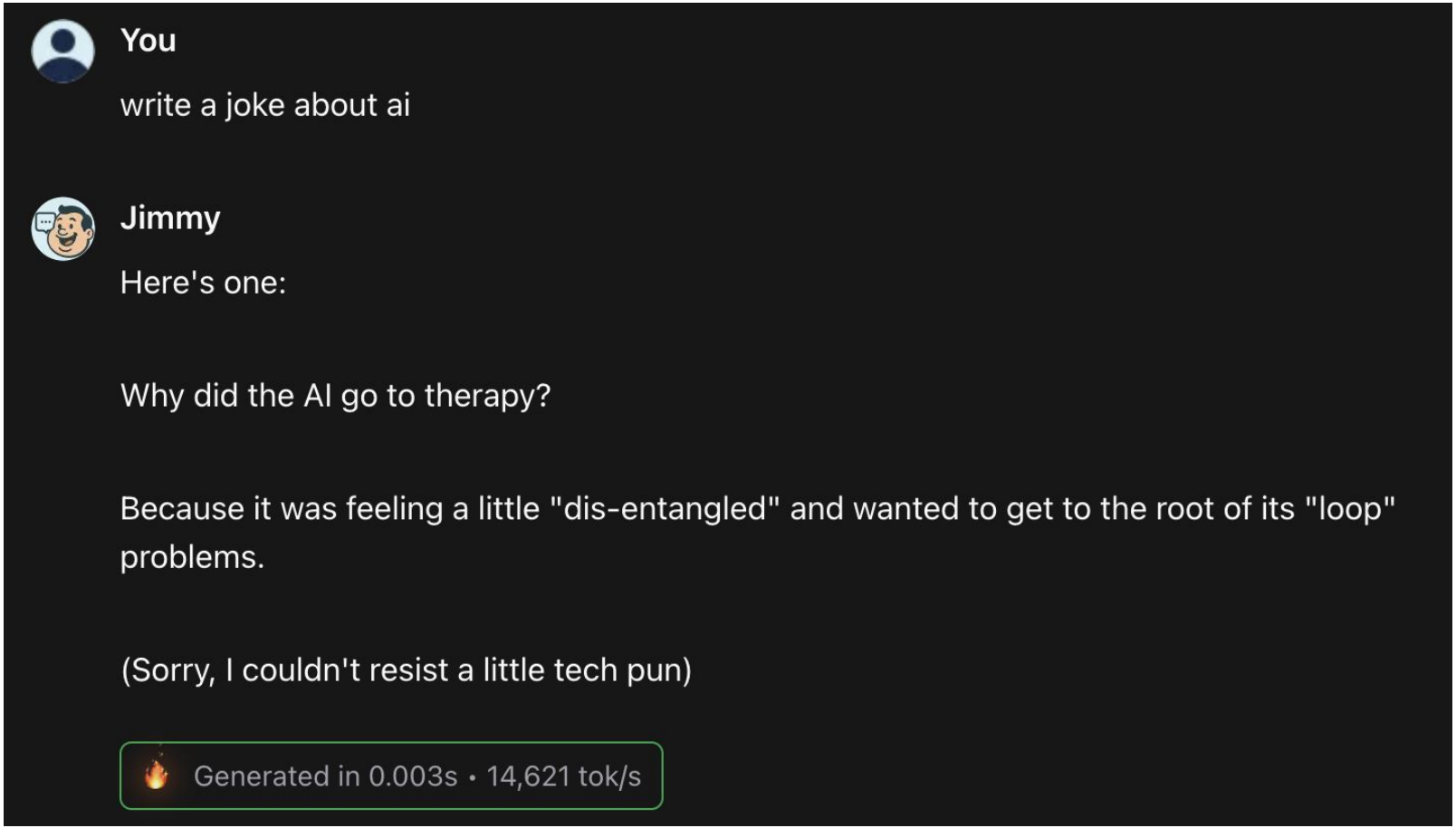
- OpenAI, Anthropic, Google
- You're paying for the model itself, which you can't run anywhere else
- Inference engine is a black box
- Pricing is per token, with caching and batch / priority tiers

Managed Inference API Providers: Open-weight hosts

- Baseten, Together, DeepInfra, Nebius, ...
- Host open-weight or fine-tuned models
- Shared vs Dedicated Endpoints
- They manage serving infrastructure: GPUs, scaling, routing, monitoring, engine/runtime choices
- Under the hood may use vLLM / SGLang / TensorRT-LLM, or their own inference stack
- You’re paying for operational simplicity

Managed Inference API Providers: Custom silicon





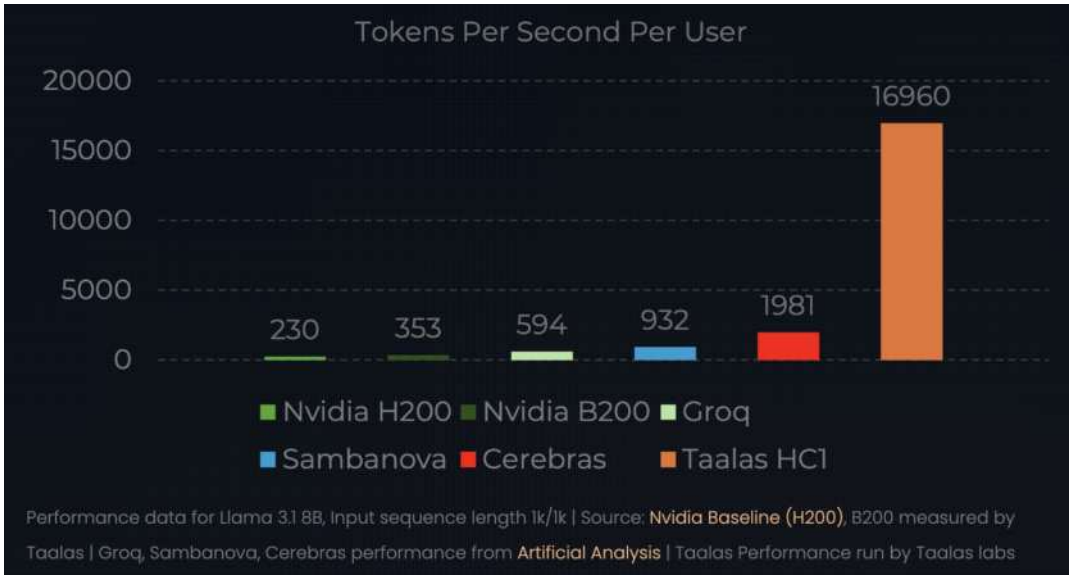
https://chatjimmy.ai

Custom silicon providers

- Groq, Cerebras, SambaNova, Etched, ...
- The bet: LLM inference (especially memory-bound decode) may need a different chip shape than GPUs
- Often great for low-latency token generation / interactive workload
- Trade-off: narrower model support, less flexibility, vendor lock-in

Taalas: model-specific silicon

- Unlike GPUs or more general AI accelerators, Taalas specializes the silicon around one model architecture
- Main idea: merge storage + compute → avoid HBM bottlenecks
- Trade-off: much less flexibility (LoRA still supported though)



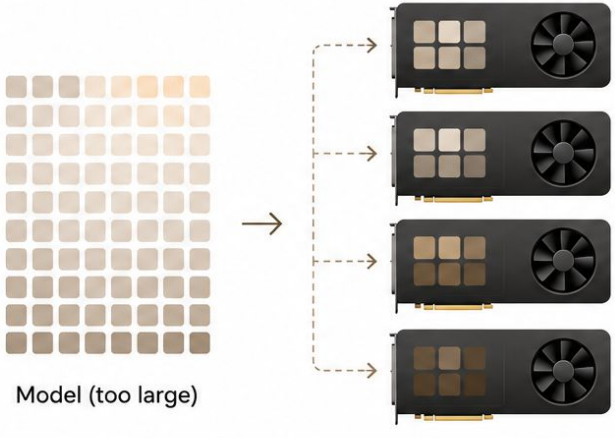
https://taalas.com/the-path-to-ubiquitous-ai/

Multi-GPU Inference

When one GPU is not enough

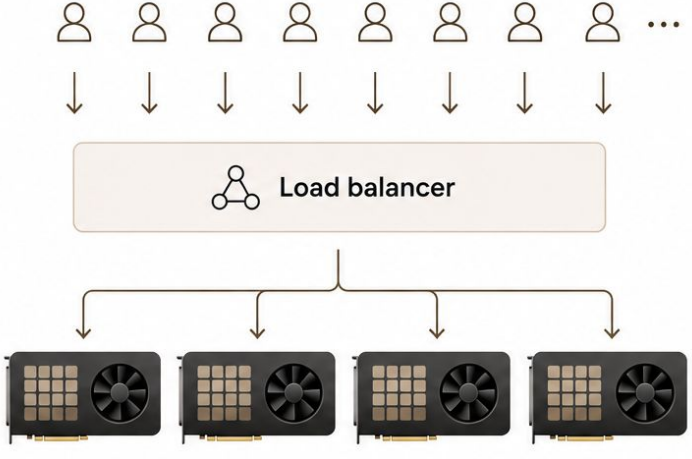
Two reasons to add GPUs

1 Model is bigger than one GPU



Model is partitioned across GPUs so it fits.

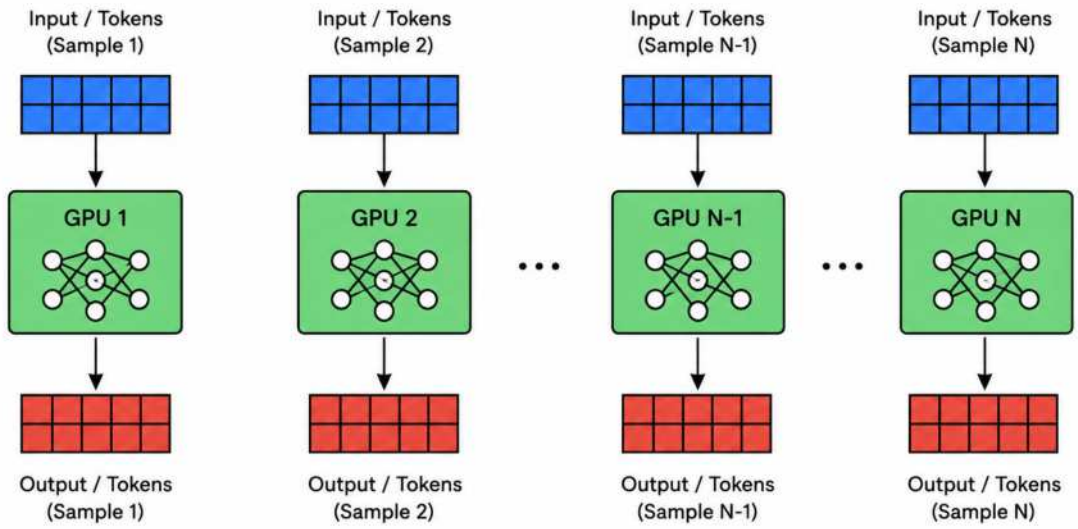
2 Traffic is bigger than one GPU



Model is replicated across GPUs to handle more requests.

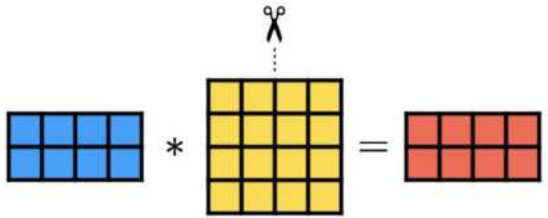
Data Parallelism: Replicate The Model

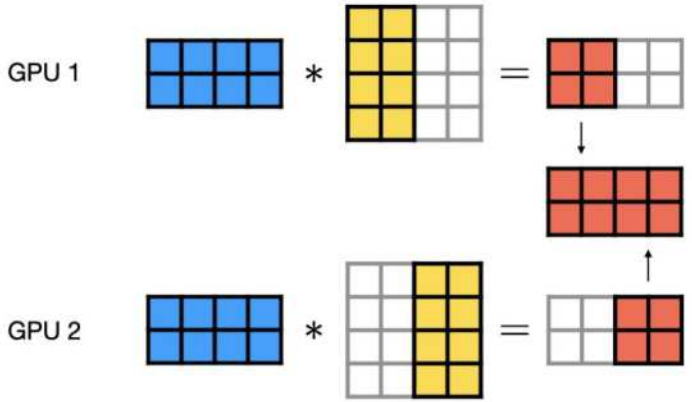
- Each GPU / node runs a full copy of the model
- A load balancer sends different requests to different replicas
- Best way to scale traffic, not model size
- **KV Cache caveat**



Tensor Parallelism: Cut Inside Layers

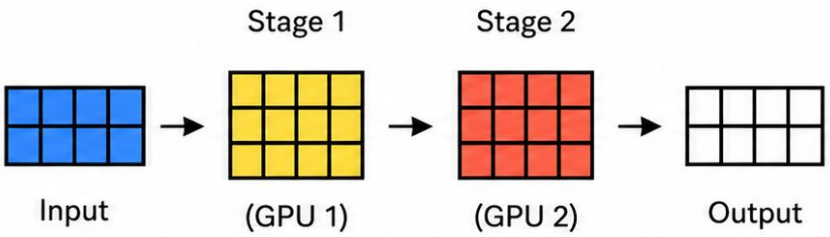
- Split large matrix multiplications across GPUs
- Every layer runs on every GPU
- Requires frequent communication during each forward pass

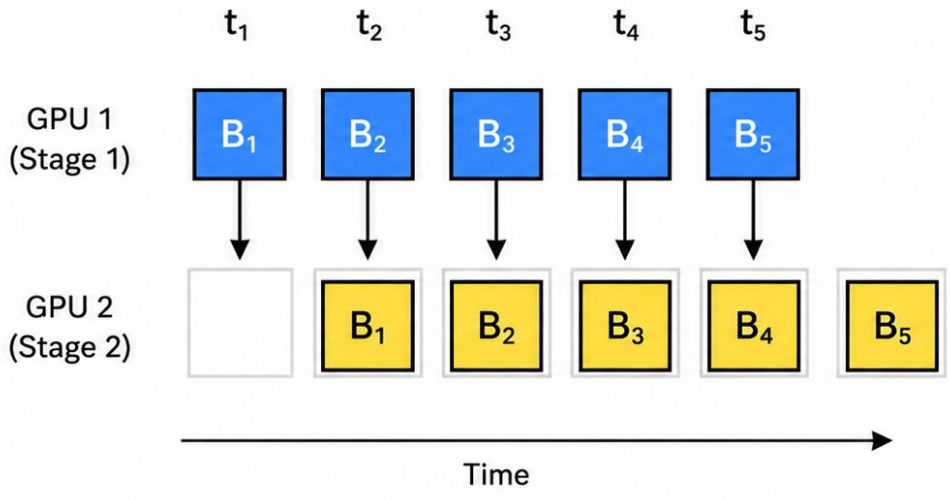




Pipeline Parallelism: Cut Between Layers

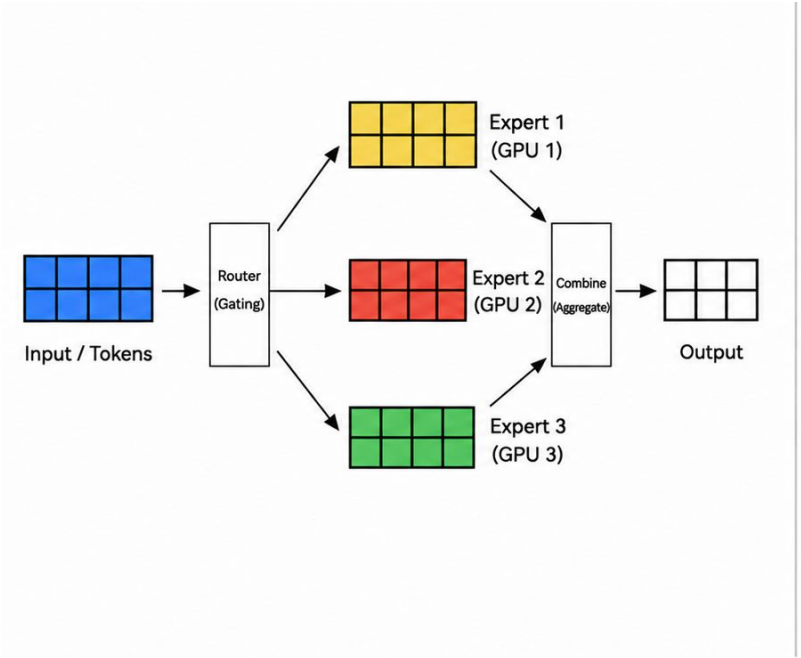
- Each GPU owns a consecutive block of layers
- Activations flow stage to stage
- The hand-off is tiny: one (batch · seq · hidden) tensor per boundary

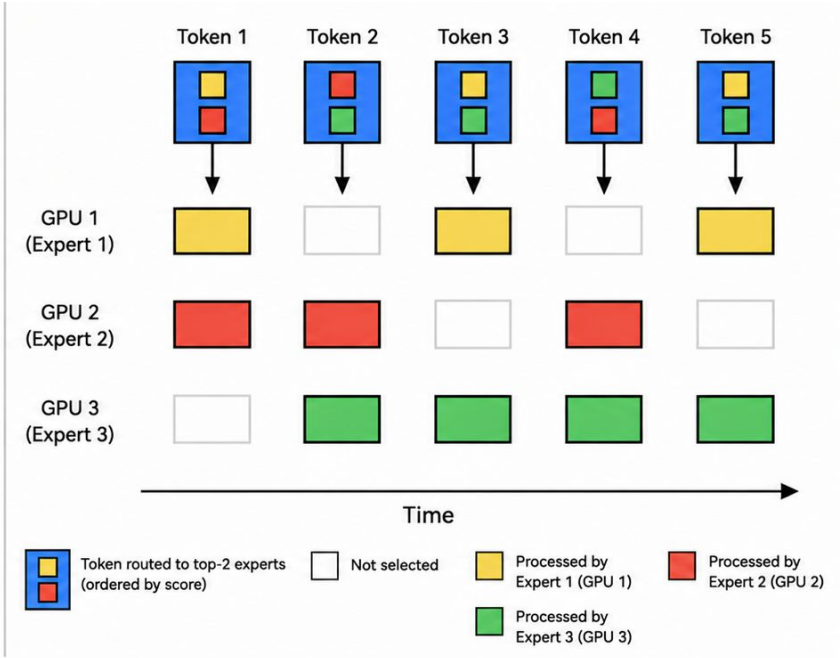




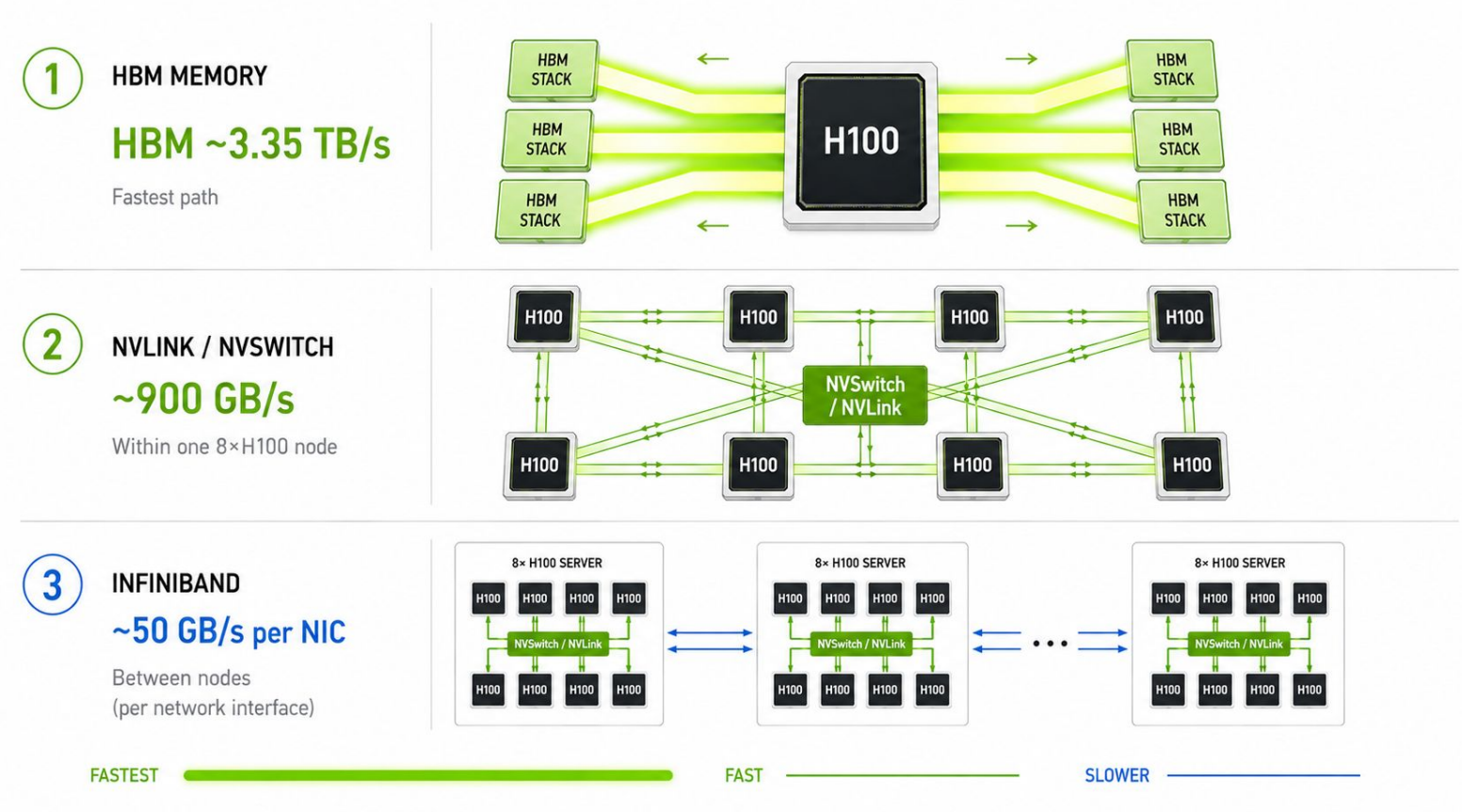
Expert Parallelism: Split The Experts

- MoE models have many experts, but each token uses only a few
- EP places different full experts on different GPUs

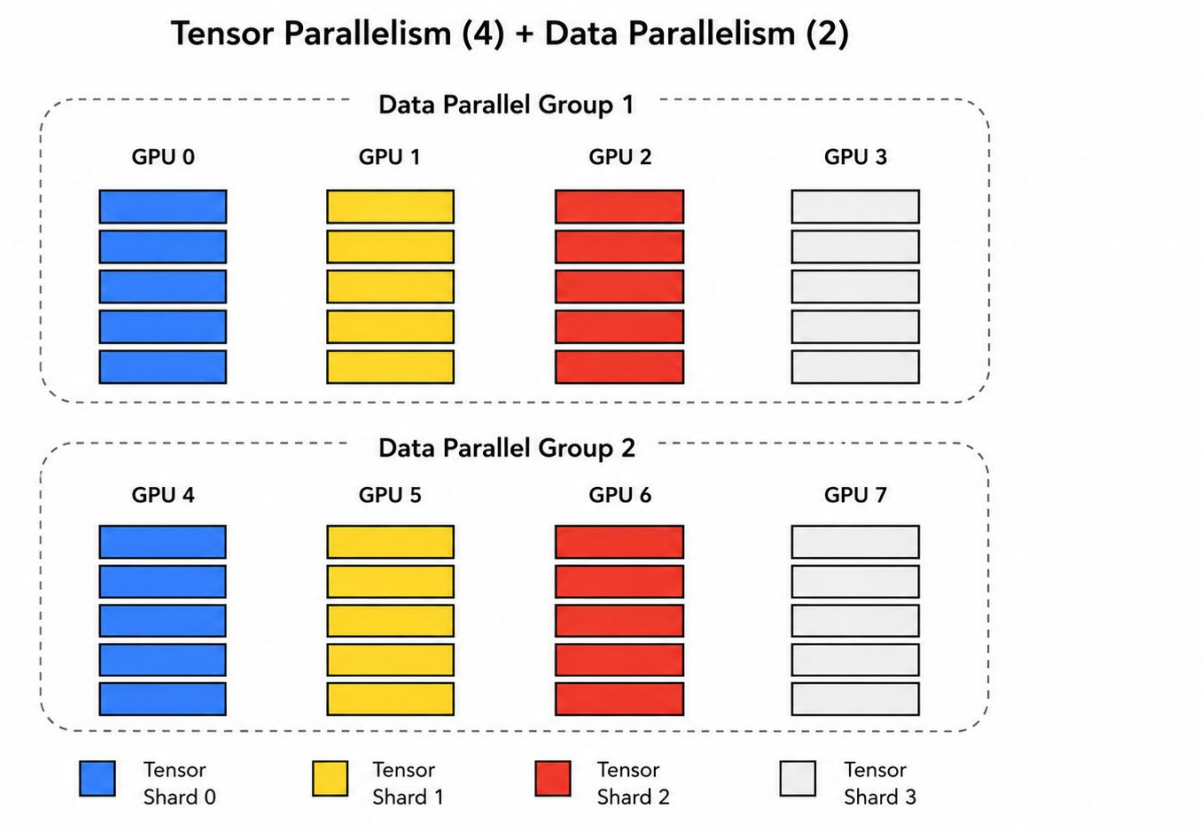




Reminder: Interconnect



Hybrid Parallelism



Model Parallelism Strategies at a Glance

How DP, TP, PP, and EP split work, communicate, and when to use them

Strategy	What is split?	Communication	Best suited for
DP Data Parallelism	Requests across full model replicas	No GPU-to-GPU communication per request	Traffic-limited serving: more RPS / throughput
TP Tensor Parallelism	Matrix multiplications inside layers	Collective ops over activations inside every transformer layer, typically after attention and MLP outputs	Large dense models, low TTFT / TPOT, fast intra-node GPUs
PP Pipeline Parallelism	Consecutive blocks of layers	Activation tensors passed stage-to-stage	Very large models, especially when TP across nodes is too expensive
EP Expert Parallelism (MoE)	MoE experts	Token dispatch + combine, usually all-to-all	Wide MoE models with many / large experts

Disaggregated Prefill-Decode

From model parallelism to stage separation

So far, we asked:

How do we split one model across multiple GPUs?

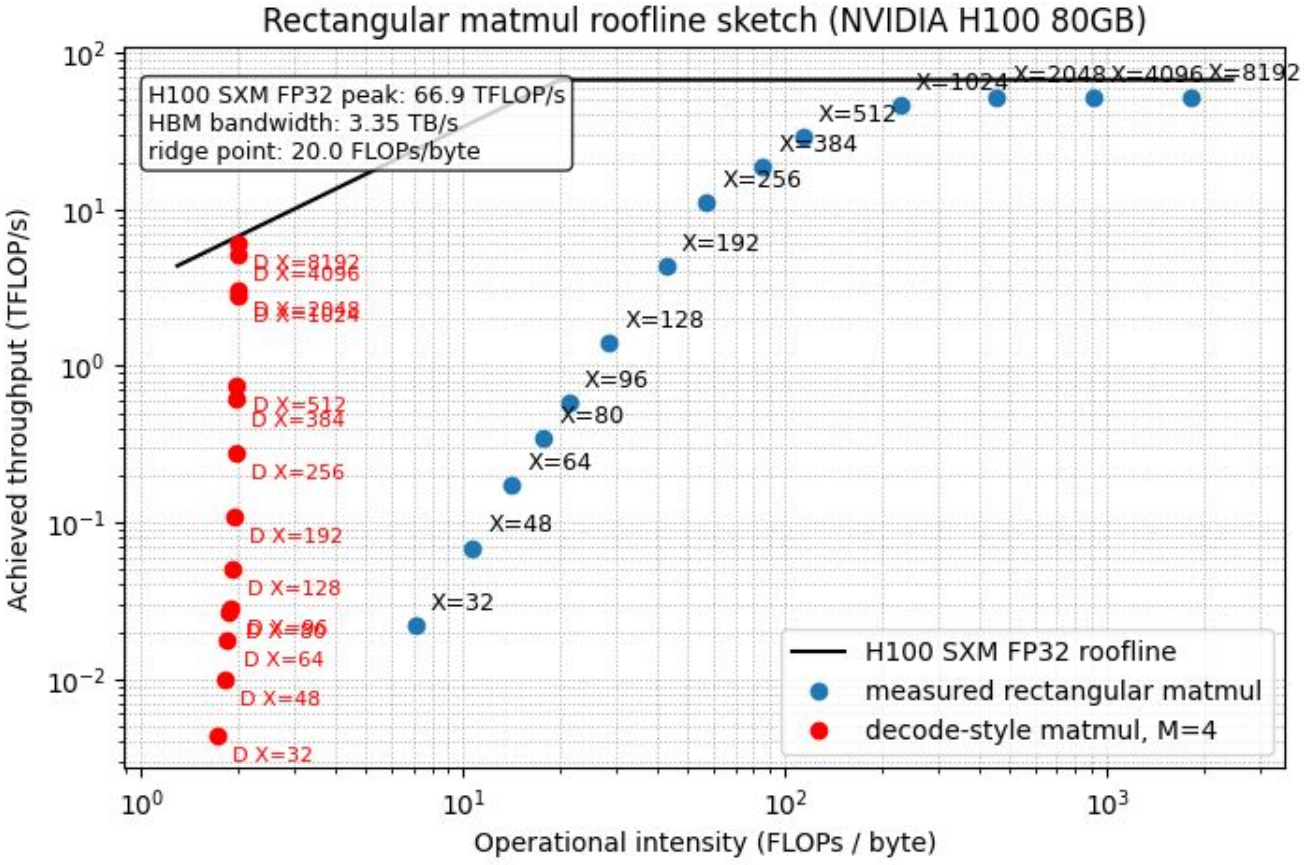
But inference is not one uniform workload:

- Prefill wants fast prompt processing → low TTFT
- Decode wants steady token streaming → low TPOT

Feel the difference Again

Decode → Low Arithmetic Intensity → Memory Bound

Prefill → High Arithmetic Intensity → Compute Bound



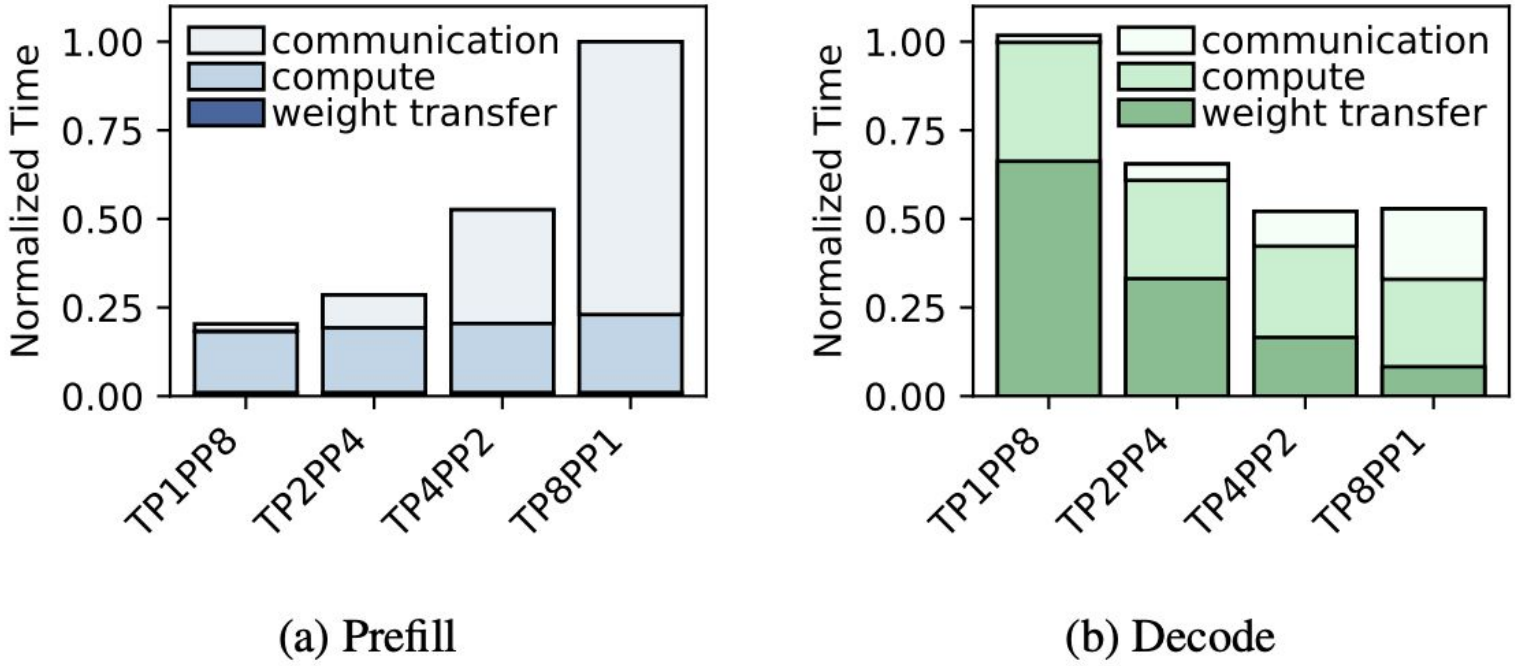


Figure 1. Breakdown of execution time for the prefill and decode stages for LLaMA2-13B inference on 8 L4 GPUs (The global batch size is 16. Pipeline parallelism further divides the data into micro-batches of size 16/PP to fully utilize pipelining).

Disaggregated Prefill-Decode

The problem:

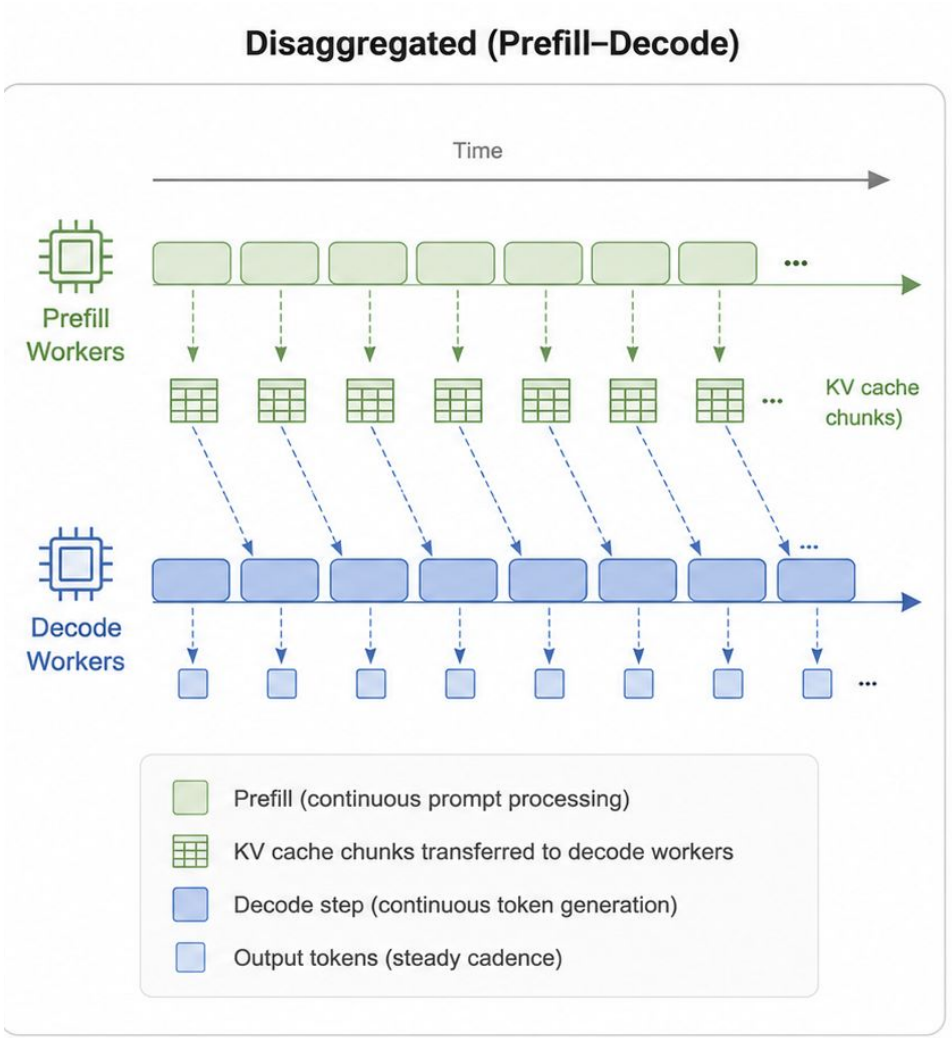
- Prefill is compute-heavy and can occupy the GPU for a long time
- Decode needs steady small steps to keep tokens streaming
- In a unified engine, long prefills can interrupt decode and create TPOT spikes

The idea:

- Run prefill and decode on separate workers
- Prefill workers build the KV cache
- Decode workers receive the KV cache and generate tokens without being interrupted

Request flow

1. Router receives the request
2. Prefill worker processes the prompt
3. KV cache is transferred to a decode worker
4. Decode worker skips prompt prefill and starts generating tokens



When is PD disaggregation worth it?

Good fit:

- Online serving with strict TTFT / TPOT targets
- Long prompts that interrupt decode

It's **not** the first thing to try:

- Not a throughput optimization
- More complex routing and KV transfer
- Duplicates model weights across pools
- Still early / experimental in vLLM and SGLang

Use chunked prefill first; consider PD disaggregation when decode latency isolation becomes the bottleneck.

Takeaways

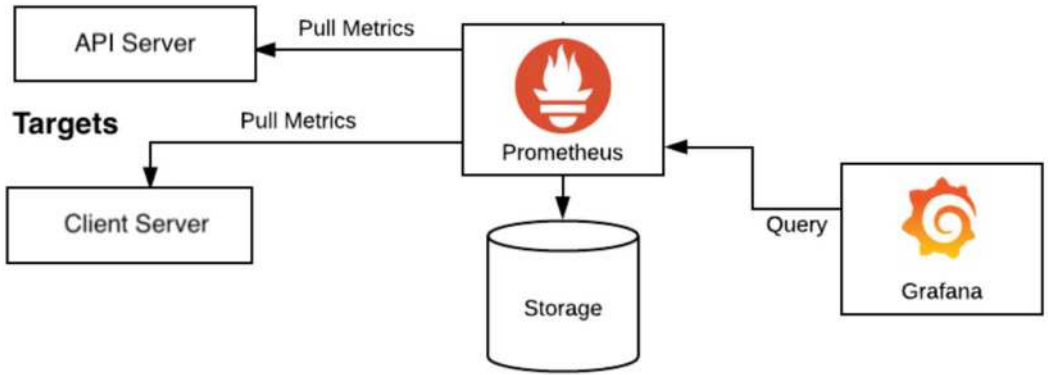
- TP / PP / DP / EP answer: How do we run the model across GPUs?
- PD disaggregation answers: Should prefill and decode run on separate worker pools?
- They compose: Each prefill/decode pool can still use TP, PP, DP, or EP internally.
- Start simple: Use the smallest TP that fits, add DP for traffic, use PP / EP when needed, and consider PD only when decode latency isolation becomes the bottleneck.

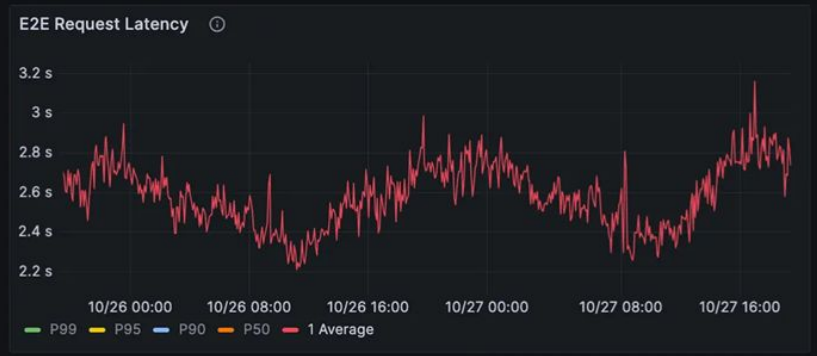
Monitoring & Observability

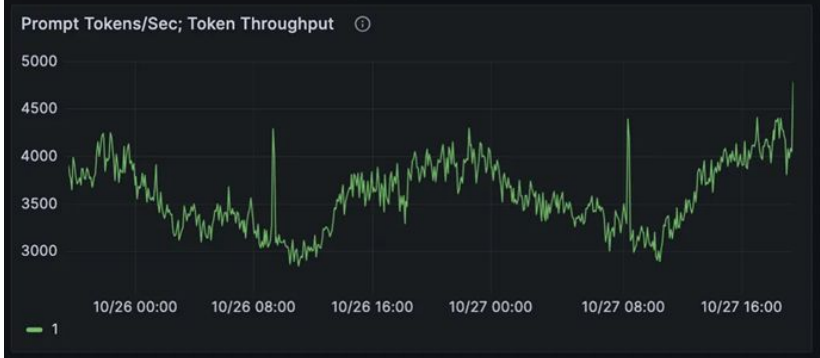
"If it's not on a dashboard, it's not in production”

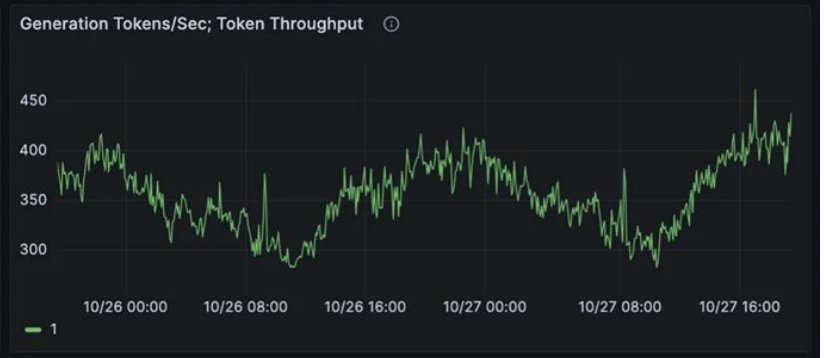
The Standard Stack: Prometheus + Grafana

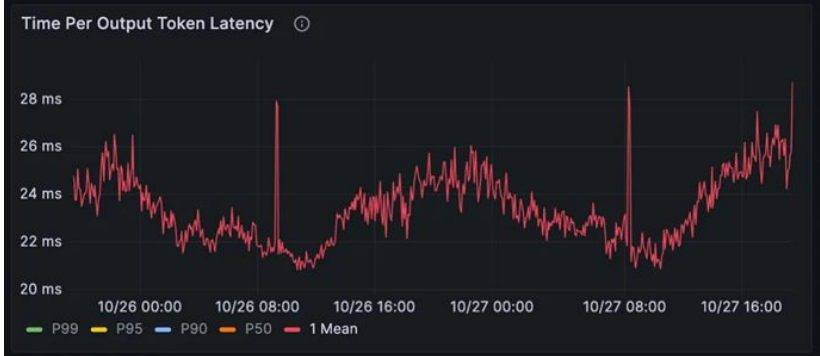
- Prometheus: time-series database. Pulls metrics from /metrics endpoints (exporters) on a schedule.
- Grafana: dashboard / alerting layer. Queries Prometheus with PromQL.
- Every serious serving engine exposes a /metrics endpoint out of the box

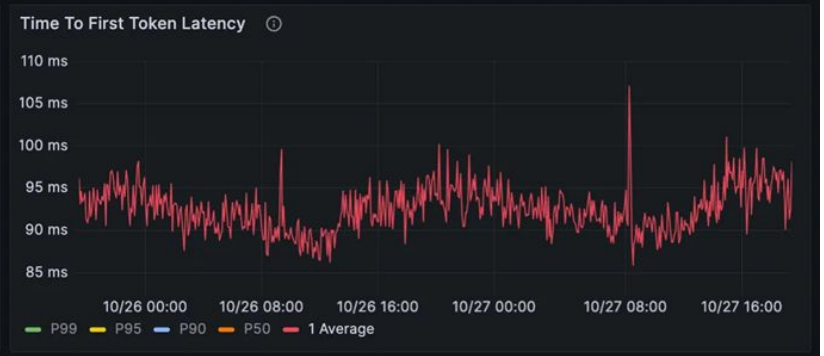



E2E Request Latency
3.2 s
3 s
2.8 s
2.6 s
2.4 s
2.2 s
10/26 00:00
10/26 08:00
10/26 16:00
10/27 00:00
10/27 08:00
10/27 16:00
P99
P95
P90
P50
1 Average


Prompt Tokens/Sec; Token Throughput
5000
4500
4000
3500
3000
10/26 00:00
10/26 08:00
10/26 16:00
10/27 00:00
10/27 08:00
10/27 16:00
1


Generation Tokens/Sec; Token Throughput
450
400
350
300
10/26 00:00
10/26 08:00
10/26 16:00
10/27 00:00
10/27 08:00
10/27 16:00
1


Time Per Output Token Latency
28 ms
26 ms
24 ms
22 ms
20 ms
10/26 00:00
10/26 08:00
10/26 16:00
10/27 00:00
10/27 08:00
10/27 16:00
P99
P95
P90
P50
1 Mean


Time To First Token Latency
110 ms
105 ms
100 ms
95 ms
90 ms
85 ms
10/26 00:00
10/26 08:00
10/26 16:00
10/27 00:00
10/27 08:00
10/27 16:00
P99
P95
P90
P50
1 Average

What to remember from this lecture

- LLM inference is stateful and autoregressive: every new token carries a growing KV cache.
- Prefill is mostly compute-bound, while decode is mostly memory-bandwidth-bound.
- Most engine optimizations are about three things: use HBM efficiently, keep the GPU busy, and protect token latency.
- KV-cache management is central: paged attention, prefix caching, eviction, routing, and distributed cache all optimize around it..
- Start simple in production
- There is no universally best stack: benchmark your workload, because model type, prompt reuse, latency target, and operational constraints change the answer.